Master's thesis

Machine learning surrogate modelling and model predictive control of industrial graphite furnace

Abhyuday Pandey

Master's in Aerospace Engineering
30 ECTS study points

Aerospace systems and control

Autumn 2026

Abhyuday Pandey

Machine learning surrogate modelling and model predictive control of industrial graphite furnace

Supervisor:
Sondre Tesdal Galtung

With thanks to:
Sondre for the constant guidance and encouragement; Manuel at Vianode and Ulrik at Norse for providing additional support; and all the kind people in the AAI group for welcoming me.

Conducted at:
SINTEF, Mathematics and Cybernetics, Oslo

LaTeX template: [25]
Front page image: [35]

"[...] home is that house where you no longer live."
— John Green

Contents

Chapter 1

Introduction

1.1 Introduction

As scientific computing tools have gotten increasingly more sophisticated and compute has gotten cheaper and more accessible, ever larger and more complex models are made and used in the industry to inform the design and operation of physical systems. These models are often extremely computationally demanding, owing to their high-dimensionality and complex nonlinear dynamics. As a result, our models have somewhat outgrown our means. This has brought reduced-order modelling to the forefront, as a means to run (or approximate) these models on lower-end hardware, or faster. Machine learning surrogate modelling has risen to prominence in the reduced-order modelling scene due to its relative ease of application, ability to deal with black-box models given its data-driven nature, and potential for hyper-reduction [34]. In [34] Tomasetto et al apply a novel machine learning model, the SHallow REcurrent Decoder or SHRED, to reduced-order modelling of a multitude of fluid-dynamics simulations with impressive results.

In this thesis, this SHRED model is applied for reduced-order modelling of Vianode's graphite furnace. Their synthetic graphite process is sensitive to temperature, and they have a finite-element model to compute the temperature field inside the furnace. This model is quite large, meaning it requires special hardware to run, and even then cannot run in real-time. A reduced-order model is to be made which can well approximate the finite-element model while being lightweight and fast. This model runs fast enough that it may be used with a numerical optimiser to compute an optimal control trajectory using a model predictive controller to maintain the temperature in Vianode's desired range.

This report, which serves as the final thesis report, details most of the research and work done over the four-month thesis internship period. It is a somewhat abridged summary to preserve narrative structure. Many lines of inquiry that led to dead ends have been omitted. While all fruitful results and output are included, they are far outnumbered by the many learnings that are not.

Part I is a literature review, recapping all theory relevant to understanding the problem and the proposed solution. It is broken up into three chapters: machine learning surrogate modelling, optimal control, and model predictive control. A reader familiar with the basics of machine learning is encouraged to skip ahead to Section 2.5 in Chapter 2. Readers familiar with optimisation theory and model predictive control may

skip Chapter 3 altogether, and start from 4.4.

In Part II, a state-estimator and a state-forecaster surrogate model are trained. This surrogate model is then connected to a model predictive control scheme, and two different types of model predictive control (QIH-NMPC and EMPC) are implemented, with special focus on provable stability. The performance of the surrogate models and the controllers is analysed, and some graphs of trajectories produced are presented. Part III summarises the findings, outlines areas for further work, and concludes.

1.2 AI disclosure

All text in this report is written entirely by the author without the use of any LLMs. Claude was used as a search engine to help surface relevant research papers and articles, but not to summarise or replace the articles. All sources cited were read and parsed by the author, not synthesised or summarised by LLMs.

Most code produced is also written by the author. This is especially true for the code central to the learnings of this thesis (the MPC algorithms, the ML models). Claude Code was used extensively for debugging human-written code. The code to create the plots was often generated partly and rarely in whole using Claude Code.

Part I

Theory

Chapter 2

Machine learning surrogate modelling

To start from the very foundational theory of machine learning in this paper would be like explaining computers each time one uses them for numerical simulations. Machine learning is widely used, and it is assumed for the remainder of this paper that the reader is familiar with the basic concept. This section contains a very abridged summary of the basics. In the next sections, we will delve into the theory of the specific models relevant to this paper.

A *neural network* (NN) consists of a series of affine transformation layers (weight matrix multiplication and bias vector addition) and nonlinear activation layers. Each scalar affine transformation followed by nonlinear activation is called a *neuron*. A set of affine transformation and element-wise activations is called a *layer* - essentially a stack of neurons. The activation layers are key to the performance of NN as it gives it the ability to capture nonlinearities. Many such layers may be stack together with different (but intercompatible) sizes to form a complete neural network.

Training an NN involves finding the best weights and biases in each layer relative to some metric to be optimised - the *loss function*. In supervised learning, the loss is calculated as some error metric between the NN's output and the ground truth for each input, the exact choice of loss function is highly dependent on the application. The input-output pairs in the *training data* are iterated over multiple times, each iteration is called an *epoch*. To avoid overfitting, the performance of each epoch is measured as the loss, not over the training data called the *training loss*, but over another dataset called the *validation dataset*, called the *validation loss*. Training stops when some stopping criteria are met, usually when the maximum number of epochs is reached or *patience* is exceeded. Patience is how many epoch without improvement to the loss over the validation dataset relative to the lowest yet in training are permitted, this stops the training early if further training is only decreasing training loss without improving validation loss (textboook overfitting). Optimisation techniques are used to find the weights that minimise the loss. Optimisation, in general, requires the gradient of the objective function, which is found in NNs by error back-propagation (or simply back-prop). Back-prop is just a structured implementation of the chain rule to find derivatives of the loss with respect to each weight. The typical workflow for mini-batch gradient descent is as follows [23]:

1. the NN's output is computed for some subset (a batch) of the training data and the loss is computed for these outputs (*forward pass*);

2. the gradient of the loss is computed with back-prop (*backward pass*);

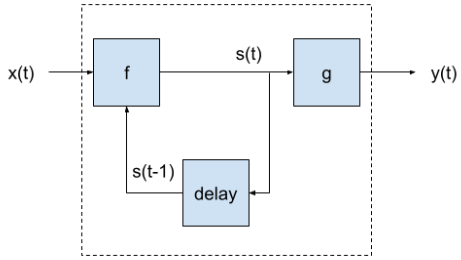


Figure 2.1: State machine

3. a numerical optimiser uses the gradient to take a step (updating the weights);
4. after each epoch, compute the validation loss. Update the *best-state* to the current state and reset patience to zero if validation loss is better, else update patience by 1. *epoch*, this is repeated until some stopping condition is met (patience, maximum epoch count, etc).
5. When the stopping condition is met, revert the model state (weights) to the best-state.

Many regularisation techniques are available that may be applied between layers during training to avoid over-fitting and related issues. We highlight one technique - *dropout* - as it is used in our application. Dropout randomly switches the outputs of the previous activation layer to zero with probability p during training, such that the NN cannot become over-reliant on any particular unit, but learns to generalise [23]. Dropout becomes inactive after training (in `PyTorch`, when `.eval()` is called it becomes inactive).

2.1 Recurrent neural networks (RNNs)

In a traditional feedforward neural network, each input-output pair is computed independently from other pairs. This is analogous to the concept of a *state function* in thermodynamics - the next state depends only on the current state, not the path taken to get there. However, in applications concerning sequential data such as natural language processing and time series data, it is advantageous to also take into account the previous states (called the context window). State machines are well suited to such sequential problems.

A state machine consists of sets $\mathcal{S}$ of states, $\mathcal{X}$ of inputs, $\mathcal{Y}$ of outputs as well as mappings f the transition function $\mathcal{X} \times \mathcal{S} \to \mathcal{S}$ and g the output function $\mathcal{S} \to \mathcal{Y}$. The state in a state machine allows for the output to depend on previous inputs (through the previous state) will makes them well suited for sequential problems. *Recurrent neural networks* (RNNs) are state machines whose transfer and output functions are feedforward NNs, so by feeding back the *hidden state*, of each *recurrent cell* as an input for the next element in the sequence information is captured about the whole sequence, serving as a kind of short-term memory for the network [26]. The sequences can in general be of arbitrary length, and do not have be to be of the same length within a training or testing set. The output can be a sequence (sequence-to-sequence RNN) or just the final output at the end (many-to-one RNN) [26].

Dynamical systems such as the one at the heart of this paper can be very sensitive to initial conditions, and estimating the spatio-temporal dynamics accurately is challenging

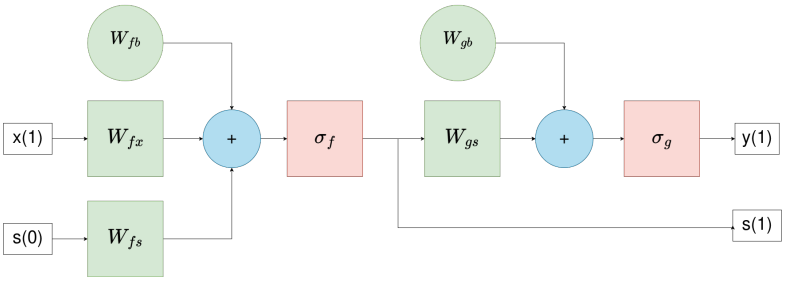


Figure 2.2: Recursive unit

in general due to the butterfly effect, a small difference in initial conditions may lead to a large deviation of state in the future. For a model that has to learn to predict time series data, the RNN is a natural machine-learning model to turn to as it makes the time dependence explicit. They mirror the structure of a dynamical system as, like dynamical systems, they calculate future (hidden) states by running previous (hidden) states through the same function (the RNN cell). Furthermore, the real full state of a physical problem is not always accessible. The full state of a thermal system would be the continuous temperature field in the medium, whereas even with the best numerical solvers the field values are estimated with known temperatures at nodes and interpolation with shape functions. This further makes the dynamics difficult to learn for a state-function-like NN. RNNs can take advantage of their recursive nature to use a sequence of previous states to help learn the temporal dynamics explicitly. A recursive unit is depicted in Figure 2.2, where the weight matrices W_{fs} and W_{fx} act on the previous cell-state $s(t-1)$ and current input $x(t)$. These are added together with a bias vector W_{fb} and the sum passes through activation σ_f to give the new cell-state $s(t)$. The new cell-state multiplies another weight matrix W_{gs}, sums with another bias vector W_{gb}, and passes through the activation σ_g to give the output.

Computing the gradient of the loss for RNNs is a little more complicated as the same weights are applied many times to get to the output. A modified back-propagation method called *back-propagation-through-time* or BPTT is used to train RNNs. The forward pass can be visualised by *unrolling* the RNN (2.3). The many-to-one output is simply $y(N)$ and the loss is computed as $\text{Loss}\,(y(N), \tilde{y}(N))$ where $\tilde{y}$ is the ground truth. The sequence-to-sequence output would be the complete output sequence $(y_1, ..., y_N)$ and the loss would be the sum of the loss of each element of the sequence. The derivative of the many-to-one output loss with respect to the output later weight and bias is straightforward as they appear once and at the very end. Finding the derivative of the loss with respect to W_{fs}, W_{fx}, W_{fb} is not as straightforward as it appears multiple times in the graph. Applying chain rule for the many-to-one loss with respect to W_{fs} is

$$\frac{\partial \text{Loss}}{\partial W_{fs}} = \sum_{i=0}^{N} \frac{\partial \text{Loss}}{\partial s_i} \frac{\partial s_i}{\partial W_{fs}},$$

where

$$\begin{aligned} s_i &= \sigma_f(W_{fs} s_{i-1} + W_{fx} x_{i-1} + W_{fb}) \\ \implies &\begin{cases} \dfrac{\partial s_i}{\partial s_{i-1}} = \sigma_f'(W_{fs} s_{i-1} + W_{fx} x_{i-1} + W_{fb}) W_{fs}^T \\ \dfrac{\partial s_i}{\partial W_{fs}} = \sigma_f'(W_{fs} s_{i-1} + W_{fx} x_{i-1} + W_{fb}) s_{i-1} \end{cases} \end{aligned}$$

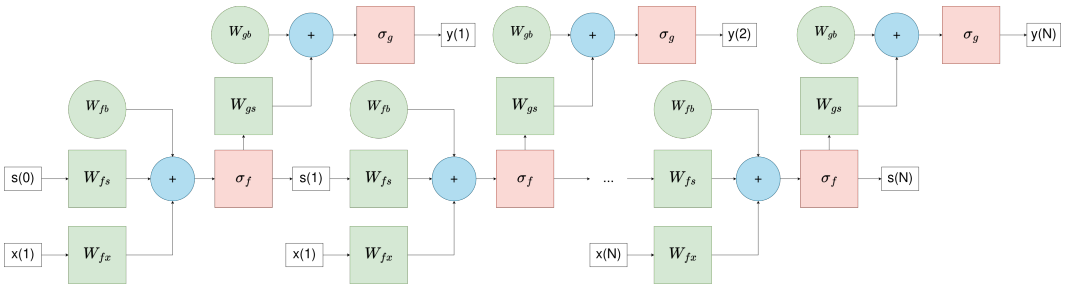


Figure 2.3: Unrolled RNN

Define

$$\delta_i = \frac{\partial \text{Loss}}{\partial s_i}.$$

Hence,

$$\delta_{i+1} = \frac{\partial \text{Loss}}{\partial s_{i+1}} \frac{\partial s_{i+1}}{\partial s_i} = \delta_i \sigma_f'(W_{fs} s_{i-1} + W_{fx} x_{i-1} + W_{fb}) W_{fs}^T \tag{2.1}$$

so δ can be defined recursively. Hence, the derivative of the loss with respect to W_{fs} is

$$\frac{\partial \text{Loss}}{\partial W_{fs}} = \sum_0^N \delta_i \frac{\partial \text{Loss}}{\partial s_i} \frac{\partial s_i}{\partial W_{fs}}.$$

The others follow similarly. Note that, due to the recursivity of δ, there is repeated multiplication by W_{fs} in the derivative, which depending on the magnitude of the norm of W_{fs} will either grow or shrink exponentially. This is generally called the "vanishing gradient problem" and simple RNNs (called simple recurrent networks or SRNs, as depicted in 2.2) are especially prone, which can make them very difficult to train [10][14].

One solution to vanishing gradient problem is batch normalisation between layers, whereby the output from one layer is normalised and re-centred around zero before being sent to the next layer [16]. A better structural solution for RNNs is to use the concept of *gating* - meaning having a gate decide how much of the state is actually updated at each recursive pass. The most prevalent gated RNN is the *Long Short-Term Memory* (LSTM) model, which is the focus of the next section.

2.2 Long short-term memory (LSTM)

As mentioned earlier recursive model's hidden state can be thought of as its short-term memory: what it should remember from the previous inputs. By extending the analogy the weights can be thought of as its long-term memory: what has it learned from all the training data. The longer the sequence the RNN has, the longer the short-term memory the better the performance might be, but the challenge is deciding what information to keep in short-term memory that will be important for future outputs. In a SRN, the hidden state (referred to as s in the previous section, but as h henceforth) had to contain both this information and the information useful for the current output. This along with the vanishing gradient problem motivated the LSTM, so called as its structure allows for longer short-term memory without the same problems as SRNs.

First introduced in 1997 by Hochreiter and Schmidhuber[14] (though building upon by Hochreiter's 1991 thesis on gradient flow problems in RNNs [19][1]), LSTMs would take their full "vanilla" form in Graves and Schmidhuber's 2005 paper in Neural Networks [9](where the derived the full gradient of the LSTM allowing BPTT to be used for training). The big innovation was the introduction of gates that decide how much of the new information to pass through.

The vanilla LSTM has two recurrent states, the *hidden state* h which also serves as the output at each timestep, and the *cell-state* c which serves as the short-term memory. The states can only be interacted with through the three gates: the input gate, the output gate, and the forget gate. The *input gate* is responsible for deciding how much of the new input should inform the new cell-state, the *output gate* is responsible for deciding how much of the new cell-state should inform the new hidden state (which is also the output), and the *forget state* determines how much of the old cell-state should inform the new call state.

Starting with the new input x_t and previous hidden state h_{t-1}, a standard neural layer (weights and activation) computes the candidate cell-state $\tilde{c}_t$. The input i_t, forget f_t, and output o_t gates are computed similarly but with necessarily a sigmoid activation. The new cell-state c_t is given by adding the Hadamard product between the input gate i_t and the candidate cell-state $\tilde{c}_t$ and the Hadamard product between the forget gate f_t and the previous cell-state c_{t-1}. The new hidden state (output) h_t is given by the Hadamard product between the output gate and the activation of the new cell-state. The full equation are as follows:

$$\begin{aligned} i_t &= \sigma(W_{ix}x_t + W_{ih}h_{t-1} + b_i) \\ f_t &= \sigma(W_{fx}x_t + W_{fh}h_{t-1} + b_f) \\ o_t &= \sigma(W_{ox}x_t + W_{oh}h_{t-1} + b_o) \\ \tilde{c}_t &= \tanh(W_{cx}x_t + W_{ch}h_{t-1} + b_c) \\ c_t &= i_t \odot \tilde{c}_t + f_t \odot c_{t-1} \\ h_t &= o_t \odot \tanh(c_t) \end{aligned} \tag{2.2}$$

The important part that addresses the vanishing gradient problem is (2.2), where unlike the multiplicative updates in the SRN, the updates are almost additive, and the weights do not recursively multiply in the gradient $\frac{\partial c_t}{\partial c_{t-1}} = f_t$ unlike in SRN ((2.1)). Additionally, since the cell-state is now separate from the hidden state, it can be forced to reset if desired (for example the problem conditions change significantly).

2.3 Derivatives and bounds of LSTM

For later sections of this thesis, we must find the first and second order derivatives of the LSTM, and their bounds. Let us start by defining some useful variables for the sake of cleaner notation

$$\begin{aligned} a_i &= W_{ix}x_t + W_{ih}h_{t-1} + b_i, & i &= \sigma(a_i), \\ a_f &= W_{fx}x_t + W_{fh}h_{t-1} + b_f, & f &= \sigma(a_f), \\ a_o &= W_{ox}x_t + W_{oh}h_{t-1} + b_o, & o &= \sigma(a_f), \\ a_c &= W_{cx}x_t + W_{ch}h_{t-1} + b_c, & \tilde{c} &= \tanh(a_t), \\ p &= f \odot c + i \odot \tilde{c}, & h &= o \odot \tanh(p). \end{aligned} \tag{2.3}$$

[1]The citation is to the 2002 English survey of the paper as the original is in German

The goal is to find the second derivative of h with respect to x. We follow back propagation to compute the first and second derivatives. Let us start by finding the simpler derivatives:

$$\begin{aligned}
\frac{\partial o}{\partial x} &= \sigma'(a_o)W_{ox} \\
\frac{\partial^2 o}{\partial x^2} &= \sigma''(a_o)W_{ox}^2 \\
\frac{\partial p}{\partial x} &= \sigma'(a_f)W_{fx} \odot c + \sigma'(a_i)W_{ix} \odot \tilde{c} + i \odot \operatorname{sech}^2(a_c)W_{cx} \\
\frac{\partial^2 p}{\partial x^2} &= \sigma''(a_f)W_{fx}^2 \odot c + \sigma''(a_i)W_{ix}^2 \odot \tilde{c} + \sigma'(a_i)W_{ix} \odot \operatorname{sech}^2(a_c)W_{cx} \\
&\quad + \sigma'(a_i)W_{ix} \odot \operatorname{sech}^2(a_c)W_{cx} - 2i \odot \tanh(a_c)\operatorname{sech}^2(a_c)W_{cx}^2.
\end{aligned}$$

The derivatives of h are then given as

$$\begin{aligned}
\frac{\partial h}{\partial x} &= \frac{\partial o}{\partial x} \odot \tanh p + o \odot \operatorname{sech}^2(p)\frac{\partial p}{\partial x} \\
\frac{\partial^2 h}{\partial x^2} &= \frac{\partial^2 o}{\partial x^2} \odot \tanh p + 2\frac{\partial o}{\partial x} \odot \operatorname{sech}^2(p)\frac{\partial p}{\partial x} + o \odot \operatorname{sech}^2(p)\frac{\partial^2 p}{\partial x^2} \\
&\quad - 2o \odot \tanh(p)\operatorname{sech}^2(p)\frac{\partial p}{\partial x}.
\end{aligned}$$

Let the all the nonlinear operations be contained in the activation function

$$\sigma_1(i, f, o, \tilde{c}, c) = o \odot \tanh(f \odot c + i \odot \tilde{c}),$$

then the derivatives of the activation layer are

$$\begin{aligned}
\frac{\partial \sigma_1}{\partial x} &= \sigma'(a_o)W_{ox} \odot \tanh\left(f \odot c + i \odot \tilde{c}\right) \\
&\quad + o \odot \operatorname{sech}^2(f \odot c + i \odot \tilde{c})\sigma'(a_f)W_{fx} \odot c \\
&\quad + \sigma'(a_i)W_{ix} \odot \tilde{c} + i \odot \operatorname{sech}^2(a_c)W_{cx}
\end{aligned} \tag{2.4}$$

$$\begin{aligned}
&\frac{\partial^2 \sigma_1}{\partial x^2} = \\
&\sigma''(a_o)W_{ox}^2 \odot \tanh(f \odot c + i \odot \tilde{c}) + 2\sigma'(a_o)W_{ox} \odot \operatorname{sech}^2(p)\sigma'(a_f)W_{fx} \odot c \\
&+ \sigma'(a_i)W_{ix} \odot \tilde{c} + i \odot \operatorname{sech}^2(a_c)W_{cx} + o \odot \operatorname{sech}^2(p)\sigma''(a_f)W_{fx}^2 \odot c \\
&+ \sigma''(a_i)W_{ix}^2 \odot \tilde{c} + \sigma'(a_i)W_{ix} \odot \operatorname{sech}^2(a_c)W_{cx} + \sigma'(a_i)W_{ix} \odot \operatorname{sech}^2(a_c)W_{cx} \\
&- 2o \odot \tanh(f \odot c + i \odot \tilde{c})\operatorname{sech}^2(f \odot c + i \odot \tilde{c})\sigma'(a_f)W_{fx} \odot c \\
&- 2i \odot \tanh(a_c)\operatorname{sech}^2(a_c)W_{cx}^2 + \sigma'(a_i)W_{ix} \odot \tilde{c} + i \odot \operatorname{sech}^2(a_c)W_{cx}.
\end{aligned} \tag{2.5}$$

Note that these expressions of the derivatives are slightly simplified and do not have strictly correct dimensions. However they are sufficient for finding the bounds numerically.

Next we use the method outlines in [32] and [31]to bound the derivatives. To be consistent with their method, we break up the LSTM into a weight layer and an activation layer. For generalisability we consider a final weight layer on the output of the LSTM, this can be identity if there is no projection after the LSTM. The activation

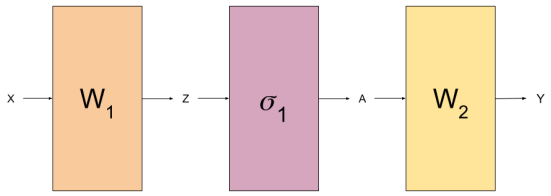


Figure 2.4: LSTM as a 2-layer fully connected NN

layer σ_1 in Figure 2.4 represents all the nonlinear operations in the LSTM, and W_1 represents all the linear operations such that $A = h$ and $Z = \begin{pmatrix} a_i & a_f & a_0 & a_c & c \end{pmatrix}^T$.

In [31], the notation $|\cdot|$ applied to a matrix represents an elementwise absolute value of the matrix, and $\|\cdot\|$ represents the induced 2-norm. They define h and g and bounds on the derivatives of the activation layer

$$|\sigma_1^{'}| \leq g, \;\; |\sigma_1^{''}| \leq h.$$

In practice, h and g can be found easily by propagation of bounds. Next, [32] defines the following two quantities for each layer I of an L-layer neural network recursively

$$r^{(I)} = \begin{cases} \|W^{(1)}\|, & I = 1; \\ g\|W^{(1)}\|r^{(I-1)}, & I \geq 2; \end{cases}$$

$$S^{(L,I)} = \begin{cases} |W^{(L)}|, & I = L-1; \\ g|W^{(1)}|S^{(L-1,I)}, & I \leq L-2. \end{cases}$$

Note, $S^{(L,I)}$ is a matrix, whereas $r^{(I)}$ is a scalar. Theorem 4 in [32] finds the bound on Hessian of the jth component of Y (see Figure 2.4) as

$$\|\nabla_X^2 Y_j\| \leq h \sum_{I=1}^{L-1} \left(r^{(I)}\right)^2 \max_k \left(S_{j,k}^{(L,I)}\right).$$

2.4 Reduced-order modelling and surrogate modelling

Model order reduction the blanket term approaches aimed at reducing the size and/or numerical complexity of a model, usually with the aim of speeding up computation for use in some other applications where repeated predictions are needed.

Let the high fidelity high-dimensional full-order model be expressed as a partial differential equation

$$\dot{y} = f(y, x, \partial_x y, ..., u, t),$$

y is the quantity of interest, where x is some spatial variable, and u is some parameter (for example the control input), t is time, and $\dot{y}$ represents the time derivative. The aim of the reduced-order model (ROM) is to find a reduced-dimension embedding and time-evolution dynamics that reconstruct the high-fidelity state as accurately as possible.

The most common way to approach, especially for linear systems, is through proper orthogonal decomposition. Many samples of the state vector are captured from the full-order model (FOM),

$$\mathbf{X} = \begin{bmatrix} | & | & & | \\ y_1^{u_1} & y_2^{u_1} & ... & y_m^{u_k} \\ | & | & & | \end{bmatrix},$$

where each y_k is a single sample and the superscript u_k denotes the parameters for that sample. A modal basis can be calculated by applying singular value decomposition (SVD) on $\mathbf{X}$, which can be ordered by the magnitude of the singular values and truncated up to a desired order, l, to obtain a reduced basis, Φ_l. This method is proper, meaning it guarantees that for a given l, Φ_l is the optimal l-rank basis for the $\|\cdot\|_2$-norm [36, p. 6]. The high-dimensional state vector (of rank n) can be reduced to rank l by

$$\begin{aligned} y(t) \in \mathbb{R}^n \rightarrow a(t) = \Phi^{\mathrm{T}} y(t), a \in \mathbb{R}^l \\ y(t) = \Phi a(t), \end{aligned} \tag{2.6}$$

where a is the vector of reduced POD-coefficients, and Φ_l is written as Φ for convenience. If f is linear (meaning it may be replaced by matrix operation with a matrix $A \in \mathbb{R}^{n \times n}$) the dynamics (time evolution) of a can be derived from the full-order governing PDEs

$$\begin{aligned} \dot{y} &= Ay(t) = A\Phi a(t) \\ &\implies \Phi^T \dot{y} = \dot{a} = \Phi^T A \Phi a(t) \\ &\implies \dot{a} = \hat{A} a(t), \ \hat{A} \equiv \Phi^T A \Phi \in \mathbb{R}^{l \times l}. \end{aligned} \tag{2.7}$$

This essentially gives a separation of variable solution to the governing PDE, with spatial projection handled by the POD basis in (2.6) and the time evolution handled by the POD-reduced dynamics in (2.7).

When f is nonlinear, however, the nonlinearity must be computed by lifting the state back to full-dimension at each timestep and running it through the full-order governing PDE - hence losing much of the speed-up. Techniques that avoid computing the full nonlinearity are called hyper-reduction, wherein the nonlinearity in approximated instead. One hyper-reduction techniques is discrete empirical interpolation method (DEIM) which samples the nonlinearity and follows a process similar to POD; projecting the nonlinearity onto its own SVD basis and using a greedy algorithm to compute the best points to interpolate the nonlinearity from[3].

Another well-known technique is QR/POD. Unlike POD with DEIM which relies on state samples and yet more samples samples of the nonlinearity to interpolate the nonlinearity, QR/POD relies on sensor measurements; solving the problem of where to best place sensors to approximate the full-state from measurements alone. This is an important distinction and moves us in the direction of the next section, whereas POD with DEIM uses the governing equations in the loop to make predictions about the future state, QR/POD is a purely data-driven approach, using measurements to reconstruct the state. Let C be some output matrix, y the state z the output,

$$\begin{aligned} z = Cy, \ y \approx \Phi a \implies z \approx C\Phi a \\ \implies a \approx (C\Phi)^{-1} z \implies y \approx \Phi (C\Phi)^{-1} z. \end{aligned}$$

Notice, however that for the inverse to be well-defined the output matrix must be carefully chosen such that $C\Phi$ is well conditioned. This is a field of study on its own called principal sensor placement[34][3]. Optimal sensor placement is both difficult to compute (combinatorially hard) and might violate practical constraints (placing sensors in places that are impossible).

Furthermore, the POD-reduced dynamics for a can be unstable [34]. This is where machine learning surrogate modelling might be useful, to learn the dynamics of a. Surrogate modelling is a closely related concept with many of the same goals, however unlike reduced-order modelling, there is less focus on reducing the governing PDE and

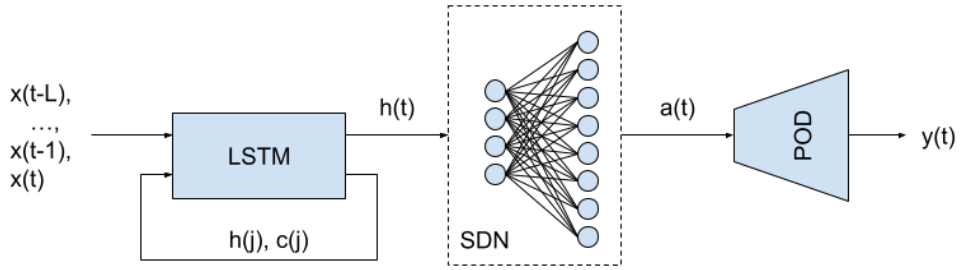


Figure 2.5: Structure of SHRED-ROM model

more on fitting a smaller model to predict the outputs of the high-fidelity model in a data-driven approach [8], treating the underlying governing PDEs as a black-box. Machine learning models have been widely used for surrogate modelling as they are capable of leaning complex dynamics and have fast inference times. The next section introduces the SHRED-ROM model, which is a machine-learning based ROM, combining the concepts of reduced order modelling and surrogate modelling.

2.5 SHallow REcurrent Decoder network (SHRED)

Since SHRED-ROM is motivated by separation of variables as a technique to solve PDEs, it would behove us to quickly discuss how that works. Let the state y be governed by a linear PDE

$$\dot{y} = \mathcal{L}(\partial_x, \partial_x^2, ...)y,$$

where $\mathcal{L}$ is a linear operator that may include spatial derivatives. Assume the solution can written as the product of a temporal and spatial term - this is the central assumption of separation of variables. Then the solution may be written as

$$y = \sum_{0}^{N} c_n a_n(t)\phi_n(x),$$

where a_n is typically a complex exponential, and ϕ_n is an eigenfunction of $\mathcal{L}$. The solution is truncated to N modes to be numerically viable. Initial or boundary conditions may be used to solve for the constant c_n to give a unique solution as some superposition of the N modes.

One could instead use time series of N measurements $\{ y(t_1, \hat{x}), y(t_2, \hat{x}), ..., y(t_N, \hat{x}) \}$ of y at a given point $\hat{x}$ to uniquely solve for the constants. Or equivalently, 2 sensors with $N/2$ measurements each, and so on, ensuring that each sensor is actually measuring the physics that are relevant (putting a second thermometer in your drink to help measure the pool temperature is not helpful).

The idea behind SHRED is built by analogy to this[38]: training a recurrent model learn a latent (a reduced state) encoding and to predict it, and a shallow decoder network (SDN) to project back to the high-dimensional state-space. As opposed to POD method where the latent representation is a SVD based projection, the latent state here is learned, and - similarly to QR/POD - it is a purely data-driven method which does not use the governing equations in the loop. A shallow decoder network is simply a few-layered fully connected NN (as opposed to a deep network with many layers), named such as it does the job of decoding from the latent space to a higher-dimensional space. Being a

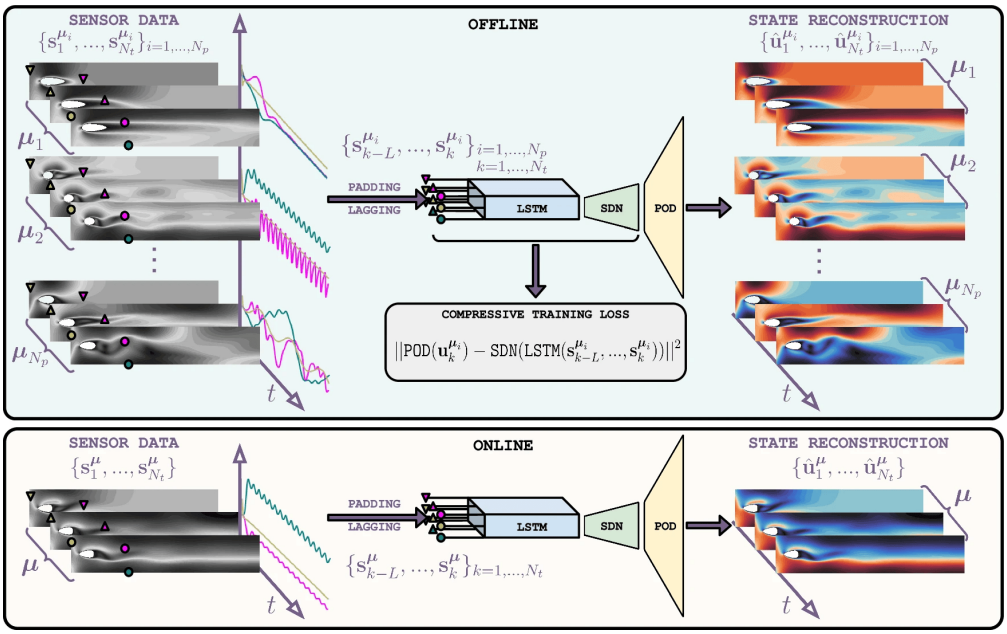


Figure 2.6: Diagram of SHRED compressive training scheme, taken from [34]

decoder-only network it can be much smaller and simpler. Hence the full network is a shallow recurrent decoder based reduced order model: SHRED-ROM.

In the original SHRED paper [38], William, Zahn, and Kutz used limited sensor measurements with time series of previous measurements to estimate the full-state with SHRED for a series of problems (turbulent flow, sea-surface temperature, atmospheric ozone concentration). This is conceptually similar to QR/POD (which also uses sensor measurements to estimate the full state), showed remarkable better reconstruction and forecasting ability, while using much fewer sensors and proved to be agnostic to sensor placement (in sharp contrast to QR/POD). It also showed how sensor trajectories can encode significant amounts of information, and made the case for SHRED as both a state-estimation and state-forecasting model.

In [34] Tomasetto et. al. used SHRED with compressive training, targetting the POD coefficients (or some other physics-informed basis) as the target high-dimensional projection for SDN, which allows the model to be smaller and reduce computational complexity and memory usage [34]. This is referred to as ultra-hyper-reduction in the paper, as it goes one step further than hyper reduction: the latent encoding learned by the recurrent network is a latent encoding not of the full state but of the truncated POD state. The full scheme is (illustrated in Figure 2.6) as follows: the recurrent network learns an encoding of the POD state and learns its dynamics, the shallow decoder network learns a one-way projection onto the POD space, and the POD basis reconstructs the full state. The state snapshots are transformed by POD and truncated, these will be used as the target outputs for the SDN. The sensor measurements are made into sequences of some desired length, adding padding to the front to avoid burn-in time (this is explained further in the next chapter), these sequences serve as the inputs. The model can then be trained to predict the POD modes, and at testing the full-state can be reconstructed from the predicted POD modes.

They tested the model on a range of fluid-dynamics problems: shallow water equations, the GoPro physics test (reconstructing a footage of fluid dynamics experiments shot on GoPro), Kuramoto-Sivashinsky equation which model chaotic

nonlinear fluid dynamics, and flow around an obstacle. The results showed they were able to get quite accurately reconstructions using only the time-history measurements (trajectory) of 3 sensors, and much faster than the original simulations, using a much lighter model.

2.6 Automatic differentiation

Optimisation is discussed in the next chapter, and since the models of interest are neural networks, we need the ability to run numerical optimisation on the outputs of neural networks. At its core, numerical optimisation simply requires the gradient of the model, which requires automatic differentiation - automatic computation of the gradient of the model. As all of machine learning rest on backward propagation, automatic differentiation is a vital tool. Machine learning libraries like `PyTorch` and `TensorFlow` have built-in automatic differentiation into their tensor objects to facilitate the computation and flow of gradients that enables backprop.

The most prevalent optimisation tools, however, expect model to be CasADi objects[1]. CasADi is a symbolic mathematics and automatic differentiation library. We will use the Learning 4 CasADi library created by Salzmann et al[30][29].

Chapter 3

Optimal control

Optimal control in general and the online computation of the MPC control inputs in particular rest on numerical optimisation. However, the theory of numerical optimisation is not the focus. In the first section of this chapter, therefore, we present useful results from optimisation theory without proof (which can be found in [24, ch. 17]). In later sections, we introduce optimal control theory more rigorously.

3.1 Constrained optimization

A general optimisation problem consists of minimising an *objective function* f subject to a set of constraints $\{ c_i \mid i \in \mathcal{E} \cup \mathcal{I} \}$ where $\mathcal{E}$ are the indices of the *equality constraints*, and $\mathcal{I}$ is the set of indices of the *inequality constraints*

$$i \in \mathcal{E} \implies c_i(x) = 0,\ i \in \mathcal{I} \implies c_i(x) \geq 0.$$

The problem can be stated formally as

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to } \begin{cases} c_i = 0,\ i \in \mathcal{E}, \\ c_i \geq 0,\ i \in \mathcal{I}. \end{cases} \tag{3.1}$$

The set of points that fulfil the constraints is the *feasible set* Ω. Generally, inequality constraints may be converted into equality constraints by implementation of a *slack vector* $s \geq 0$ such that

$$c_i(x) - s = 0\ \forall i \in \mathcal{I}.$$

The optimal solution x^* in that case must be accompanied by an optimal slack vector z^*. Find a solution requires f to be "sufficiently" smooth (smooth enough for a given optimality condition to be used).

Definition 3.1.1. *The active set $\mathcal{A}(x)$ at a feasible point x is the set of all equality constraints and any inequality constraint for which $c_i(x) = 0$, as these inequality constraints are active as x. The rest of the inequality constraints (those such that $c_i(x) > 0$) inactive.*

Definition 3.1.2. *A sequence $\{x_k\}_{\mathbb{N}}$ is a feasible sequence if $\exists N$ such that $\forall k > N$ $x_k \in \Omega$.*

3.1.1 First-order necessary conditions

First order optimality conditions are based on linearisation (first order Taylor expansion) of the objective and constraint problem around a point, hence they require the objective and constraint functions to be at least once continuously differentiable, $\mathcal{C}^1$. If there is a *feasible descent step*, meaning it is possible to take a step along a direction where the objective decreases without violating any constraints, it cannot be the optimum.

This first-order approach assumes that the linearisation correctly captures the nonlinear optimisation problem at the point of interest, otherwise the linearisation does not give useful insight on the nonlinear problem. To verify this we introduce *constraint qualification* - a set of assumptions that that ensure the constraint set is similar to its linearisation in a neighbourhood of the point of interest.

Definition 3.1.3. *The vector d is tangent to set Ω at point x if $\exists\{z_k\}_{\mathbb{N}}$ a feasible sequence whose limit is x, and a scalar sequence $\{t_k\}_{\mathbb{N}} \xrightarrow{k\to\infty} 0$ from above such that*

$$\frac{z_k - x}{t_k} \xrightarrow{k\to\infty} d.$$

Definition 3.1.4. *The tangent cone $T_\Omega(x^*)$ is the set of all tangents to Ω at point x^*.*

The tangent cone is the set of direction feasible descent directions at feasible point x.

Definition 3.1.5. *At feasible point x, the linearised feasible direction set $\mathcal{F}(x)$ is*

$$\mathcal{F}(x) = \{\, d \mid d^T \nabla c_i(x) = 0 \;\forall i \in \mathcal{E},\; d^T \nabla c_j(x) \geq 0 \;\forall i \in \mathcal{I} \,\}$$

The linearised feasible set is the can be understood as the set of directions from feasible point x along which the constraints are satisfied to first order (*first-order feasibility*), as indeed, to first order $c_i(x + d) = c_i(x) + \nabla c_i(x)^T d + \mathcal{O}(s^2)$, and since x is feasible $c_i(x) = 0$ for $i \in \mathcal{E}$, hence

$$d^T \nabla c_i(x) = 0 \implies c_i(x + d) \approx 0.$$

The same line of reasoning can be applied to inequality constraints to get first-order feasibility if

$$c_i(x) + \nabla c_i(x)^T d \geq 0.$$

Constraint qualifications ensure $\mathcal{F}(x)$ is sufficiently close (or equal) to $T_\Omega(x^*)$. One commonly used constraint qualification is the *linear independence constraint qualification* (LICQ).

Definition 3.1.6. *The linear independence constraint qualification (LIQC) holds at point x if the gradient of all active constraints*

$$\{\, \nabla c_i(x) \mid i \in \mathcal{A}(x) \,\}$$

are linearly independent.

To encode both descent and feasibility into a single object, we introduce the Lagrangian

$$\mathcal{L}(x, y, z) : x \mapsto f(x) - \sum_{i\in\mathcal{E}} y_i c_i(x) - \sum_{j\in\mathcal{I}} z_j c_j(x),$$

where y and z are the vector whose entries are the Lagrange multipliers for the equality and inequality constraints, respectively. The gradient of the Lagrangian

$$\nabla_x \mathcal{L}(x, y, z) = \nabla f(x) - \sum_{i \in \mathcal{E}} y_i \nabla c_i(x) - \sum_{j \in \mathcal{I}} z_j \nabla c_j(x) = 0.$$

Consider an equality constraint c_i, $i \in \mathcal{E}$. At some feasible point x if there is a first-order feasible descent step s available then

$$\nabla f(x)^T s < 0 \text{ and } \nabla c_i(x)^T s = 0,$$

meaning d is normal to $\nabla c_i(x)$. If x^* is an optimum, then this step may not exist, meaning all descent directions must violate the constraint. This holds if the two gradients are parallel, hence $\exists y \in \mathbb{R}$ such that

$$\nabla f(x) - y \nabla c_i(x) = 0$$

Now consider an inequality constraint c_j, $i \in \mathcal{I}$. When the constraint is inactive, need $\nabla f(x)^T s < 0$ to disprove optimality, as if $c_j(x) > 0$, then $\exists s > 0$ a step small enough that $c_j(x + s) > 0$ (by continuity of c_j). In the active set we need, $\nabla f(x^*) < 0$ as well as $c_j(x) + \nabla c_j(x)^T s = \nabla c_j(x)^T s \geq 0$, hence if x^* is optimal then necessarily the two gradients must be parallel (and pointing in the same direction), such that $\exists z_j \geq 0$ such that

$$\nabla f(x^*) - z_j \nabla c_j(x^*) = 0.$$

We combine the two cases for the inequality constraint by introducing the *complementarity conditions* which states $z_j c_j(x^*) = 0$ such that when the constraint is inactive, the multiplier must be zero. Now we may state the first-order necessary conditions.

Theorem 3.1.1. *(First-order necessary conditions / KKT-conditions) If x^* is a local solution to the optimisation problem* (3.1)*, f an c_i are $\mathcal{C}^\infty$, and LIQC holds at x^*, then $\exists y^*, z^*$ such that*

$$\begin{aligned}
\nabla_x \mathcal{L}(x^*, y^*, z^*) &= 0 && && \textit{(Stationarity)} \\
c_i(x^*) &= 0, && \forall i \in \mathcal{E} && \textit{(Primal feasibility)} \\
c_j(x^*) &\geq 0, && \forall j \in \mathcal{I} && \textit{(Primal feasibility)} \\
z_j^* &\geq 0, && \forall j \in \mathcal{I} && \textit{(Dual feasibility)} \\
\sum_{j \in \mathcal{I}} c_j(x^*) z_j^* &= 0, && && \textit{(Complementary slackness)}
\end{aligned}$$

Stationarity ensures that the gradient of the Lagrangian is zero. Primal feasibility checks that constraints are satisfied (the primal problem) by x^*. Dual feasibility checks that the Lagrange multipliers satisfy the dual problem. Complementarity slackness connects the primal and dual problems, ensuring that either the inequality constraint is inactive or its Lagrange multiplier is zero.

If the inequality constraints are converted into equality constraint with slack variables, the first-order optimality conditions remain as they are except, complementary slackness may be replaced with

$$\sum_{j \in \mathcal{I}} s_j z_j^* = s^T z = diag(s) z = 0$$

3.1.2 Second-order sufficient conditions

As the name suggest, the first-order conditions are necessary but not sufficient. First-order approximations cannot conclude whether a stationary point is a minimum or a maximum. To determine this, we use the second-order sufficient conditions. Note, that by first using the first-order conditions to rule out all points that cannot be minima, and then only applying the second-order conditions on possible candidates, we minimise the use of the more expensive second-order conditions.

Let x^* be point that meets the necessary conditions. The set of *undecided direction*

$$\{\, w \in \mathcal{F}(x^*) \mid w^T \nabla f(x^*) = 0 \,\}$$

is the set of descent directions in $\mathcal{F}(x^*)$ for which we cannot determine from the first-order expansion if the step would increase or decrease the objective.

Definition 3.1.7. *Assume the objective and constraint functions are $\mathcal{C}^2$, and let x^*, y^*, z^* such that the first-order conditions are met. The critical cone $C(x^*, y^*, z^*) \subset \mathcal{F}(x^*)$*

$$w \in C(x^*, y^*, z^*) \iff \begin{cases} \nabla c_i(x^*)^T w = 0, & \forall i \in \mathcal{E} \\ \nabla c_j(x^*)^T w = 0, & \forall j \in \mathcal{I} \cap \mathcal{A}(x^*) \text{ such that } z_j^* > 0 \\ \nabla c_j(x^*)^T w \geq 0, & \forall j \in \mathcal{I} \cap \mathcal{A}(x^*) \text{ such that } z_j^* = 0 \end{cases}$$

is the set of descent directions in $\mathcal{F}(x^*)$ that satisfy all equality constraints and active inequality constraints for a small enough step.

From the first-order conditions, we know that if an inequality constraint c_j is inactive, then $z_j^* = 0$. So it follows that $\forall w \in (x^*, y^*, z^*)$, $y_i^* \nabla c_i^T(x) w = 0$ and $z_i^* \nabla c_j^T(x) w = 0$ for every equality and inequality constraint. Hence for $w \in C(x^*, y^*, z^*)$ which makes each term in the gradient of the Lagrangian zero

$$w^T \nabla_x f(x^*) = 0,\ \sum_{i \in \mathcal{E}} y_i^* \nabla w^T c_i(x^*) = 0,\ \sum_{j \in \mathcal{I}} z_j^* \nabla w^T c_j(x^*) = 0,$$

This fulfils the stationarity conditions without telling us if f increases or decreases along w. To find that out, we consider the second derivative (Hessian) which encodes information about local curvature.

Theorem 3.1.2. *(Second-order sufficient conditions) Let x^* be a feasible point which satisfies the first-order conditions with corresponding Lagrange multipliers y^*, z^*, if $\forall w \neq 0 \in C(x^*, y^*, z^*)$*

$$w^T \nabla_x^2 \mathcal{L}(x^*, y^*, z^*) w > 0,$$

then x^* is a local minimum and a solution to (3.1).

3.2 Nonlinear programming and IPOPT

The numerical methods for solving nonlinear optimisation problems are called nonlinear programming (NLP). The most commonly used NLP algorithm is IPOPT, introduced in its current form in Andreas Wächter's PhD thesis in 2002 [5]. In this section we briefly explain how it works. A more detailed derivation can be found in [24, ch. 19]

IPOPT is a form of *interior point method* for nonlinear problems. Interior point methods use the line search method to solve the KKT-conditions. They are derived originally from the *barrier problem*, and owe the name to that the original barrier problem

statement did not use the slack variable, instead assuming the initial point was "interior" to the constraint, in the sense that it was feasible. The barrier problem is now stated with the slack variable as such

$$\min_{x,s} \left[f(x) - \mu \sum_{j=1} \ln(s_j) \right],$$

subject to

$$c_i = 0 \; \forall i \in \mathcal{E}, \; c_j \geq 0 \; \forall j \in \mathcal{I} \, , \mu > 0.$$

The need for inequality constraint on s are obviated by the natural logarithm function (which serves as a barrier, hence the name), minimising which requires $s \geq 0$. The barrier problem coincides with the optimisation problem only when *barrier parameter* $\mu = 0$, so the idea is to iteratively solve the barrier problem for a sequence $\{ \mu_k \}_{\mathbb{N}} > 0$ that converges to zero. Let us state the *perturbed-KKT* conditions as a vector function with the barrier parameter

$$F(x, s, y, z) = \begin{pmatrix} \nabla_x \mathcal{L}(x, y, z \\ s^T z - \mu e \\ c_E(x) \\ c_I(x) - s \end{pmatrix} \tag{3.2}$$

where c_E and c_I are row vector functions of the equality and inequality constraints, respectively; and e is a row vector of ones. When $\mu = 0$, (3.2) being equal to zero is equivalent to the KKT conditions, and we have the constraints

$$s_j \geq 0, \; z_j \geq 0 \; \forall j \in \mathcal{I}.$$

Line search methods are a family of root finding algorithms that work by moving in a "line" from the current point determined by a step size α_k, and a step direction p_k, by the following formula

$$x_{k+1} = x_k + \alpha_k p_k.$$

Different line search methods find α_k and p_k differently, although many have p_k of the form

$$p_k = -\mathbf{B}_k^{-1} f(x_k) \tag{3.3}$$

where $\mathbf{B}_k$ is symmetric and invertible. In Newton's method, the $\mathbf{B}_k$ is the Jacobian of the function F at the current point $\nabla F(x_k)$. In quasi-Newton methods, an approximation of the Jacobian is used[1]. In nonlinear programming (and indeed in linear and quadratic programming) Newton or quasi-Newton methods are usually employed to find the roots of the KKT-conditions (Theorem 3.1.1). The Jacobian of F is

$$\nabla F = \begin{bmatrix} \nabla^2_{xx} \mathcal{L} & 0 & -\nabla c_E^T(x) & -\nabla c_I^T(x) \\ 0 & diag(z) & 0 & diag(s) \\ \nabla c_E^T(x) & 0 & 0 & 0 \\ \nabla c_I^T(x) & -\mathbb{I} & 0 & 0 \end{bmatrix}.$$

The Newton step direction (3.3) is the solution of the *primal-dual system*, which is as follows

$$\left(\begin{bmatrix} \nabla^2_{xx} \mathcal{L} & 0 & -\nabla c_E^T(x) & -\nabla c_I^T(x) \\ 0 & diag(z) & 0 & diag(s) \\ \nabla c_E^T(x) & 0 & 0 & 0 \\ \nabla c_I^T(x) & -\mathbb{I} & 0 & 0 \end{bmatrix} \right)^{-1} \begin{pmatrix} p_x \\ p_s \\ p_y \\ p_z \end{pmatrix} = \begin{pmatrix} \nabla_x \mathcal{L}(x, y, z \\ s^T z - \mu e \\ c_E(x) \\ c_I(x) - s \end{pmatrix}.$$

[1]The Jacobian of F requires calculating the Hessian of the Lagrangian, which may be computationally and dimensionally intractable

The step-size in IPOPT is given by *fraction to the boundary rule* condition, which is as follows

$$\alpha_x^{max} = \alpha_s^{max} = \max_{0<\alpha\leq 1} \{ \alpha \mid s + \alpha p_s \geq (1-\tau)s \}$$
$$\alpha_y^{max} = \alpha_z^{max} = \max_{0<\alpha\leq 1} \{ \alpha \mid z + \alpha p_z \geq (1-\tau)z \},$$

where $0 < \tau < 1$ is a constant. The Newton iteration for the point x, as well as the Lagrange multipliers y, z and the slack variable s is

$$x^+ = x + \alpha_x^{max} p_x,$$
$$y^+ = y + \alpha_y^{max} p_y,$$
$$z^+ = z + \alpha_z^{max} p_z,$$
$$s^+ = s + \alpha_s^{max} p_s.$$

This iteration combined with a sequence of barrier parameters converging to zero (from above) is the basis of IPOPT, which converges to a solution at a superlinear (faster than linear) rate [24, ch. 19]. A lot of finesse and care is put into exactly what the sequence of barrier parameters should be, whether the perturbed-KKT should be solved with respect to a μ_k before proceeding to μ_{k+1}, or whether μ should be updated at each Newton iteration. There are also modifications made to deal with the problems that arise from the non-convex, nonlinear nature of the objective [24, ch. 19]. For the exact IPOPT algorithm implemented in CasADi used in this thesis, the reader is referred to [37].

3.3 The optimal control problem

Let a system (S) described by the difference equation

$$(S) : x^+ = f(x, u) \; f : \mathbb{X} \times \mathbb{U} \to \mathbb{R}, \tag{3.4}$$

where $x \in \mathbb{X}$ is the state at the initial timestep, $u \in \mathbb{U}$ is the control input at that same timestep, and x^+ is the state at the next timestep. The system is assumed to evolve in discrete-time for two reasons: the ML surrogate model we will build works on discrete-time, and the online optimization step required in MPC inevitable leads to the finite parametrization of the control input, so starting with discrete dynamics makes things cleaner. This is because continuos dynamics imply solving an infinite-dimensional optimization problem online, which becomes quickly intractable. The sets $\mathbb{X}$ and $\mathbb{U}$ are the state and control sets, respectively. For a control sequence $\mathbf{u} = \begin{pmatrix} u(0) & u(1) & \dots \end{pmatrix}$, the function $\Phi(x, \mathbf{u}, k)$ gives the state at k timesteps after the initial state x following the control sequence $\mathbf{u}$. The continuity of Φ follows from that of f by composition of continuous functions. A finite-horizon optimal control problem is the problem of finding a sequence of N control inputs that steer the system (S) into the terminal set $\mathbb{X}_f \subset \mathbb{X}$, subject to the constraint

$$(x, u) \in \mathbb{Z} \subset \mathbb{X} \times \mathbb{U}, \; x(N) \in \mathbb{X}_f$$

that minimises some cost function.

The finite-horizon cost function is denoted $V_N : (x, \mathbf{u}) \to V_N(x, \mathbf{u}) \in \mathbb{R}$

$$V_N(x, \mathbf{u}) = \sum_{0}^{N-1} l\left((x(i), u(i)\right) + V_f(x(N))$$

where $x(i)$ is governed by (3.4), l is the stage cost, and V_f is the terminal cost. The role of the terminal cost is to penalise the final state based on some user defined criteria (like distance to the target). The finite-horizon optimal control problem is denoted as

$$\mathbb{P}_N(x) : \min_{u \in \mathbb{U}} \{V_N(x, \mathbf{u}) | (x, u) \in \mathbb{Z}\}. \tag{3.5}$$

The set of control inputs that can steer the system to the desired final state $x(N) \in \mathbb{X}_f \subset \mathbb{X}$ in N timesteps is

$$\mathcal{U}_N(x) \equiv \{\mathbf{u} \in \mathbb{U}^N \,|\, \Phi(x, \mathbf{u}, k) \in \mathbb{X}\ \forall k < N,\ \Phi(x, \mathbf{u}, N) \in \mathbb{X}_f\}$$

and the corresponding set of states is

$$\mathcal{X}_n \equiv \{x \in \mathbb{X} \,|\, \mathcal{U}_n(x) \neq \varnothing\}.$$

So far we have discussed the optimal control problem of steering the system (S) to a desired state in N timesteps. More commonly, however, an optimal control problem requires control over infinite time. A common example is set-point tracking, where the set-point trajectory can be arbitrarily long. This is called infinite-horizon optimal control.

The optimal control problem is denoted

$$\mathbb{P}_\infty(x) : \min_{u \in \mathbb{U}} \{V_\infty(x, \mathbf{u}) | (x, u) \in \mathbb{Z}\}.$$

The infinite-horizon cost function is

$$V_\infty(x, \mathbf{u}) = \sum_0^\infty l(x(i), u(i)),$$

where $x(i)$ is governed by (3.4), and l is the stage cost. Notice, the infinite-horizon cost function does not have a terminal cost as there is no "termination".

In the next section, we discuss the foundational theory behind MPC - dynamic programming.

Explain that there are 3 ways of solving optimal control problems - DP, indirect methods, and direct methods. Indirect are not relevant to us and rarely used in practice. SO we will explain dynamic programming and direct methods.

3.4 Dynamic programming

Dynamic programming (DP) is built up on the Bellman optimality principle which states that the optimal control policy from initial time 0 to final time τ_f has the property that all it also constitutes an optimal policy at any later time $0 < t < \tau_f$ with regards to the current state $\mathbf{x}(t)$ [28, ch. 2]. This can be stated formally as $\forall x \in \mathcal{X}_N$, if $\mathbf{u} = \begin{pmatrix} u(0) & u(1) & ... & u(N-1) \end{pmatrix} \in \mathcal{U}_N(x)$ is the optimal control sequence - meaning it is the solution to $\mathbb{P}_N(x)$ - and $\mathbf{x} = \begin{pmatrix} x(0) & x(1) & ...x(N) \end{pmatrix}$ is the corresponding trajectory, then $\forall 0 \leq i \leq N-1$, $\mathbf{u}(i) = \begin{pmatrix} u(i) & u(i+1) & ... & u(N-1) \end{pmatrix}$ is the solution to $\mathbb{P}_{N-1}(x(i))$. Equivalently,

$$V_N^0(x) = \min_{u \in \mathbb{U}} \{l(x, u) + V_{N-1}^0(f(x, u))\}.$$

This is simple to prove by contradiction. Assume for contradiction that $\mathbf{u}$ is optimal for $\mathbb{P}_N(x)$ but $\mathbf{u}(i)$ is not optimal for $\mathbb{P}_{N-1}(x(i))$, that means $\exists\tilde{\mathbf{u}}$ that has a lower cost from $\mathbf{x}(i)$ to $\mathbf{x}(N)$. Let us then create $\hat{\mathbf{u}} = \begin{pmatrix} u(0) & \dots & u(i-1) & \tilde{u}(i) & \dots & \tilde{u}(N-1) \end{pmatrix}$, much must therefore have lower cost than the optimal control sequence which is absurd.

DP recursion can be used to find the optimal control sequence as follows. Starting from time $N-1$, we find the control sequence than minimizes

$$\mathbb{P}_{N-1}(x(N-1) : \min_u \{l(x(N-1), u)\}$$

and denote the optimal cost $V^0_{N-1}(x(N-1))$ which is parametrised by the state in the previous timestep. Then we move to timestep $N-2$ where the optimal control problem is parametrised in $x(N-1)$

$$\mathbb{P}_{N-2}(x(N-2)) : \min_u \{l(x(N-2), u) + V^0_{N-1}(f(x(N-2), u))\}.$$

Since V^0_{N-1} was parametrised in $x(N-1)$ which is fully determined by $(x(N-2), u(N-2))$ this problem is parametrised only in $x(N-2)$. We continue backwards induction until the initial time which finally yields an optimization problem parametrized by the initial state, $\mathbb{P}_0(x(0))$, solving which we can do the forward pass and solve each subsequent optimization problem since it is parametrised in the previous state. This gives an optimal control input sequence from the initial state $x(0) = x$ at timestep 0 to $x(N) = x_f$ at timestep N.

The Hamilton-Bellman-Jacobi theorem provides sufficient condition for a control law to be optimal. To proceed with the derivations in the next subsections, we admit it without proof (which can be found in [7]).

Theorem 3.4.1. *(Hamilton-Bellman-Jacobi sufficiency theorem) For the system $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ with some (possibly time-variant) cost function J. Let $J^0 \in \mathcal{C}^\infty$ be positive definite and be the solution to the Hamilton-Jacobi-Bellman equation*

$$-\frac{\partial J^0}{\partial t} = \min_{\mathbf{u}\in\mathbb{U}} \left[l(\mathbf{x}, \mathbf{u}) + \frac{\partial J^0}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right], \ \forall \mathbf{x}. \tag{3.6}$$

Then J^0 is the optimal cost function (equivalently cost-to-go function). The corresponding controller κ is the optimal controller.

3.4.1 Finite-horizon LQR

Let us derive the optimal cost and controller for a linear system with a quadratic cost function (LQ). These derivations are informed by [33]. Consider a continuos time control system

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$$

and the optimal control problem is to move the system from initial state $\mathbf{x}_0$ to final state 0 within the time horizon τ_f, such that $\mathbf{x}(\tau_f) = 0$, while minimizing some quadratic cost J defined by positive the symmetric positive definite matrix $\mathbf{R}$ and symmetric positive semidefinite matrices $\mathbf{Q}, \mathbf{Q}_f$

$$J = \langle \mathbf{x}(\tau_f), \mathbf{x}(\tau_f) \rangle_{\mathbf{Q}_f} + \frac{1}{2} \int_0^{\tau_f} \left(\langle \mathbf{x}, \mathbf{x} \rangle_{\mathbf{Q}} + \langle \mathbf{u}, \mathbf{u} \rangle_{\mathbf{R}} \right) dt. \tag{3.7}$$

Let J^0 be the optimal cost function. Since J^0 is positive definite, $\exists \mathbf{P} : t \to \mathbf{P}(t)$ symmetric positive definite such that

$$J^0(\mathbf{x}, t) = \mathbf{x}^T \mathbf{P}(t) \mathbf{x} \implies \partial_t J^0(\mathbf{x}, t) = \mathbf{x}^T \dot{\mathbf{P}}(t) \mathbf{x},\ \partial_{\mathbf{x}} J^0(\mathbf{x}, t) = 2\mathbf{x}^T \mathbf{P}(t).$$

Injecting this into the Hamilton-Jacobi-Bellman equation and finding the min by setting the partial derivative equal to zero as it is a the positive definite quadratic form on $\mathbf{u}$, we get

$$\begin{aligned} 0 &= \frac{\partial}{\partial \mathbf{u}} \left[\langle \mathbf{x}, \mathbf{x} \rangle_{\mathbf{Q}} + \langle \mathbf{u}, \mathbf{u} \rangle_{\mathbf{R}} + \frac{\partial J^0}{\partial \mathbf{x}} (\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}) \right] \\ &= 2\mathbf{u}^T \mathbf{R} + 2\mathbf{x}^T \mathbf{P}(t) \mathbf{B} \\ &\implies \mathbf{u}^0(\mathbf{x}, t) = -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}(t) \mathbf{x}. \end{aligned}$$

To determine J^0, we must determine $\mathbf{P}$, which we can do by plugging $\mathbf{u}^0$ back into (3.6):

$$\begin{aligned} -\frac{\partial J^0}{\partial t} &= l(\mathbf{x}, \mathbf{u}^0) + \frac{\partial J^0}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}^0) \\ -\mathbf{x}^T \dot{\mathbf{P}} \mathbf{x} &= \mathbf{x}^T \mathbf{Q} \mathbf{x} + (\mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}(t) \mathbf{x})^T \mathbf{R} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}(t) \mathbf{x} + 2\mathbf{x}^T \mathbf{P}(t) \mathbf{A} \mathbf{x} - 2\mathbf{x}^T \mathbf{P}(t) \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}(t) \mathbf{x} \\ -\mathbf{x}^T \dot{\mathbf{P}} \mathbf{x} &= \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{x}^T \mathbf{P}(t) \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}(t) \mathbf{x} + 2\mathbf{x}^T \mathbf{P}(t) \mathbf{A} \mathbf{x} - 2\mathbf{x}^T \mathbf{S}(t) \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}(t) \mathbf{x} \\ -\mathbf{x}^T \dot{\mathbf{P}} \mathbf{x} &= \mathbf{x}^T \mathbf{Q} \mathbf{x} - \mathbf{x}^T \mathbf{P}(t) \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}(t) \mathbf{x} + \mathbf{x}^T \mathbf{P}(t) \mathbf{A} \mathbf{x} + \mathbf{x}^T \mathbf{A}^T \mathbf{P}(t) \mathbf{x} \\ &\implies -\dot{\mathbf{P}} = \mathbf{Q} - \mathbf{P}(t) \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}(t) + \mathbf{P}(t) \mathbf{A} + \mathbf{A}^T \mathbf{P}(t), \end{aligned}$$

which is the differential Ricatti equation. Solving it with the boundary condition $J^0(\mathbf{x}, \tau_f) = \langle \mathbf{x}(\tau_f), \mathbf{x}(\tau_f) \rangle_{\mathbf{Q}_f} \implies \mathbf{P}(\tau_\mathbf{f}) = \mathbf{Q}_f$ completes the definition of the optimal cost function. The associated optimal control law is

$$\kappa_N(\mathbf{x}, t) = -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}(t) \mathbf{x} = \mathbf{K}_N \mathbf{x}.$$

3.4.2 Infinite-horizon LQR

Now we derive the infinite-horizon LQ controller. The cost function is the same as (3.7) except the upper limit of the integral is ∞,

$$J = \frac{1}{2} \int_0^\infty \left(\langle \mathbf{x}, \mathbf{x} \rangle_{\mathbf{Q}} + \langle \mathbf{x}, \mathbf{x} \rangle_{\mathbf{R}} \right) dt.$$

As in with finite-horizon LQR, min in the above can be replaced with $\frac{\partial}{\partial \mathbf{u}}$. Let J^0 be the optimal cost, since it must the positive definite, we can write is as

$$J^0 = \mathbf{x}^T \mathbf{P} \mathbf{x}$$

for some symmetric positive definite $\mathbf{P}$. Hence

$$\frac{\partial J^0}{\partial \mathbf{x}} = 2\mathbf{x}^T \mathbf{P}.$$

Plugging this into the Hamilton-Jacobi-Bellman equation

$$\begin{aligned} 0 &= \frac{\partial}{\partial \mathbf{u}} \left[\langle \mathbf{x}, \mathbf{x} \rangle_{\mathbf{Q}} + \langle \mathbf{u}, \mathbf{u} \rangle_{\mathbf{R}} + 2\mathbf{x}^T \mathbf{P} (\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}) \right] \\ &= 2\mathbf{u}^T \mathbf{R} + 2\mathbf{x}^T \mathbf{S} \mathbf{B} \\ &\implies \mathbf{u}^0(x) = -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{S} \mathbf{x}. \end{aligned}$$

It still remains to find $\mathbf{P}$, to this end we plug $\mathbf{u}^0$ back into the Hamilton-Jacobi-Bellman equation,

$$\begin{aligned}
0 &= \mathbf{x}^T\mathbf{Q}\mathbf{x} + \mathbf{x}^T\mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{R}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}\mathbf{x} + 2\mathbf{x}^T\mathbf{P}\left(\mathbf{A}\mathbf{x} - \mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}\mathbf{x}\right) \\
&= \mathbf{x}^T\left[\mathbf{Q} + \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P} + 2\mathbf{P}\mathbf{A} - 2\mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}\right]\mathbf{x} \\
&= \mathbf{x}^T\left[\mathbf{Q} + \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P} + 2\mathbf{P}\mathbf{A}\right]\mathbf{x} \\
&= \mathbf{x}^T\left[\mathbf{Q} + \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P} + \mathbf{P}\mathbf{A} + \mathbf{A}^T\mathbf{P}\right]\mathbf{x} \\
&= \mathbf{Q} + \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P} + \mathbf{P}\mathbf{A} + \mathbf{A}^T\mathbf{P}.
\end{aligned}$$

This is the continuos algebraic Riccati equation and it admits a unique positive definite solution $\mathbf{P}$ if and only if the system is controllable. The associated control law is

$$\kappa_\infty(\mathbf{x}, t) = -\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}\mathbf{x} = \mathbf{K}_\infty\mathbf{x}.$$

Note that these results are illustrative, and for simpler derivation a continuos time system was used. Similar (although more cumbersome) derivation can be made for the discrete time systems by solving the discrete algebraic Riccati equations

$$\mathbf{P} = \mathbf{A}^T\mathbf{P}\mathbf{A} + \left(\mathbf{A}^T\mathbf{P}\mathbf{B}\right)\left(\mathbf{R} + \mathbf{B}^T\mathbf{P}\mathbf{B}\right)^{-1}\left(\mathbf{B}^T\mathbf{P}\mathbf{A}\right) + \mathbf{Q}$$

and the discrete LQ infinite horizon controller is [2] $\mathbf{K}_\infty = -\left(\mathbf{R} + \mathbf{B}^T\mathbf{P}\mathbf{B}\right)^{-1}\mathbf{B}^T\mathbf{P}\mathbf{A}$.

Chapter 4

Model predictive control (MPC)

This chapter introduces model predictive control from the ground up, starting from the foundations of optimal control. The focus shifts swiftly to nonlinear model predictive control (NMPC) as that is of most relevance to this body of work. Much of the content is informed by Chapter 2 of [28], with certain sections from [4], [11], [27], [15], [18].

4.1 Preliminaries

This section covers some of the definitions and results that aid in the understanding of the subsequent chapters.

Definition 4.1.1. *A set $X \subseteq \mathbb{R}^n$ is positive invariant by some function $f : \mathbb{R}^n \to \mathbb{R}^n$ if $\forall x \in X \ f(x) \in X$.*

Definition 4.1.2. *A set $X \subseteq \mathbb{R}^n$ is control invariant by some function $f : \mathbb{R}^n \times \mathbb{U} \to \mathbb{R}^n$ if $\forall x \in X \ \exists u \in \mathbb{U}$ such that $f(x, u) \in X$.*

Proposition 4.1.1. *(Weierstrass) If $f : \mathbb{R}^\ltimes \to \mathbb{R}^\ltimes$ is continuos and $X \subset \mathbb{R}^\ltimes$ is compact, then f admits a minimum on X.*

Proof. Since X is compact, it is bounded, and by continuity of f, as is $f(X)$. Since $f(X)$ is bounded, it must admit an infimum $\alpha = \inf(f(X))$, we must show that the infimum is indeed in $f(X)$. Let $(x_i)_\mathbb{N}$ be a sequence on X such that $(f(x_i))_\mathbb{N}$ is non-increasing and converges to α, then by compactness $(x_i)_\mathbb{N}$ must admit a converging subsequence to some limit $\tilde{x}$. Continuity implies $(f(x_i))_\mathbb{N} \xrightarrow{k} f(\tilde{x}) = \alpha$ (by uniqueness of limits). Hence $\alpha \in f(X)$. □

Definition 4.1.3. *A trajectory is a solution to a dynamical system with a particular initial state. A maximal solution is one whose domain cannot be extended. A solution is complete if its domain is unbounded.*

Definition 4.1.4. *(Classes $\mathcal{K}$, $\mathcal{K}_\infty$) A function $\alpha : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$ of class $\mathcal{K}$, meaning $\alpha \in \mathcal{K}$, has the following properties: α is continuos, $\alpha(0) = 0$, and it is strictly increasing. It is also in $\mathcal{K}_\infty \subset \mathcal{K}$ if it is radially unbounded.*

Definition 4.1.5. *(Class $\mathcal{KL}$) A function $\beta : \mathbb{R}_{\geq 0} \times \mathbb{N} \to \mathbb{R}_{\geq 0}$ is of class $\mathcal{KL}$ if it is continuos in both arguments, the mapping $s \mapsto \beta(s, k)$ is of class $\mathcal{K}$ $\forall k \in \mathbb{N}$, and the mapping $k \mapsto \beta(s, k)$ is non-increasing and converges to 0 as $k \to \infty$ $\forall s \in \mathbb{R}_{\geq 0}$.*

Proposition 4.1.2. *(Image of a compact set by a continuous function is compact) Statement and proof needed.*

Definition 4.1.6. *(Difference inclusion) A difference inclusion is the discrete-time parallel of a differential inclusion, which itself is a generalisation of differential equations. The idea is that instead of the state dynamics being defined by an element-valued map (a function), they are defined by a set-valued map. This is denoted for a differential inclusion as*

$$\dot{x} \in F(x,t) \subset R^n,$$

and for a difference inclusion as

$$x^+ \in F(x,t) \subset R^n.$$

A somewhat frivolous (but hopefully illuminating) example could be a parametric differential equation, where the system

$$\dot{x} = f(x,t,\mu)\ \mu \in G,$$

can equivalently written as

$$\dot{x} \in \{f(x,t,\mu) | \mu \in G\}.$$

4.2 Model predictive control

Consider once again an infinite-horizon optimal control problem. In the LQ case, dynamic programming yields an optimal control law; however, this requires solving an infinite-dimensional optimisation problem, which is in general intractable for nonlinear systems. It is even intractable for LQ systems if there are hard constraints imposed (which makes the closed-loop system nonlinear). The idea of model predictive control (MPC) is to find an optimal infinite-horizon controller by iteratively solving a finite-horizon optimal control problem.

Since optimisation over the infinite horizon is intractable, the next obvious approach is to optimise the input over a shorter finite horizon N and repeat at timesteps $N\mathbb{N}$. This means running a two-part process starting from the initial state x - *(prediction)*: use the model (S) (3.4) to predict the future states; and *(optimisation)*: an optimization routine to find the optimal inputs $\mathbf{u} = \begin{pmatrix} u(0) & u(1) & ... & u(N-1) \end{pmatrix}$. Since this does not take the real state measurement into account for the future states, this can be thought of as an open-loop control law. As always in control theory, open-loop control laws are susceptible to disturbances. Furthermore, since the system (S) is in general an approximation of the real system, even in the absence of any other source of disturbance, if the open-loop control is applied for multiple timesteps, the errors would accumulate, and the system would not follow the predicted trajectory. This control law can be closed by applying only the first element of the open-loop control input sequence, $u(0)$, then, at the next timestep, the new state becomes available, and the prediction and optimisation routines are repeated. Hence, at each iteration, the prediction horizon recedes by one, which explains the original name for this type of controller, *receding horizon control*, now more commonly called *model predictive control*.

This receding horizon property also means MPC is time-invariant, as at each iteration it is always at zero local time. This is the key difference between MPC and DP. In the finite-horizon case, the DP optimal control problem is inherently time-variant. The same state may require a different control policy depending on the time-to-go. DP gives an

optimal control policy - that is, a sequence of control laws $\mu_i^0 : \mathcal{X}_i \to \mathbb{U}$, so that the controlled system follows the trajectory

$$x_{i+1} = f(x, \mu_i^0(x_i)),\ 0 \leq i \leq N-1.$$

The time-variance is captured in the time dependence of the control law. In the infinite-horizon case, the DP solution is time-invariant and encodes the cost out to infinity in the cost function. In MPC, on the other hand, due to the receding horizon, the controller is always at the first timestep of the current horizon. The controller applies the first input from the predicted horizon and shifts the horizon, making it time-invariant. This means even if the optimal solution is found on each MPC iteration, the MPC control sequence is not necessarily the optimal control sequence for a fixed time interval[28, ch.2].

For the same horizon, N, and initial state, x, MPC and DP will give the same control sequence and state trajectory, but in practice the horizon given to DP is the full finite-horizon of a finite-horizon problem, whereas the horizon for MPC is just a computational trick. Consequently, while they would give the same trajectory for the same horizon, applying DP to an optimal control problem with a fixed time interval $[0, T]$ and MPC to the same problem with $N < T$, the MPC solution might differ from the optimal solution given by DP.

As with finite-horizon LQR, if MPC is implemented with with the only stage-cost then it has no information about the future cost, and might myopically steer the system to a state which minimises the cost in the prediction horizon but is expensive (or impossible) later. To add information about the "cost-to-go" in MPC, a terminal cost V_f may be added that applies to the final state in each iteration; we define

$$V_f : x \in \mathbb{X}_f \to V_f(x) \in \mathbb{R}_{\geq 0}.$$

The optimal control problem then is

$$\mathbb{P}_N(x) : \min_{\mathbf{u} \in \mathbb{U}^N} \{V_N(x, \mathbf{u}) \mid (x, u) \in \mathbb{Z},\ x(N) \in \mathbb{X}_f\},$$
$$V_N(x, \mathbf{u}) = \left[\sum_0^{N-1} l(x(i), u(i))\right] + V_f(x(N)),$$

and the solution is the optimal MPC cost function V_N^0.

Key to understanding MPC design is that, unlike finite-horizon LQR where the terminal cost is a user-defined penalty for suboptimal final behaviour, the terminal cost in MPC is an artificial term the user does not have freedom over. It is dictated by stability requirements and serves to bound the infinite-horizon cost function by the MPC cost function[28, ch.2].

Assume in the following that the origin is an equilibrium, which can be generalised to any other equilibrium by shifting the origin. The MPC optimal control problem is well-posed and admits a solution if the following conditions are met

Condition 4.2.1. *Conditions on system and cost functions*

- *The state-update function $f : \mathbb{Z} \to \mathbb{X}$ is continuos, and $f(0, 0) = 0$.*
- *The stage-cost function $l : \mathbb{Z} \to \mathbb{R}_{\geq 0}$ is continuos, and $l(0, 0) = 0$.*
- *If it exists, terminal-cost function $V_f : \mathbb{X} \to \mathbb{R}_{\geq 0}$ is continuos, and $V_f(0) = 0$.*

Condition 4.2.2. *Conditions on the constraints*

- *The state-control set* $\mathbb{Z} \subset \mathbb{X} \times \mathbb{U}$ *is closed.*
- *The terminal constraint set* $\mathbb{X}_f \in \mathbb{X}$ *is closed.*
- *The control set* $\mathbb{U}$ *is bounded.*

Proposition 4.2.1. *If Conditions 4.2.2 and 4.2.1 are met, then the optimal control problem in* (3.5) *admits a solution.*

Proof. Since the cost function V_N is defined as a sum and composition of continuous functions of l, V_f, f, its continuity follows from the continuity of the constituents. Boundedness of $\mathcal{U}_N(x) \subset \mathbb{U}$ follows naturally from the boundedness of $\mathbb{U}$, and its closedness comes from that of $\mathbb{Z}$, and that is, it is defined by finitely many inequalities. Hence $\mathcal{U}_N(x)$ is compact. Then it follows by Weierstrass that the optimal control problem admits a solution. □

4.3 Stability

The most prevalent question in control theory is "can we guarantee stability of the system and controller?". This is the focus of this chapter: to develop a method for proving stability of the MPC controller using Lyapunov stability theory. There are a multitude of different types of stability in the world of control. We begin by providing a few relevant definitions of stability, then discuss how to go about proving the stability of an MPC controller.

Definition 4.3.1. *(Local stability) A point* $\tilde{x}$ *is locally stable if* $\forall \epsilon > 0$, $\exists \delta > 0$ *such that for all initial state* x *such that* $x \in B(\tilde{x}, \delta) \implies x(k) \in B(\tilde{x}, \epsilon)\ \forall t$.

Definition 4.3.2. *(Global attractivity) A point* $\tilde{x}$ *is globally attractive if all maximal (but not complete) solutions are bounded, and all complete solutions converge to* $\tilde{x}$.

Definition 4.3.3. *(*$\mathcal{KL}$*-stability) The equilibrium* $\tilde{x}$ *of discrete time system governed by the state update equation* $x^+ = f(x)$ *(assuming* $\mathbb{X}$ *is positively invariant by* f*)* $\mathcal{KL}$*-stable if* $\exists \beta \in \mathcal{KL}$ *such that*

$$\forall x \in \mathbb{X}, \forall k \in \mathbb{N}\ \ \|\tilde{x} - \Phi(x, k)\| \leq \beta(\|x - \tilde{x}\|, k)$$

. Then $\mathbb{X}$ *is the region of attraction.* $\mathcal{KL}$*-stability implies asymptotic stability.*

Theorem 4.3.1. *(Lyapunov stability theorem) Let* $\mathbb{X}$ *be positively invariant by* $(x, u) \mapsto f(x, u) = x^+$*, and* $\tilde{x}$ *be an equilibrium. Let* $\kappa : x \mapsto \kappa(x)$ *be a control law. If* $\exists V : \mathbb{R} \to \mathbb{R}_{\geq 0}$ *a Lyapunov function such that* $\exists \alpha_1, \alpha_2 \in \mathcal{K}_\infty$ *and* α_3 *positive definite and continuos such that*

- $V(x) \geq \alpha_1(\|x - \tilde{x}\|)\ \forall x$ *(Positive definite and radially unbounded)*
- $V(x) \leq \alpha_2(\|x - \tilde{x}\|)\ \forall x$ *(Sandwich property)*
- *such that* $V(f(x, \kappa(x))) - V(x) \leq -\alpha_3(\|x - \tilde{x}\|)\ \forall x$ *(Descent property, derivative is negative semidefinite)*

then $\tilde{x}$ is asymptotically stable for the closed-loop system. Note also that if α_3 is merely positive semi-definite, the equilibrium is marginally stable.

Proof. Without loss of generality (to make notation less cumbersome) assume $\tilde{x} = 0$. In the case the equilibrium is not the origin or indeed is a set $\mathcal{A}$, we replace the norm $|\cdot|$ with the distance to $\tilde{x}$, $|\cdot|_{\tilde{x}}$ or the set, $|\cdot|_{\mathcal{A}}$. Let the Lyapunov function V exist; let us show that this proves $\mathcal{KL}$-stability.

Lemma B.14 in [28] proves that the existence of a positive definite α_3 implies the existence of a $\mathcal{K}_\infty$ function that satisfies the same bounds; let us call this function α_3 instead.

The ultimate goal is to bound $\Phi(x, K)$ with a $\mathcal{KL}$ function. We start by bounding the cost by a $\mathcal{K}$ function. Using the Lemma and the descent property, $\exists \alpha_3 \in \mathcal{K}_\infty$ such that

$$V(\Phi(x, i+1)) \leq V(\Phi(x, i)) - \alpha_3(|\Phi(x, i)|).$$

Since class-$\mathcal{K}$ functions are positive definite, they admit inverses, so using the upper bound

$$V(x) \leq \alpha_2(|x|) \implies \alpha_2^{-1}(V(x)) \leq |x|,$$

hence, since α_3 is non-decreasing,

$$-\alpha_3(|x|) \leq -\alpha_3\left(\alpha_2^{-1}(V(x))\right).$$

Combining with the descent property gives

$$V(\Phi(x, i+1)) \leq V(\Phi(x, i)) - \alpha_3\left(\alpha_2^{-1}(V(\Phi(x, i)))\right) := \sigma_1(V(\Phi(x, i))).$$

The function $\sigma : s \in \mathbb{R}_{\geq 0} \to s - \alpha_3 \circ \alpha_2^{-1}(s)$ is continuos, $\sigma_1(0) = 0$, and $\sigma_1(s) < s$, however it is not necessarily non-decreasing, hence not class-$\mathcal{K}$. Let us define $\sigma_2 : s \to \max_{s' \leq s} \sigma_1(s')$, which is non-decreasing by definition and well-defined as $\sigma_1 \in \mathcal{C}$ (continuous functions admit maxima in bounded sets), and $\sigma_2(0) = 0$. We must prove it is continuous.

Assume for contradiction that σ_2 is not continuos, hence $\exists \tilde{s} \in \mathbb{R}_{\geq 0}$ such that

$$\sigma_2 \xrightarrow{s \nearrow \tilde{s}} a_1 \text{ and } \sigma_2 \xrightarrow{s \searrow \tilde{s}} a_2 \text{ with } a_1 \neq a_2.$$

Since σ_2 is non-decreasing and defined by the maximum of σ_1, it follows that $\sigma_1(\tilde{s}) \leq a_1 < a_2$. The non-decreasing property of σ_2 also implies that $\sigma_2 \geq a_2$ for $s > \tilde{s}$, but this means $\exists s' \in [0, \tilde{s}]$ such that $\sigma_1(s') = a_2$ as it is continuos (by intermediate-value theorem). But this would imply $\sigma_2 \geq a_2$ before $\tilde{s}$, which contradicts the non-decreasing property. Hence σ_2 is continuous. Now we define

$$\sigma : s \in \mathbb{R}_{\geq 0} \to \frac{1}{2}(s + \sigma_2(s))$$

which inherits the continuity of σ_2, and $\sigma(0) = 0$, and the addition of the linear term makes it strictly increasing (as σ_2 is already non-decreasing). So we conclude that $\sigma \in \mathcal{K}_\infty$ as well as $\sigma_1 < \sigma(s) < s$.

Since $\sigma_1 < \sigma$, using the descent property recursively, we have

$$V(\Phi(x, i)) \leq \sigma^i(V(x)),$$

where the superscript on σ denotes composition with itself. Combining this with the upper bound by α_2

$$V(\Phi(x,i)) \leq \sigma^i(\alpha_2(|x|)).$$

Next, we use the lower bound α_1

$$\alpha_1(|\Phi(x,i)|) \leq V(\Phi(x,i)) \leq \sigma^i(\alpha_2(|x|))\,\forall x, i,$$

hence

$$|\Phi(x,i)| \leq \alpha_1^{-1}(\sigma^i(\alpha_2(|x|))) := \beta(|x|,i)\,\forall x, i.$$

We have shown that β bounds the Φ as required for $\mathcal{KL}$-stability. Now it only remains to prove that $\beta : (s,k) \in \mathbb{R}_{\geq 0} \times \mathbb{N} \to \left(\alpha_1^{-1} \circ \sigma^k \circ \alpha_2\right)(s)$ is of class $\mathcal{KL}$. Since the first input is $|x| \in \mathbb{R}_{\geq 0}$), and the second $i \in \mathbb{N}$, the domain is correct. Let $w_i(s) := \sigma^i(\alpha_2(s))$. Since $\sigma(s) < s$, $w_{i+1} < w_i$, so it is strictly non-increasing in i. Furthermore, it is bounded from below by 0, so it must also be convergent. Let this limit be a, then by continuity of σ

$$\lim_{i\to\infty} \sigma(w_i(s)) = \sigma(\lim_{i\to\infty} w_i(s)) = \sigma(a).$$

Notice that $\sigma(w_i) = w_{i+1}$, hence $\sigma(w_i) \xrightarrow{i\to\infty} \sigma(a) = a$. Given $\sigma(s) < s$ for $s > 0$, $a = 0$, hence the mapping $i \mapsto \beta(s,i)$ is non-increasing and converges to 0 as $i \to \infty$, for all s. All the composite functions $\alpha_1^{-1}, \alpha_2, \sigma$ are of class $\mathcal{K}$, hence so is the mapping $s \mapsto \beta(s,i)$. This proves that $\beta \in \mathcal{KL}$, which concludes the proof. □

The meat-and-potatoes work of proving stability is finding a Lyapunov function. In DP with infinite-horizon, the natural candidate is the optimal cost function V_∞^0, so the natural candidate for MPC would be V_N^0. While it is not always a Lyapunov function, one can impose some conditions under which it is.

Condition 4.3.1. *(Stabilising conditions) V_f, l, and $\mathbb{X}_f$ have the following properties:*

- *$\mathbb{X}_f$ is control invariant by f,*
- *$\exists \alpha_f \in \mathcal{K}_\infty$ such that $V_f(x) \leq \alpha_f(|x|)\ \forall x \in \mathbb{X}_f$,*
- *$\forall x \in \mathbb{X}_f$, $V_f(f(x,u)) - V_f(x) \leq -l(x,u)$, combined with Condition 4.2.1 and the previous point this makes V_f a local control Lyapunov function on $\mathbb{X}_f$*
- *$\exists \alpha_l \in \mathcal{K}_\infty$ such that $l(x,u) \geq \alpha_l(|x|)\ \forall x \in \mathcal{X}_N$, $\forall u \in \mathbb{U}$.*

Proposition 4.3.1. *Under Conditions 4.2.1, 4.2.2, and 4.3.1, the optimal MPC cost function V_N^0 is a control Lyapunov function and the equilibrium $\tilde{x}$ of the system $x^+ = f(x,u)$ is asymptotically stable in $\mathcal{X}_N$.*

Proof. It is quite clear that by Theorem 4.3.1, if V_N^0 is a control Lyapunov function, then stability follows. Let us prove that it is a control Lyapunov function.

The lower bound is the easiest to prove. Since $\forall N \in \mathbb{N}, x \in \mathcal{X}_N, u \in \mathbb{U}$, $V_N^0(x) \geq l(x,u)$ and by assumption of Condition 4.2.2, l is bounded by below by a $\mathcal{K}_\infty$ function, hence so is V_N^0.

To prove the upper bound, we must first prove the following Lemmas

Lemma 4.3.1. *V_N^0 is locally bounded ($\forall x \in \mathcal{X}_N\ \exists$ neighbourhood X of x such that $V_N^0(X)$ is bounded).*

Proof. Take some arbitrary $x \in \mathcal{X}_N$ and let X be any compact neighbourhood of x. Since (by Condition 4.2.2) $\mathbb{U}$ is compact, as is $X \times \mathbb{U}$, and by continuity of V_N (Condition 4.2.1) it must admit an upper bound on $X \times \mathbb{U}$. Since $V_N^0(x) \leq V_N(x,u)\ \forall x \in \mathcal{X}_N, u \in \mathcal{U}_N(x)$, it admits the same upper bound on X, meaning $V_N^0(X)$ is bounded. Hence V_N^0 is locally bounded. □

Lemma 4.3.2. *Suppose $\tilde{x} \in \mathbb{X}_f^\circ$ (interior), and $\exists \alpha \in \mathcal{K}_\infty$ which bounds V_f from above on $\mathbb{X}_F$. Then $\exists \tilde{\alpha} \in \mathcal{K}_\infty$ that bounds V_f from above on $\mathcal{X}_N$.*

Proof. See proof in [28] Proposition 2.16 and Proposition B.25. □

Lemma 4.3.3. *Under conditions 4.3.1, with j the time-to-go*

- $V_{j+1}^0(x) \leq V_j^0(x)\ \forall x, j,$
- $V_j^0(x) \leq V_f(x)\ \forall x, j.$

Proof. Recall the definition of V_N is

$$V_N(x,u) = \sum_0^{N-1} l(x(i), u(i)) + V_f(x(N)),$$

hence

$$V_j(x,u) = \sum_0^{j-1} l(x(i), u(i)) + V_f(x(j)),$$

and so by DP recursion

$$V_{j+1}^0(x) = l(x, u^0(j+1)) + V_j^0(f(x, u^0(j+1))).$$

Starting at $j = 0$, $V_0^0(x) = V_f(x)$, and for $j = 1$, $V_1^0(x) = l(x, u^0(1)) + V_f(f(x, u^0(1)))$. By condition 4.3.1 ($V_f(x^+) \leq V_f(x) + l(x,u)$) we get that $V_1^0(x) \leq V_f(x) = V_0^0(x)$.

Assume for induction that $V_j^0 \leq V_{j-1}^0$ and $V_j^0 \leq V_f$. By the definition of the optimal cost,

$$V_{j+1}^0 = l(x, u^0(j+1)) + V_j^0(f(x, u^0(j+1))),$$

so

$$\left(V_{j+1}^0 - V_j^0\right)(x) = l(x, u^0(j+1)) - l(x, u^0(j)) + V_j^0(f(x, u^0(j+1))) - V_{j-1}^0(f(x, u^0(j))).$$

Since x is the initial state at time-to-go j, $u^0(j+1)$ is not optimal, hence

$$\left(V_{j+1}^0 - V_j^0\right)(x) \leq l(x, u^0(j)) - l(x, u^0(j)) + V_j^0(f(x, u^0(j))) - V_{j-1}^0(f(x, u^0(j))) \leq 0.$$

Finally, since $V_{j+1}^0(x) \leq V_j^0$ and $V_j^0 \leq V_f(x)$, it follows that $V_{j+1}^0(x) \leq V_f(x)$. □

Now assume $\tilde{x} \in \mathbb{X}_f^\circ$. By Lemma 4.3.3, $V_N^0(x) \leq V_f(x)$, and by Condition 4.3.1, V_f is bounded from above by $\alpha_f \in \mathcal{K}_\infty$ function on $\mathbb{X}_f$, hence so is V_N^0. Then by Lemmas 4.3.1 and 4.3.2 we can extend this bound to $\mathcal{X}_N$.

Finally, let us prove the descent property. Take some $x \in \mathcal{X}_N$ so that the optimal control sequence is $\mathbf{u}^0(x) = \begin{pmatrix} u_0^0(x) & u_1^0(x) & \dots & u_{N-1}^0(x) \end{pmatrix}$ and the corresponding trajectory is $\mathbf{x}^0(x) = \begin{pmatrix} x_0^0(x) = x & x_1^0(x) & \dots & x_N^0(x) \end{pmatrix}$. At the next timestep, the state is $x^+ = f(x, u_0^0(x))$ and now the optimal control sequence is $\mathbf{u}^0(x^+)$. By definition,

$\forall \tilde{\mathbf{u}} \in \mathcal{U}_N(x^+)$, $V_N(x^+, \mathbf{u}^0(x^+)) \leq V_N(x^+, \tilde{\mathbf{u}})$. Let $\tilde{\mathbf{u}} = \begin{pmatrix} u_1^0(x) & u_1^0(x) & ... & u_{N-1}^0(x) & u \end{pmatrix}$ with u any valid control input. Then

$$
\begin{aligned}
V_N(x^+, \tilde{\mathbf{u}}) &= l(f(x_N^0(x), u), u) + \sum_1^{N+1} l(x_k^0(x), u_k^0(x)) + V_f(f(x_N^0(x), u)) \\
V_N^0(x^+) &\leq l(f(x_N^0(x), u), u) + \sum_1^{N+1} l(x_k^0(x), u_k^0(x)) + V_f(f(x_N^0(x), u)) \\
&\leq V_N^0(x) + l(f(x_N^0(x), u), u) - l(x, u_0^0(x)) + V_f(f(x_N^0(x), u)) - V_f(f(x_{N-1}^0(x), u_{N-1}^0(x))) \\
&\leq V_N^0(x) - l(x, u_0^0(x)) \text{ (by Condition 4.3.1 } V_f(x^+) - V_f(x) \leq l(x, u))
\end{aligned}
$$

Since l is bounded from below by $\alpha_l \in \mathcal{K}_\infty$, we get the descent property.

Since V_N^0 is a Lyapunov function for the system $x^+ = f(x, u)$, it is asymptotically stable for the equilibrium $\tilde{x}$ on the set $\mathcal{X}_N \subset \mathbb{X}$. □

Note that we assumed that $\tilde{x} \in \mathbb{X}_f^\circ$, however, this assumes $\mathbb{X}_f$ has an interior, which is not necessarily true (although in most cases one can expand the set to make it so). In this case, the upper bound cannot be proved; it must be assumed that $\exists \alpha \in \mathcal{K}_\infty$ that bounds V_N^0 from above on $\mathcal{X}_N$. This is also called "weak controllability" as we limit ourselves to those states where the cost to get to the terminal set $\mathbb{X}_f$ in the N-timestep horizon is finite.

4.4 Quasi-Infinite Horizon Nonlinear MPC (QIH-NMPC)

Proving the properties required for stability can be quite challenging for nonlinear systems, the tricky step being finding a terminal constraint set and a terminal cost function. This section explains one method for determining these and how to apply MPC to nonlinear systems practically, called quasi-infinite horizon nonlinear model predictive control (QIH-NMPC). This strategy works by linearising the nonlinear systems in a neighbourhood around the equilibrium, computing a linear controller, and using it to find the terminal cost and constraint set.

Let us first understand the mechanics of the methods before proceeding with the stability proof. Assume l and f are of class C^2 (twice continuously-differentiable) and l is quadratic, of form

$$l(x, u) = \frac{1}{2} \left(\langle x, x \rangle_{\mathbf{Q}} + \langle u, u \rangle_{\mathbf{R}} \right),$$

and for the sake of simplicity assume $\tilde{x} = 0$ (although the results is general, the notation is more cumbersome for $\tilde{x} \neq 0$). Linearise the system around the equilibrium as

$$x^+ \approx [\partial_x f(\tilde{x}, \tilde{u})] \cdot x + [\partial_u f(\tilde{x}, \tilde{u})] \cdot u = \mathbf{A}x + \mathbf{B}u$$

in some neighbourhood of the equilibrium, and assume the linearised system is stabilisable. Let the stage-cost be quadratic, such that an LQ-controller, $\mathbf{K}$ may be computed for the linearised system.

The idea for the terminal constraint set is to ensure the final state in each iteration is within some set where the linear controller is stabilising for the nonlinear system. Let

$$e(x) = f(x, \mathbf{K}x) - \mathbf{A}_K x \tag{4.1}$$

be the linearisation error (or the nonlinear component of the system as in [27]) for the closed-loop system with the LQ-controller. We would like to bound this error.

Theorem 4.4.1. *Assume $f \in C^2$ and $\mathbb{X}$ is closed and bounded, then the linearisation error is quadratically bounded: $\exists c \in \mathbb{R}_{\geq 0}$ such that*

$$\|e(x)\| \leq c\|x\|^2 \ \forall x \in \mathbb{X}.$$

Proof. Since $f \in C^2$ by assumption, $\partial_x^2 e$ exists, is continuous, and is equal to 0 at $\tilde{x} = 0$. Given that e has dimension n_x, we have to be careful with its derivatives; in particular, we define $\partial_x^2 e_j$ to be the Hessian of the jth element of e. By Taylor's theorem

$$\begin{aligned} |e_j(x)| &= \left| e_j(0) + \partial_x e_j(0) + \frac{1}{2}\int_0^1 x^T \partial_x^2 e_j(xs) x \, ds \right| \\ &= \left| \frac{1}{2}\int_0^1 x^T \partial_x^2 e_j(xs) x \, ds \right| \ \ \forall j \in [1 : n_x] \end{aligned}$$

Due to the continuity of e and the closed and boundedness of $\mathbb{X}$, we can determine a value $\lambda_m = \max_{j \in [1:n_x], x \in \mathbb{X}} |\partial_x^2 e_j(x)|$.

$$\begin{aligned} |e_j(x)| &\leq \left| \frac{1}{2}\int_0^1 x^T c x \, ds \right| \leq \frac{1}{2}\lambda_m \|x\|^2 \\ \implies \|e(x)\| &\leq \frac{\lambda_m \sqrt{n_x}}{2}\|x\|^2 = c\|x\|^2 \end{aligned}$$

□

Theorem 4.4.1 proves that the bound of the Hessian exists but provides no method for calculating it. In [11], the bound is estimated from simulation. Another method to compute the bound is outlined in Section 2.3.

The goal is to define a terminal constraint set where the linearisation error is sufficiently small such that the nonlinear system is positive invariant when closed by the linear controller, to satisfy the first part of Condition 4.3.1. In fact, we wish to find the largest such terminal set to make optimisation easier. As for the terminal cost, as discussed earlier, its purpose is to approximate the "cost-to-go". We specify further here that we wish to bound the "cost-to-go" by the terminal cost. The terminal cost will be the candidate Lyapunov function to fulfil the rest of Condition 4.3.1 to prove stability with the results of Proposition 4.3.1.

Theorem 4.4.2. *Suppose Conditions 4.2.1, 4.2.2, and 4.3.1 are met, and that $V_f \in \mathcal{C}^1$, $V_f(0) = 0$, $\exists \mathbb{X}_f \subset \mathbb{X}$ closed that contains the origin. If there exists a local control law κ_f such that $\kappa_f(0) = 0$,*

$$V_f(f(x, \kappa_f(x))) - V_f(x) \leq -l(x, \kappa_f(x)) \text{ and } \kappa_f(x) \in \mathbb{U}, \forall x \in \mathbb{X}_f, \tag{4.2}$$

and the closed loop is positive invariant in $\mathbb{X}_f$, $f(x, \kappa_f(x)) \in \mathbb{X}_f \, \forall x \in \mathbb{X}_f$, then for the closed-loop (with the MPC controller) the origin is asymptotically stable with the region of attraction $\mathcal{X}_N$ (set the of initial states from where the system can be steered with feasible control inputs into the desired state).

Proof. Let $\mathbf{K}$ be some static linear locally asymptotically stable state-feedback controller (for example, the infinite-horizon LQ-controller), such that $u = -\mathbf{K}x$ for the system $(\mathbf{A}, \mathbf{B})$, meaning the closed-loop dynamics are $x^+ \approx (\mathbf{A} - \mathbf{B}\mathbf{K})x = \mathbf{A}_K x$, where $\mathbf{A}_K$ is asymptotically stable. The stage-cost for the closed-loop linearised system is

$$l(x, \mathbf{K}x) = \langle x, x \rangle_{\mathbf{Q}_K}$$

with $\mathbf{Q}_K = \mathbf{Q} + \mathbf{K}^T\mathbf{R}\mathbf{K}$. Since $\mathbf{Q}_K$ is positive-definite and $\mathbf{A}_K$ is asymptotically stable (hence positive-definite), the discrete-time Lyapunov equation

$$\mathbf{A}_K^T P \mathbf{A}_K + \mathbf{Q}_K = P$$

admits a unique symmetric positive-definite solution $P = \sum_0^\infty \left(\mathbf{A}_K^T\right)^j \mu \mathbf{Q}_K \mathbf{A}_K^j$. Let the terminal cost $V_f : x \mapsto \langle x, x\rangle_P$ be our candidate Lyapunov function. Let $\sigma_{\mathbf{A}_K}$ be the largest singular value of $\mathbf{A}_K$, and $\lambda_{\mathbf{Q}_K}^{max}$ and $\lambda_{\mathbf{Q}_K}^{min}$ the largest and smallest eigenvalues of $\mathbf{Q}_K$, respectively, given that $\mathbf{Q}_K$ is diagonalisable its eigenvalues are its singular values. Applying the singular-value property of the induced matrix norm of the vector 2-norm gives

$$\|P\|_2 \leq \lambda_{\mathbf{Q}_K}^{max} \sum_0^\infty \left(\sigma_{\mathbf{A}_K}^{max}\right)^{2j} = \frac{\lambda_{\mathbf{Q}_K}^{max}}{1 - \left(\sigma_{\mathbf{A}_K}^{max}\right)^2} := \Lambda_P. \tag{4.3}$$

We would like to show that V_f is a local control-Lyapunov function on a neighbourhood of the equilibrium point for the nonlinear system $x^+ = f(x, u)$. Let $lev_a V_f = \{x | V_f(x) \leq a\}$ for some $a \geq 0$. Since P is positive definite, $lev_a V_f$ is ellipsoidal and contains the origin. To be a local control Lyapunov function for f on $lev_a V_f$ it would be sufficient to show the descent property as positive definiteness and sandwich properties come from the positive definiteness of P. So we would like to show for

$$V_f(f(x, \mathbf{K}x)) - V_f(x) \leq -l(x, \mathbf{K}x) = -\langle x, x\rangle_{\mathbf{Q}_K} \ \ \forall x \in lev_a(V_f). \tag{4.4}$$

Recall $e(x) = f(x, \mathbf{K}x) - \mathbf{A}_k x$, hence $f(x, \mathbf{K}x) = \mathbf{A}_k x + e(x)$, therefore

$$\begin{aligned}
&V_f(f(x, \mathbf{K}x)) - V_f(x) \\
&= (\mathbf{A}_k x + e(x))^T P (\mathbf{A}_k x + e(x)) - x^T P x \\
&= x^T \mathbf{A}_k^T P \mathbf{A}_k x - x^T P x + e(x)^T P e(x) + e(x)^T P \mathbf{A}_k x + x^T \mathbf{A}_k^T P e(x) \\
&= x^T \mathbf{A}_k^T P \mathbf{A}_k x + e(x)^T P e(x) + 2e(x)^T P \mathbf{A}_k x \\
&= -x^T \mathbf{Q}_K x + e(x)^T P e(x) + 2e(x)^T P \mathbf{A}_K x \\
&\leq \lambda_{\mathbf{Q}_K}^{min} \|x\|^2 + c^2 \|P\| \|x\|^4 + 2c\|P\|\|x\|^3.
\end{aligned}$$

We introduce the variable $\delta > 0$ as a degree of freedom, which determines the size of the terminal region. Using the previous result, we enforce

$$\begin{aligned}
V_f(f(x, \mathbf{K}x)) - V_f(x) &\leq \delta l(x, \mathbf{K}\mathbf{x}) \\
&\leq \delta x^T \mathbf{Q}_K x \\
\implies |V_f(f(x, \mathbf{K}x)) - V_f(x)| &\leq \delta \lambda_{\mathbf{Q}_K}^{max} \|x\|^2
\end{aligned}$$

as that automatically satisfies (4.4). Hence

$$\begin{aligned}
&-\lambda_{\mathbf{Q}_K}^{min} \|x\|^2 + c^2 \|P\| \|x\|^4 + 2c\|P\|\|x\|^3 \leq -\delta \lambda_{\mathbf{Q}_K}^{max} \|x\|^2 \\
&\implies c^2 \|P\| \|x\|^2 + 2c\|\mathbf{A}_K^T P\| \|x\| + (\delta \lambda_{\mathbf{Q}_K}^{max} - \lambda_{\mathbf{Q}_K}^{min}) \leq 0 \\
&\implies \|x\| \leq \frac{-c\|\mathbf{A}_K^T\|\|P\| + \sqrt{\left(c\|\mathbf{A}_K^T\|\|P\|\right)^2 + (c^2\|P\|)\left(\lambda_{\mathbf{Q}_K}^{min} - \delta\lambda_{\mathbf{Q}_K}^{max}\right)}}{c^2\|P\|}.
\end{aligned}$$

Plugging in (4.3), and simplifying a bit we obtain an expression of $\|x\|$

$$\|x\| \leq \frac{-\sigma_{\mathbf{A}_K}^{max}\Lambda_P + \sqrt{(\sigma_{\mathbf{A}_K}^{max})^2\Lambda_P^2 + \Lambda_P\left(\lambda_{\mathbf{Q}_K}^{min} - \delta\lambda_{\mathbf{Q}_K}^{max}\right)}}{c\Lambda_P} = \alpha. \tag{4.5}$$

Hence α (which depends on δ) may be used to define the terminal set $\mathbb{X}_f = \{\, x \mid \|x\| \leq \alpha \,\} = \mathcal{B}(0,\alpha)$. Hence $\forall x \in \mathbb{X}_f$,

$$V_f(f(x,\mathbf{K}x) - V_f(x) \leq l(x,\mathbf{K}x),$$

which fulfils the descent property on the terminal set as in Condition 4.3.1.

We must also ensure that $\forall x \in \mathbb{X}_f$ $\mathbf{K}x \in \mathbb{U}$. To this end, we impose an upper bound α_l on α, such that α_l defines the largest ball around the origin such that $\forall x \in \mathcal{B}(0,\alpha)$, $u = \mathbf{K}x \in \mathbb{U}$.

To summarize, by careful choice of δ and α_l, we get the following properties:

- l is positive definite as $\mathbf{Q}$ and $\mathbf{R}$ are (and hence bounded from below by a $\mathcal{K}_\infty$ function).
- $\kappa_f : x \ \rightarrow \mathbf{K}x$ is a stabilising linear control law in $\mathbb{X}_f$,
- $\kappa_f(x) \in \mathbb{U}$ for all $x \in \mathbb{X}_f$, and $f(x,\mathbf{K}x) \in \mathbb{X}_f$, so f is control invariant in $\mathbb{X}_f$.
- $\mathbb{X}_f = \mathcal{B}(0,\alpha)$ contains the equilibrium,
- in the terminal set, $V_f(x^+) - V_f(x) \leq -l(x,\kappa_f(x))$,
- by since P is positive definite, $\exists \alpha_f \in \mathcal{K}_\infty$ that bounds it V_f from above. (Hence, V_f is locally a Lyapunov function on the terminal set $\mathbb{X}_f$.)

This is sufficient to prove, using Proposition 4.3.1, that the closed-loop QIH-NMPC system for $x^+ = f(x,u)$ is asymptotically stable about the equilibrium on $\mathcal{X}_N$. □

This proof also outlines the method for deriving the terminal cost and terminal constraint set. The variable δ is a degree of freedom, which can be tuned to maximise the size of $\mathbb{X}_f$ (while ensuring $\alpha(\delta) < \alpha_l$) as a larger constraint set will make it easier to find a trajectory that ends in it.

Now let us show that this terminal cost function is an upper bound on the infinite-horizon "cost-to-go" function. This is not generally true for nonlinear systems; however, if the closed-loop system is restricted to trajectories that remain within some neighbourhood of the equilibrium after the horizon N, an upper bound can be found. Notice that we have already imposed this restriction through the definition of the terminal control law and the terminal constraint set. The optimal infinite-horizon cost function can be split up into two parts

$$V_\infty^0 = \min_{\mathbf{u}} \left[\sum_0^{N-1} l(x(i), u_\infty^0(i)) + \sum_N^\infty l(x(i), u_\infty^0(i)) \right],$$

where the first term (the stage-cost) is the same as the first term in the MPC cost function, and the second term is the cost-to-go. Replacing the infinite horizon control

law with the terminal control law for timesteps after $N-1$ is feasible as the $x(N) \in \mathbb{X}_f$ and $\mathbb{X}_f$ is positive invariant by $f(x, k_f(x))$, hence

$$V_\infty^0 \leq \min_{\mathbf{u}} \left[\sum_0^{N-1} l(x(i), u_\infty^0(i)) + \sum_N^\infty l(x(i), k_f(x(i))) \right].$$

Using (4.2) and the property of telescopic series,

$$\sum_N^\infty l(x(i), k(x(i)) \leq \sum_N^\infty (V_f(x(i)) - V_f(x(i+1))) = V_f(x(N)).$$

Substituting this the previous inequality gives

$$V_\infty^0(x) \leq \min_{\mathbf{u}} \left[\sum_0^{N-1} l(x(i), u_\infty^0(i)) + V_f(x(N)) \right] = V_N^0(x).$$

Hence, an asymptotically stable quasi-infinite-horizon nonlinear model predictive controller (QIH-NMPC) bounds the infinite-horizon optimal cost. The point of this upper bound is that it shows that decreasing the QIH-NMPC cost also forces the infinite-horizon MPC cost to become smaller, meaning we are not just optimising the control problem over the small horizon of N timesteps, but also implicitly decreasing the infinite-horizon cost. Note that while the terminal controller is vital in stability analysis of this controller, at no point is the terminal controller ever used to control the system, as only the first element of the control sequence is applied to the system.

4.5 Terminally unconstrained-MPC

Finding the terminal constraint is quite involved, and its size has a meaningful impact on the optimisation, as explained in the previous section. However, if the optimal control problem does not already have state constraints, even a large terminal constraint set has a meaningful impact on the optimisation, as any state constraint complicates the optimisation step of the MPC algorithm. In this section, the method from [22] is explained, where the terminal constraint is imposed implicitly as a constraint on the initial state.

We start by defining a modified optimal control problem where the terminal constraint is scaled by a factor $\beta \geq 1$, such that the cost function to minimise at each MPC step is

$$V_N^\beta(x, u) = \sum_0^{N-1} l(x(i) + u(i)) + \beta V_f(x(N)), \tag{4.6}$$

and let the optimal cost be denoted

$$V_N^{\beta 0} = \min_{\mathbf{u}} \{ V_N^\beta(x, \mathbf{u}) \mid \mathbf{u} \in \hat{\mathcal{U}}_N(x) \},$$

where $\hat{\mathcal{U}}_N(x)$ is the same as $\mathcal{U}_N(x)$ from the previous sections except the terminal constraint is removed

$$\hat{\mathcal{U}}_N(x) = \{ \mathbf{u} \mid u(i) \in \mathbb{U}, \;\; f(x(i), u(i)) \in \mathbb{X} \text{ for } 0 \leq i \leq N-1, \}.$$

We denote $d > 0$ a lower bound on the stage-cost on $\mathbb{X} \setminus \mathbb{X}_f$ and define the *initial constraint set*

$$\Gamma_N^\beta = \{ x \mid V_N^{\beta 0}(x) \leq Nd + \beta a \},$$

where $\mathbb{X}_f = \{ x \mid V_f(x) \leq a \}$. We can prove stability without a terminal constraint by ensuring the trajectory starts within Γ_N^β.

Theorem 4.5.1. *Assume Conditions 4.2.1, 4.2.2, 4.3.1 are met. Given $x_0 \in \Gamma_N^\beta$, the closed-loop system $x^+ = f(x, \kappa_N^\beta(x))$ where the implicit controller $u = \kappa_N^\beta(x)$ comes from minimisation of* (4.6) *is asymptotically stable.*

Proof. For ease of notation, let a trajectory given by the terminal controller be denoted $x^f(i)$, and by the implicit controller κ_N^β be denoted $x^\beta(i)$. Let us assume for contrapositive that $x^\beta(N) \notin \mathbb{X}_f$ (hence $V_f(x^\beta(N)) > a$), and let us prove that that would imply $x_0 \notin \Gamma_N^\beta$.

From the conditions, there exists a locally stabilising controller κ_f on $\mathbb{X}_f$ that follows the descent property and leads to a positive invariant closed-loop:

$$V_f(f(x, \kappa_f(x)) - V_f(x) \leq -l(x, \kappa_f(x)) \;\; \forall x \in \mathbb{X}_f$$
$$f(x, \kappa_f(x)) \in \mathbb{X}_f \;\; \forall x \in \mathbb{X}_f.$$

For $\beta > 0$, multiplying the left-hand side by β, the above inequality still holds. Hence we may use Lemma 4.3.3 for V_N^β to get for any $j < N - 1$

$$\sum_j^{N-1} l(x^f(i), u^f(i)) + \beta V_f(x^f(N)) \leq \beta V_f(x).$$

As κ_f is not the optimal implicit controller for $\mathbb{P}_j^\beta$, we get

$$V_N^{\beta 0} \leq \sum_j^{N-1} l(x^f(i), u^f(i)) + \beta V_f(x^f(N)) \leq \beta V_f(x).$$

It is obvious from the definition of V_{N-j}^β that

$$V_{N-j}^{\beta 0} \geq \beta V_f(x^\beta(N)).$$

Assume for some $j < N - 1$, that $x^\beta(j) \in \mathbb{X}_f$. The the Bellman optimality theorem it follows that if $\mathbf{u}^\beta = \begin{pmatrix} u^\beta(0) & \dots & u^\beta(N-1) \end{pmatrix}$ is optimal for time-to-go N, then $\begin{pmatrix} u^\beta(j) & \dots & u^\beta(N-1) \end{pmatrix}$ is optimal for time-to-go $N - j$. Hence,

$$\beta V_f(x^\beta(j)) \geq V_{N-j}^{\beta 0}(x^\beta(j)) \geq \beta V_f(x^\beta(N)) > \beta a,$$

such that $x^\beta(j) \notin \mathbb{X}_f \; \forall j < N - 1$. Since d bounds l on $\mathbb{X} \setminus \mathbb{X}_f$, this implies

$$\sum_0^{N-1} l(x^\beta(i), u^\beta(i)) > Nd,$$

which would imply $V_N^{\beta 0} > Nd + \beta a$ which implies $x_0 \notin \Gamma_N^\beta$. Hence $x_0 \in \Gamma_N^\beta \implies x^\beta(N) \in \mathbb{X}_f$. This completes our proof by contrapositive. □

Since all trajectories starting in Γ_N^β converge to the equilibrium, this is the region of attraction. Furthermore, Γ_N^β is positive invariant for the closed-loop. Let $x \in \Gamma_N^\beta$ hence $V_N^{\beta 0}(x) \leq Nd + \beta a$, and by Bellman's optimality principle

$$V_N^{\beta 0}(x^\beta(1)) \leq V_N^{\beta 0}(x) - l(x, \kappa_N^\beta(x)),$$

hence the optimal cost gets smaller for $f(x, \kappa_N^\beta(x))$ meaning it is also in Γ_N^β.

This method of proving stability is helpful for easing numerical optimisation by not applying the terminal constraint; however, it does not alleviate the burden of finding a terminal constraint set as its definition is used in the definition of Γ_N^β. Given a terminal constraint set, the only way to loosen the implicit terminal constraint Γ_N^β is by increasing N, which may make the problem computationally intractable.

[12] shows another technique for proving stability without a terminal cost, but once again stability is only guaranteed for a "sufficiently large horizon".

4.6 Suboptimal-MPC

In the previous sections, we assumed that the solution, V_N^0, to the optimal control problem can indeed be found and used as the Lyapunov function to prove stability. This is a very strong assumption and cannot be guaranteed except for linear cases where the cost function is convex. If the system is nonlinear, the online numerical optimisation scheme in the MPC loop can only find local optima - this is called suboptimal-MPC. This section explains how, in the absence of the optimal cost function, we can use a *warm start* and *extended state* to get stability guarantees.

Definition 4.6.1. *(Warm start) A control sequence $\tilde{\mathbf{u}}$ is a warm start if it can steer the system from the initial state x to the terminal set in N timesteps and the cost of the trajectory produced by $\tilde{\mathbf{u}}$ starting from any x in the terminal set is less than the terminal cost of x, that is $V_N(x, \tilde{\mathbf{u}}) \leq V_f(x)\ \forall x \in \mathbb{X}_f$. The set of all such trajectories is*

$$\tilde{\mathcal{U}}_N(x) = \{\tilde{\mathbf{u}} \in \mathcal{U}_N(x) \,|\, V_N(x, \tilde{\mathbf{u}}) \leq V_f(x)\, \forall x \in \mathbb{X}_f\}.$$

The idea, then, is that to aid the optimiser at each MPC iteration, it is given a warm start as the initial guess. This is generally done as follows: if the previous step's control sequence was $\begin{pmatrix} u_0 & u_1 & \dots & u_{N-1} \end{pmatrix}$, u_0 is used to get the to state at the current timestep. At this timestep we can use

$$\tilde{\mathbf{u}}_w = \begin{pmatrix} u_1 & \dots & u_{N-1} & \kappa_f(x(N)) \end{pmatrix}$$

as the warm start, where the last input is obtained using some terminal control law, κ_f. The terminal controller $\kappa_f : \mathbb{X}_f \to \mathbb{U}$ is such that $f(x, \kappa_f(x)) \in \mathbb{X}_f\ \forall x \in \mathbb{X}_f$. All but the last input come from the optimisation routine in the previous timestep, just time-shifted to match the new time.

In some cases $\tilde{\mathbf{u}}_w \notin \tilde{\mathcal{U}}_N(x)$, in such a case one can use

$$\tilde{\mathbf{u}}_f = \begin{pmatrix} \kappa_f(x) & \kappa_f(f(x, \kappa_f(x))) & \dots & \end{pmatrix}.$$

We define the warm-start update function as

$$\tilde{\mathbf{u}}^+ = \zeta(x, \mathbf{u}) = \begin{cases} \tilde{\mathbf{u}}_f(x) \text{ if } x^+ \in \mathbb{X}_f \text{ and } V_N(x^+, \tilde{\mathbf{u}}_f(x^+)) \leq V_N(x^+, \tilde{\mathbf{u}}_w(x^+)) \\ \tilde{\mathbf{u}}_w(x, \mathbf{u}) \text{ else} \end{cases} \tag{4.7}$$

This ensures that the warm-start is always in the set of admissible warm-start set as we already proved $\tilde{\mathbf{u}}_f \in \tilde{\mathcal{U}}_N(x)$ and $\tilde{\mathbf{u}}_w$ is used only when $V_N(x, \tilde{\mathbf{u}}_w) < V_N(x, \tilde{\mathbf{u}}_f)$ and we have that $V_N(x, \tilde{\mathbf{u}}_f) \leq V_f(x)\ \forall x \in \mathbb{X}_f$. Let us define the set of admissible warm-starts that are an improvement (have lower cost) as

$$\check{\mathcal{U}}_N(x, \tilde{\mathbf{u}}) = \{\, \mathbf{u} \in \tilde{\mathcal{U}}_N(x) \mid V_N(x, \mathbf{u}) \leq V_N(x, \tilde{\mathbf{u}}) \,\}.$$

The suboptimal-MPC control law, $\kappa_N(z, \tilde{\mathbf{u}})$, is defined by the following algorithm. We begin by choosing a terminal set $\mathbb{X}_f$ and terminal cost V_f that satisfy Condition 4.3.1. For a given initial state x, choose any initial warm start $\tilde{\mathbf{u}}$. Then

1. Compute a new input sequence $\mathbf{u} \in \check{\mathcal{U}}_N(x, \tilde{\mathbf{u}}$ with a lower cost.
2. Use the first input f $\mathbf{u}$ to evolve the state to x^+
3. Evolve the warm start by $\tilde{\mathbf{u}}^+ = \zeta(x, \mathbf{u})$,
4. Repeat.

The major hindrance to proving stability in suboptimal-MPC is the lack of an obvious candidate Lyapunov function. The optimal cost V_N^0 worked well for MPC as it depends only on the state. In the absence of the optimal cost function, there are many plausible control inputs that satisfy the conditions for a Lyapunov function, but that would make V_N a set-valued function. We need to define a unique plausible control input that depends purely on the state such that we can define a function that can serve as a Lyapunov function. The warm start is exactly this, and by combining the state and warm start into the extended state, we can use the cost of the state and warm start as the Lyapunov function.

Definition 4.6.2. *(Extended state) Let us denote* $z = \begin{pmatrix} x \\ \tilde{\mathbf{u}} \end{pmatrix} \in \mathcal{Z} := \{ \begin{pmatrix} x & \mathbf{u} \end{pmatrix}^T \mid x \in \mathcal{X}_N, \mathbf{u} \in \mathcal{U}_N(x) \}$ *the extended state that contains both the current state and the warm start control sequence. We can define the system dynamics of the extended state with the following difference inclusion*

$$z^+ \in F(x) = \{ \begin{pmatrix} x^+ & \tilde{\mathbf{u}}^+ \end{pmatrix}^T \mid x^+ = f(x, u_0),\ \tilde{\mathbf{u}}^+ = \zeta(x, \mathbf{u})\ \forall \mathbf{u} = \begin{pmatrix} u_0 & ... \end{pmatrix} \in \check{\mathcal{U}}_N(z) \}.$$

We define the set $\tilde{\mathcal{Z}}_N \subset \mathcal{Z}$ *to include only warm starts, that is*

$$\tilde{\mathcal{Z}}_N = \{ \begin{pmatrix} x & \mathbf{u} \end{pmatrix}^T \mid x \in \mathcal{X}_N,\ \mathbf{u} \in \tilde{\mathcal{U}}_N(x) \}.$$

As, unlike in optimal-MPC, there are many suboptimal trajectories, the system dynamics are set-valued, and hence governed by a difference inclusion. Note the subtle difference between the set-valued nature of the system dynamics and the element-valued nature of the cost function $V_N(z) = V_N(x, \tilde{\mathbf{u}})$ for a given extended state, so it can be used as a Lyapunov function. Still, proving stability in suboptimal-MPC requires a definition of stability for difference inclusions.

Proposition 4.6.1. *(Asymptotic stability of difference inclusion) Let the set* $\mathcal{Z}$ *contain the equilibrium point,* $\tilde{z}$ *and be positive invariant by the difference inclusion* $z^+ \in F(z)$ *and* $F(\tilde{z}) = \{ \tilde{z} \}$*. Then if there exists a function* V *such that*

- V *is positive definite, and positive invariant in* $\mathcal{Z}$ *by* $z^+ = F(z)$*,*
- $\exists \alpha_1, \alpha_2 \in \mathcal{K}_\infty$ *such that*

$$\alpha_1(|z|) \leq V(z) \leq \alpha_2(|z|),\ \forall z \in \mathcal{Z}\ \textit{(Sandwich property)}$$

- $\exists \alpha_3 \in \mathcal{K}_\infty$ *such that*

$$\sup_{z^+ \in F(z)} V(z^+) - V(z) \leq -\alpha_3(|z|),\ \forall z \in \mathcal{Z}\ \textit{(Descent property)}$$

then V is a Lyapunov function for the difference inclusion on $\tilde{\mathcal{Z}}_N$ and $\tilde{z}$ is asymptotically stable.

Proof. The proof is similar to that of Theorem 4.3.1. The exact proof can be found in Appendix B of [28]. □

With this in place, we can show that the cost function for the extended state is a Lyapunov function, and under similar conditions as before, we can prove stability.

Theorem 4.6.1. *Let conditions 2.3.1, 2.3.2, and 2.4.1 be satisfied, except the function bounding the stage-cost is $\alpha(\| \begin{pmatrix} x \\ u \end{pmatrix} \|)$. Let the terminal set be defined $\mathbb{X}_f = lev_b(V_f)$ for some $b > 0$. Then $V_N : z \to V_N(z)$ is a Lyapunov function for the closed loop system*

$$z^+ \in F(z)$$

using the suboptimal-MPC control law on the $\tilde{\mathcal{Z}}_N$. Therefore, the equilibrium is asymptotically stable.

Proof. Assume without loss of generality that the origin is the equilibrium. Let us start by proving that $\tilde{\mathcal{Z}}_N$ is positive invariant.

The suboptimal-MPC algorithm dictates that at each iteration, $\mathbf{u} \in \check{\mathcal{U}}_N(x, \tilde{\mathbf{u}})$, so $x^+ = f(x, u_0) \in \mathcal{X}_N$ and $\tilde{\mathbf{u}}^+ \in \tilde{\mathcal{U}}_N(x^+)$. Hence $z^+ \in \tilde{\mathcal{Z}}_N$, meaning it is positive invariant.

Now let us prove that V_N is a Lyapunov function. Since $V_f \geq 0$,

$$V_N(z) \geq \sum_{i=0}^{N-1} l((x(i), u(i)) \geq \sum_{i=0}^{N-1} \alpha_l \left(\left| \begin{pmatrix} x(i) \\ u(i) \end{pmatrix} \right| \right).$$

Using Lemma 2.5.1,

$$\sum_{i=0}^{N-1} \alpha \left(\left| \begin{pmatrix} x(i) \\ u(i) \end{pmatrix} \right| \right) \geq \alpha \left(\frac{1}{N} \sum_{0}^{N-1} \left| \begin{pmatrix} x(i) \\ u(i) \end{pmatrix} \right| \right).$$

Define $\mathbf{x} = \begin{pmatrix} x(0) & x(1) & ... & x(N-1) \end{pmatrix}^T$ and similarly $\mathbf{u}$. Recall the triangle inequality for norms $\| \begin{pmatrix} \mathbf{x} & \mathbf{u} \end{pmatrix}^T \| \leq \sum | \begin{pmatrix} x(i) & u(i) \end{pmatrix}^T |$. Recall also the $\mathcal{L}^p$-norm property that $\| \begin{pmatrix} \mathbf{x} & \mathbf{u} \end{pmatrix}^T \| \geq \|\mathbf{x}\|$. So we can develop to

$$V_N(z) \geq \alpha \left(\frac{1}{N} \sum_{0}^{N-1} \left| \begin{pmatrix} x(i) \\ u(i) \end{pmatrix} \right| \right) \geq \alpha \left(\frac{\| \begin{pmatrix} x(0) & \mathbf{u} \end{pmatrix}^T \|}{N} \right).$$

Define $\alpha_1(\cdot) := \alpha\left(\frac{\cdot}{N}\right) \in \mathcal{K}_\infty$, hence we have established a lower bound for V_N.

Upper bound and descent prop, finish proof.

□

Lemma 4.6.1. *Let $\alpha \in \mathcal{K}$, then*

$$\alpha(\frac{1}{N} \sum_{1}^{N} b_k) \leq \sum_{1}^{N} \alpha(b_k).$$

Proof. Let $B = \max_{1\leq k\leq N}(b_k)$. Since $\alpha \in \mathcal{K}$, it is strictly increasing, its domain is $\mathbb{R}_{\geq 0}$, and zero at zero, hence non-negative. Due to the strictly increasing property,

$$\alpha(\frac{1}{N}\sum_{1}^{N} b_k) \leq \alpha(\frac{1}{N}\sum_{1}^{N} B) = \alpha(B).$$

Next, due to non-negativity,

$$\alpha(\sum_{1}^{N} B) = \alpha(NB) \leq \sum_{1}^{N} \alpha(Nb_k) \implies \alpha(B) \leq \sum_{1}^{N} \alpha(b_k).$$

Hence finally, we have $\alpha(\frac{1}{N}\sum_1^N b_k) \leq \alpha(B) \leq \sum_1^N \alpha(b_k)$. □

In suboptimal-MPC, the warm start and state are inseparably interdependent. We can go further than the previous proof and define a bound for $x(k)$ rather than z, but the bound will depend on the initial state and initial warm start.

4.7 Stability into an attractor and Economic MPC

So far in this chapter we have seen many methods for proving stability of MPC algorithms for tracking a set-point. Often, however, we only care that the system be driven into a given set, being agnostic to where in the set it ends up, so long as the closed-loop dynamics are positive-invariant (trajectories starting in the set must remain in the set). This set is called the attractor $\mathcal{A}$, and the definition of Lyapunov stability for an attractor is not too dissimilar to the definition used previously. We replace the measure of distance to the set-point with a measure of distance to the set in the definition of Lyapunov stability, Lyapunov function, and Lyapunov stability theorem (4.3.1).

We begin by defining the distance to a set $\|.\|_{\mathcal{A}}$ as the distance to the closest point in a set

$$\|x\|_{\mathcal{A}} = \inf\{\, \|x - \hat{x}\| \mid \hat{x} \in \mathcal{A}\,\}. \tag{4.8}$$

Definition 4.7.1. *($\mathcal{KL}$-stability of attractor) Let $\mathcal{A}$ be a closed, positive invariant set. Then it system governed by the state update equation $x^+ = f(x)$ is globally asymptotically stable if $\exists \beta \in \mathcal{KL}$ such that*

$$\forall x \in \mathbb{X}, \forall k \in \mathbb{N} \;\; \|\Phi(x,k)\|_{\mathcal{A}} \leq \beta(\|x\|_{\mathcal{A}}, k)$$

. Then $\mathbb{X}$ is the region of attraction. $\mathcal{KL}$-stability implies asymptotic stability.

Theorem 4.7.1. *(Lyapunov stability theorem for attractor) Let $\mathbb{X}$ be positive invariant, and $\mathcal{A} \subset \mathbb{X}$ be closed and positively invariant by the closed-loop $(x) \mapsto f(x, \kappa(x)) = x^+$, and f is locally bounded on $\mathcal{A}$. If $\exists V : \mathbb{R} \to \mathbb{R}_{\geq 0}$ a Lyapunov function for the closed-loop and $\mathcal{A}$ such that $\exists \alpha_1, \alpha_2 \in \mathcal{K}_\infty$ and α_3 positive definite and continuos such that*

- $V(x) \geq \alpha_1(\|x\|_{\mathcal{A}})\ \forall x$ *(Positive definite and radially unbounded)*
- $V(x) \leq \alpha_2(\|x\|_{\mathcal{A}})\ \forall x$ *(Sandwich property)*
- $V(f(x,u)) - V(x) \leq -\alpha_3(\|x\|_{\mathcal{A}})\ \forall x \forall u$ *(Descent property, derivative is negative semidefinite)*

then $\mathcal{A}$ is globally asymptotically stable.

The proof of Theorem 4.7.1 is similar to that of 4.3.1 and is omitted. This notion of stability of a set ties in closely with robustness as well; however, due to time limitations, we shall not treat robustness.

If the cost function is zero in $\mathcal{A}$, it is at best positive semidefinite with respect to the equilibrium point, which breaks the assumptions on which previous proofs of stability of QIH-NMPC rely. An alternative approach is presented in [18], which builds on the concept of to Economic MPC (EMPC).

Let us begin by familiarising ourselves with EMPC. Unlike the previous flavours of MPC we have studied, which focus on set-point tracking, EMPC aims to "optimise economic performance of a process" [28, ch.2]. As there may be many states that achieve this, we drop the assumption of positive definiteness on the stage-cost, and drop the terminal cost altogether. We once again define an optimal control problem with respect to this new stage-cost and null terminal cost, and denote V_N^0 the optimal cost function. The lack of positive-definiteness of the stage-cost means that V_N^0 is no longer a candidate Lyapunov function. EMPC find another candidate.

Let the *optimal steady-state pair* be

$$(x_s, u_x) = \underset{(x,u)\in\mathbb{Z}}{\operatorname{argmin}} \{\, l(x,u) \mid f(x,u) = x \,\},$$

the state and input pair that are steady (meaning $f(x,u) = x$) with the lowest stage-cost. We define the following conditions:

Condition 4.7.1. *(EMPC stability conditions)*

1. *The functions f, l are continous.*
2. *The terminal cost is zero.*
3. *$\exists (x_s, u_s) \in \mathbb{Z}$ such that $f(x_x, u_s) = x_s$.*
4. *$\mathbb{Z}$ is closed.*
5. *The terminal set is $\mathbb{X}_f = \{x_s\}$*
6. *The stage-cost admits a lower bound on $\mathbb{Z}$.*

Proposition 4.7.1. *Let Condition 4.7.1 hold, and let κ be the implicit MPC controller corresponding to V_N^0, then $\forall x \in \mathcal{X}_N$ (the states for which $\mathcal{U}_\mathcal{N}(x) \neq \emptyset$)*

$$\limsup_{t\to\infty} \sum_{k=0}^{t} \frac{l(x(k), \kappa(x(k)))}{t} \leq l(x_s, u_s)$$

.

Proof. By definition,

$$V_N^0(x(0)) = \sum_{0}^{N-1} l(x(k), \kappa(x(k))$$

and

$$V_N^0(x(1)) = \sum_{1}^{N-1} l(x(k), \kappa(x(k)) + l(x(N), \kappa(x(N)).$$

Subtracting the first from the second gives

$$l(x(0), \kappa(x(0))) = l(x_s, \kappa(x(N))) + V_N^0(x(0)) - V(x(1)).$$

Of course, there is nothing special about $V_N^0(x(0)), V_N^0(x(1))$, the following holds for any $V_N^0(x(j)), V_N^0(x(j+1))$, which yields a telescopic sequence. Taking the sum gives

$$\sum_0^t l(x(k), \kappa(x(k))) = tl(x_s, \kappa(x(N))) + V_N^0(x(0)) - V_N^0(x(t)).$$

The terminal constraint dictates $x(N) = x_S$, and by the definition of the stead-pair

$$l(x_s, u_s) \leq l(x_s, u)$$

for any u. Plugging in this inequality, we get:

$$\sum_0^t \frac{l(x(k), \kappa(x(k)))}{t} \leq l(x_s, u_s) + \frac{V_N^0(x(0))}{t} - \frac{V_N^0(x(t))}{t}.$$

While the sum on the left-hand side might not have a finite limit (which is why we took lim sup), the right-hand side does admit a limit. The first two terms have trivial limits ($l(x_s, u_s)$ and 0), and for the last term we use the lower bound on l (as per the Conditions) to get a lower bound on $V_N^0(x(t)) \geq M$ such that

$$-\frac{V_N^0(x(t))}{t} \leq -\frac{M}{t} \xrightarrow{t\to\infty} 0.$$

□

Definition 4.7.2. *(Dissipativity) A system governed by $x^+ = f(x,u)$ is strictly dissipative with respect to a supply rate function $s : \mathbb{Z} \to \mathbb{R}$ if $\exists$ a storage function $W : \mathbb{X} \to \mathbb{R}$ and $\alpha \in \mathcal{K}_infty$ such that*

$$W(f(x,u)) - W(x) \leq s(x,u) - \alpha(\|x - x_s\|) \ \forall (x,u) \in \mathbb{Z}.$$

Definition 4.7.3. *(Detectability)[15] Let $\sigma : \mathbb{X} \to \mathbb{R}$ be a positive definite and such that $\sigma(x) = 0 \implies \exists u \in \mathbb{U}$ such that $l(x,u) = 0$, and $\sigma(f(x,u)) = 0$. The function σ is "detectable" if $\exists$ a function W, $\epsilon_0 > 0$, $\gamma_1, \gamma_2 \geq 0$ such that $\forall (x,u) \in \mathbb{Z}$*

$$\gamma_1 \sigma(x) \leq W(x) \leq \gamma_2 \sigma(x),$$
$$W(f(x,u)) - W(x) \leq -\epsilon_0 \sigma(x) + l(x,u).$$

Theorem 4.7.2. *Let Conditions 4.7.1. Define the supply function s as*

$$s(x,u) = l(x,u) - l(x_s, u_s)$$

and let the system be strictly dissipative with respect to it, with the storage function W. Let $Y_N = V_N^0 + W : \mathcal{X}_N \to \mathbb{R}$ be continuos at x_s. Then x_s is asymptotically stable for the closed-loop $x^+ = f(x, \kappa(x))$ where $\kappa(x)$ is the implicit controller corresponding to V_N^0.

Proof. Define the *rotated stage-cost*

$$\tilde{l}(x,u) = l(x,u) - l(x_s, u_s) + W(x) - W(f(x,u)).$$

Given that we assumed the system is strictly dissipative with respect to $l(x,u) - l(x_s, u_s)$, we have that for some $\alpha_1 \in \mathcal{K}_\infty$

$$W(f(x,u)) - W(x) \leq l(x,u) - l(x_s, u_s) - \alpha_1(\|x - x_s\|),$$

hence

$$\tilde{l}(x,u) \geq \alpha_1(\|x - x_s\|).$$

Furthermore, as $f(x_s, u_s) = x_s$,

$$\tilde{l}(x_s,u_s) = l(x_s,u_s) - l(x_s,u_s) + W(x_s) - W(x_s) = 0.$$

We then use this rotated stage-cost to define

$$\begin{aligned}
\tilde{V}_N(x,\mathbf{u}) &= \sum_0^{N-1} \tilde{l}(x(k),u(k)) \\
&= \left[\sum_{k=0}^{N-1} l(x(k),u(k))\right] - Nl(x_s,u_s) + \sum_{i=0}^{N-1} W(x(i)) - \sum_{j=0}^{N-1} W(f(x(j),u(j))) \\
&= \left[\sum_{k=0}^{N-1} l(x(k),u(k))\right] - Nl(x_s,u_s) + \sum_{i=0}^{N-1} W(x(i)) - \sum_{j=1}^{N} W(x(j)) \\
&= \left[\sum_{k=0}^{N-1} l(x(k),u(k))\right] - Nl(x_s,u_s) + W(x(0)) - W(x(N)) \\
&= V_N(x,\mathbf{u}) - Nl(x_s,u_s) + W(x) - W(x_s).
\end{aligned}$$

Since $\tilde{V}_N$ is simply V_N plus some terms that do not depend on the optimisation parameters, the existence of an optimal solution for V_N implies the existence for $\tilde{V}_N$ - this optimal $\tilde{V}_N^0$ is our candidate Lyapunov function. The lower bound of the sandwich property is obvious, as we have already shown that $\tilde{l}$ is bounded by $\alpha_1(\|x - x_s\|) \in \mathcal{K}_\infty$. This also gives us the descent property. Let κ_N be the implicit MPC controller for the initial state x. Then

$$\begin{aligned}
\tilde{V}_N^0(f(x,\kappa(x))) &= \sum_{k=1}^{N} \tilde{l}(x(k),\kappa(x(k))) \\
&= \sum_{k=0}^{N} \tilde{l}(x(k),\kappa(x(k))) - \tilde{l}(x,\kappa(x)) \\
&\leq \sum_{k=0}^{N-1} \tilde{l}(x(k),\kappa(x(k))) - \tilde{l}(x,\kappa(x)) \\
&\leq \tilde{V}_N^0(x) - \tilde{l}(x,\kappa(x)).
\end{aligned}$$

Finally, we must show the upper bound of the sandwich property. By assumption, Y_N is continuos at x_s, hence as is $\tilde{V}_N^)$. Then by Lemme 4.3.2, we have an upper bound by some $\mathcal{K}_\infty$ function α_2. These properties are sufficient to prove $\tilde{V}_N^0$ is a Lyapunov function for the closed-loop system and the point x_s, so the x_s fr the closed-loop is asymptotically stable with region of attraction $\mathcal{X}_N$. □

While this proof from [28] once again focuses on stability for a point, it can easily be extended to a set by using the Lyapunov stability theorem for sets ((4.7.1)). Proving strict dissipativity as required in EMPC can be quite challenging. [18] outlines an alternative approach. A slightly specified version of their method (to make it applicable

to our problem) is presented here. We assert that the stage-cost is positive semi-definite, and only zero on the attractor $\mathcal{A}$. Define

$$l^*(x) := \inf_{u \in \mathbb{U}} l(u, x),$$

such that $l^*(\mathcal{A}) = \{ 0 \}$ and $l^*(\mathbb{X} \setminus \mathcal{A}) \subset \mathbb{R}_{>0}$.

Theorem 4.7.3. *Assume the following conditions are met:*

1. *(State measure)* $\exists \alpha_1, \alpha_2 \in \mathcal{K}_\infty$ *such that*

$$\alpha_1(\|x\|_{\mathcal{A}}) \leq l^*(x) \leq \alpha_2(\|x\|_{\mathcal{A}}) \ \ \forall x \in \mathbb{X}.$$

2. *(Exponential cost controllability)* $\exists \hat{\gamma} > 0$ *such that* $\forall k \leq 1 \ \exists \gamma_k \in [0, \hat{\gamma}]$ *such that*

$$V_k^0(x) \leq \gamma_k l^*(x) \ \ \forall x \in \mathbb{X}. \tag{4.9}$$

3. *(Cost detectability) The function* l^* *is detectable from* l.

Then $Y_N = V_N^0 + W$ *is a candidate Lyapunov function for the closed-loop and* $\mathcal{A}$*, and exhibits the following properties:*

$$\epsilon_0 l^*(x) \leq Y_N(x) \leq (\gamma_N + \gamma_0) l^*(x), \tag{4.10}$$

$$Y_N(f(x, \kappa(x))) - Y_N(x) \leq -\epsilon_0 \alpha_N l^*(x), \tag{4.11}$$

with $\alpha_N := 1 - \frac{\gamma_N(\gamma_N - \gamma_0)}{\epsilon_0^2 (N-1)}$*. If* N *is chosen such that* $N > 1 + \frac{\gamma_N(\gamma_N - \gamma_0)}{\epsilon_0^2}$*, then* $\alpha_N > 0$*, meaning* Y_N *is a control Lyapunov function for the closed-loop and the set* $\mathcal{A}$*, which implies* $\mathcal{A}$ *is asymptotically stable for the closed-loop.*

Theorem 4.7.3, the proof for which can be found in Appendix B of [18], guarantees stability for a long enough horizon, which comes at a performance trade off. It uses a similar argument to the proof of EMPC, which the notable difference that stage-cost in EMPC can be arbitrary whereas here it must be positive semi-definite, but the strict dissipativity requirements is replaced with detectability. [18] also incorporates a terminal cost later in another theorem, which relaxes the N requirement.

4.8 Direct single-shooting and direct multiple-shooting for MPC

Given that the implementation of MPC relies so heavily on numerical optimisation, before moving towards implementation it is first important to marry the theory presented in the two previous chapters. That is the aim of this section.

We begin by stating the optimal control problem as an optimisation problem,

$$\min_{\mathbf{x},\mathbf{u}} \left[V_f(x(N) + \sum_0^N l(x(i), u(i)) \right]$$

subject to

$$\begin{aligned} x(0) &= x, \text{ (initial value)} \\ x^+ &= f(x, u), \text{ (system dynamics)} \\ (x, u) \in \mathbb{Z} &\iff h(x, u) \geq 0, \text{ (expressed as inequality constraints value)} \\ x(N) \in \mathbb{X}_f &\iff r(x(N)) = 0. \text{ (expressed as an equality constraint)} \end{aligned}$$

Earlier in this chapter, we saw how to choose V_f and $\mathbb{X}_f$ to guarantee stability. Now we outline how to implement the optimisation itself. Given the problem of interest is discrete and high-dimensional, we focus on direct methods.

There are two classes of direct methods: *sequential* and *simultaneous*. Sequential methods are simpler in concept and implementation - each time step in the finite horizon is predicted sequentially (one after the other) in the optimisation routine, but they prove to be less performant, especially for highly nonlinear system dynamics. The sequential method most commonly used is *single-shooting*. Simultaneous methods, as the name suggests, predict and optimise each timestep in parallel and enforce continuity conditions based on the system dynamics equations to ensure the trajectory is physical. Two simultaneous methods are commonly used: *collocation* and *multiple-shooting*. The remainder of this section discusses single-shooting and multiple-shooting in detail.

In direct single-shooting, in general, we begin by discretising the control into finitely many step inputs over a time grid. As our focus is on discrete systems, this step is already done. For a finite horizon N, a single iteration of the single-shooting MPC algorithm is as follows

1. Start the optimiser with an initial guess control sequence $\mathbf{u} = \begin{pmatrix} u_0 & ... & u_{N-1} \end{pmatrix}$,
2. Use the guess to compute the trajectory over the entire horizon,
3. Compute the cost of the trajectory and control sequence,
4. The optimiser computes a descent direction, takes a step and computes a better input sequence,
5. The optimiser repeats steps 2 to 5 until some stopping criteria are met.

Using ":" to denote dependency, the optimisation problem to be solved in direct single-shooting MPC is

$$\min_{\mathbf{u}} \left[V_f(x(N:\mathbf{u}) + \sum_{0}^{N} l\left(x(i:\mathbf{u}), u(i)\right) \right] \text{ subject to } \begin{cases} (x(i:\mathbf{u}), u(i)) \in \mathbb{Z},\ i = 0, 1, 2, ... \\ x(N) \in \mathbb{X}_f. \end{cases}$$

Note that the system dynamic constraint is no longer explicit; it is implicitly fulfilled by the algorithm as states are not a degree of freedom of the optimiser, but calculated from the state dynamics. Aside from its simplicity, it has the benefit over multiple-shooting that there are fewer optimisation degrees of freedom, and it only needs initial guesses for the control sequence[6]. Its drawbacks include that it has high nonlinear sensitivity to the initial guess, creates a badly conditioned nonlinear programming problem for unstable systems, and the inability to provide an initial guess for the state trajectory[6]. The lack of the state trajectory as an initial guess is presented both as a benefit and drawback, as while in some cases it is difficult to give a guess, in applications like set-point tracking, being able to give the desired state trajectory as an initial guess is advantageous.

Direct multiple-shooting breaks up the solves optimisation problems at each timestep simultaneously, while imposing the continuity condition to enforce system dynamics. The problem statement is

$$\min_{\mathbf{s},\mathbf{u}} \left[V_f(s(N) + \sum_{0}^{N} l\left(s(i), u(i)\right) \right] \text{ subject to } \begin{cases} s_0 - x = 0 \\ s_{i+1} - f(s_i, u_i) = 0 \\ (s(i), u(i)) \in \mathbb{Z} \\ s(N) \in \mathbb{X}_f \end{cases}$$

For a finite horizon N, a single iteration of the single-shooting MPC algorithm is as follows:

1. Start the optimiser with an initial guess control sequence $\mathbf{u} = \begin{pmatrix} u_0 & ... & u_{N-1} \end{pmatrix}$ and an initial guess state trajectory $\mathbf{s} = \begin{pmatrix} s_0 & ... & s_{N-1} \end{pmatrix}$,
2. Compute the cost of the trajectory and control sequence, and whether the constraints are satisfied,
3. The optimiser computes a descent direction, takes a step,
4. The optimiser repeats steps 2 to 5 until some stopping criteria are met.

The multiple-shooting method requires solving a significantly larger optimisation problem - $N \cdot (dim(x) + dim(u))$ instead of $N \cdot dim(u)$, as well as having more constraints to satisfy. The benefits are that it handles highly nonlinear systems and unstable systems better, it is less sensitive to the initial conditions, allows us to give a state trajectory as part of the initial conditions, converges faster (than single-shooting), and copes easily with the state and terminal constraints[6].

A major conceptual difference between the two methods is that the state trajectory is always physical in single-shooting, at each iteration, at each step in the optimisation. This means even if the optimisation does not converge, the control sequence at least gives a physical trajectory. The state trajectory in multiple-shooting, on the other hand, may be unphysical for any optimisation step where the equality constraint is not met. The optimisation result, however, will still be physical, and the issue of intermediate states not being physical is minimised by the fact that multiple-shooting is much more likely to converge, and faster.

Part II

Implementation

Chapter 5

Surrogate modelling

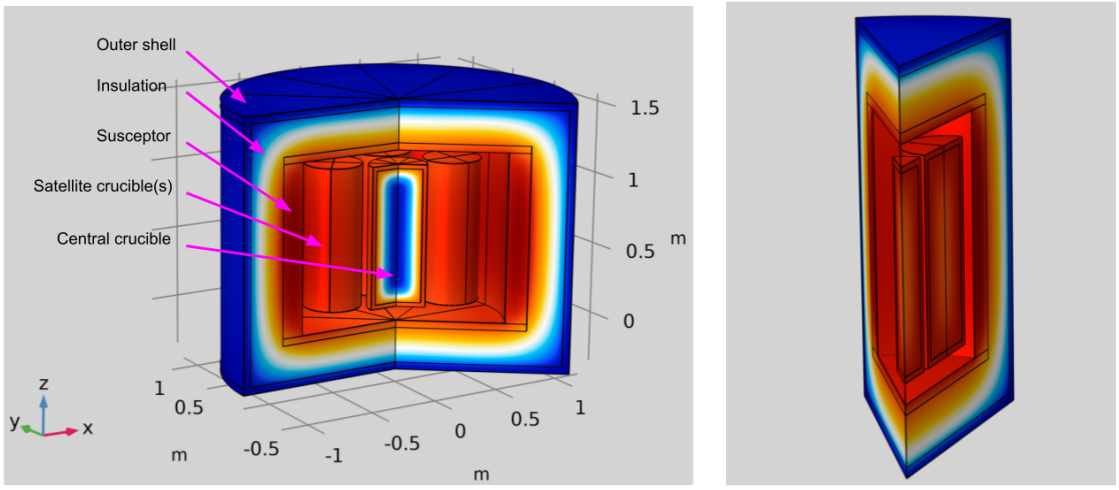
In this chapter we explain the physical system we wish to predict with the surrogate models and detail the construction of the models and well as test their performance. The goal is to build two models: a state-estimator which uses the time-series trajectory of some subset of available sensor measurements and control inputs to predict the full-state (the temperature field), and a forecaster which predicts the next state based on the current state and control inputs. As the purpose of the forecaster model to predict future states online for control, it cannot rely on sensor measurements.

5.1 COMSOL-model of the furnace

The implantation in this thesis is to build a reduced-order model for a graphite furnace for Vianode, an industrial partner of SINTEF. Vianode produces synthetic graphite for battery anodes, primarily for use in electric vehicle batteries, and the overall process promises a 90% reduction in CO_2 emissions. The furnace is proprietary to Vianode, so the exact design of the is a trade secret of Vianode's. A simplified toy model was instead provided for use in this thesis, and any information shared for the purposed of this thesis is considered public.

As can be seen in Figure 5.1, the toy model has a simple geometry with an outer shell, insulation layer, an inner shell called the susceptor, and a central crucible placed coaxially with the furnace and 5 satellite crucibles laid out radially symmetrically ($360/5 = 72°$ apart). Treatment zone is the space enclosed by the susceptor. Heat is supplied through inductive heating of the susceptor, and the outer shell is water-cooled to maintain it at 20°C. The heating power is the primary control input. The cooling energy as it stands is not a control input for the temperature management of the treatment zone, rather it is controlled independently to maintain the temperature of the outer-shell. This will be discussed further in the control design chapters.

NORCE, another partner of Vianode, has created a toy finite-element (FE) model to solve the heat equation inside the furnace in COMSOL Multiphysics which can solve the temperature field in the furnace for a given heating and cooling power profile. The toy model differs from the full model in physics and geometry. The only mode of heat transfer modelled in the toy model is convection, whereas the full model also includes conduction and radiation. The geometry of the toy model is also much simplified, whereas the full model has the true geometry of the real system. To speed up calculations, only the treatment zone is modelled is it is of most interest. Furthermore, utilising the rotational symmetry of the model geometry, only a 32° slice of the treatment zone.



(a) Full furnace model

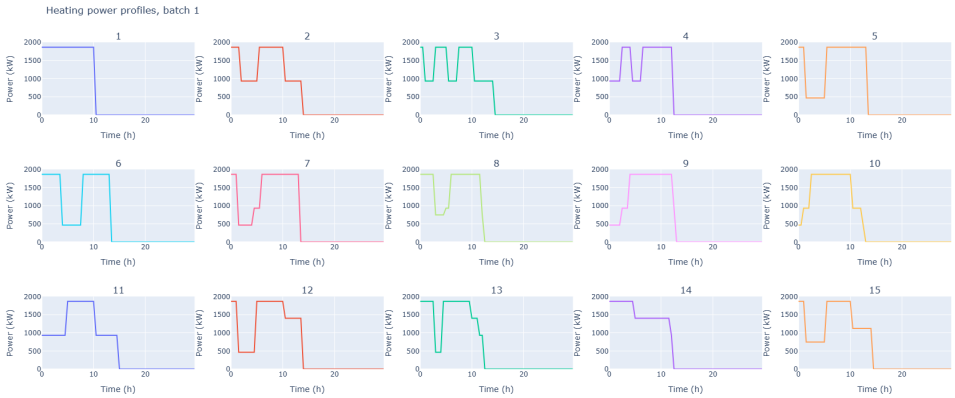
(b) Modelled slice, off-axis view

Figure 5.1: Toy furnace model. (a) shows the full model cut at a plane, (b) shows the slice of the model that was used for simulation. Figures courtesy of Vianode.

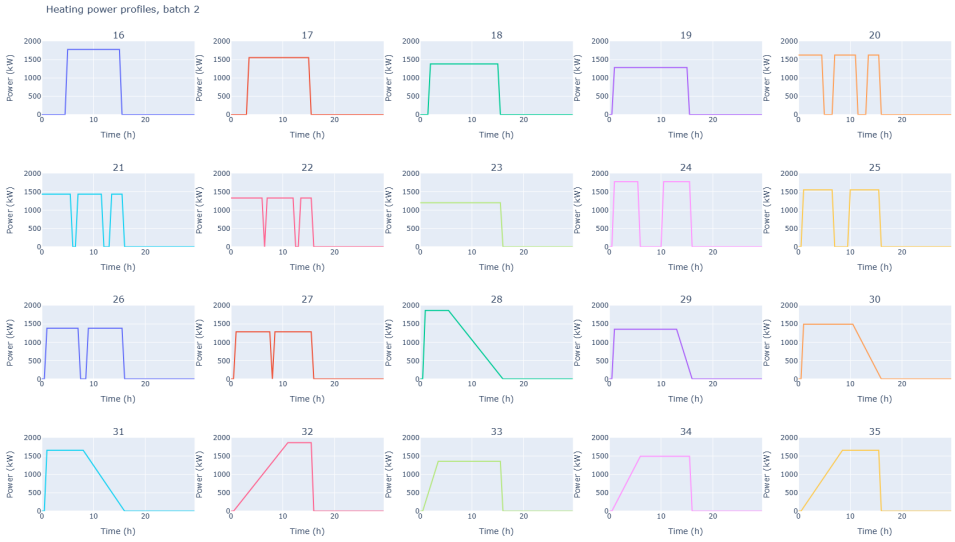
Using the COMSOL model, 35 runs were simulated with different power profiles to capture model data; each run lasted 30 hours with a polling rate of 2/hour (measurements every 30 minutes). To keep this training data close to Vianode's typical use-case, all runs follow a similar pattern: starting from ambient temperatures heating power is applied for the first 15 hours with different profiles (see Figure 5.2) all with the same total input energy; then the power is switched off and the furnace is allowed to cool for 15 hours. The furnace does not reach ambient temperature by the end of the run, hence the 35 runs may not be considered one continuos run. The goal for Vianode is to hold the furnace at a certain temperature after the heating up phase, but this is not represented in the simulation runs. There was no direct access to the COMSOL model, the data was provided directly by Vianode in the form of CSV files Each run resulted in two files: a temperature file which contained the temperature at each node point of the FE-model at each polling time-step, and the coordinates of the node; and a results file, which contained at each time-step the temperature at the pyrometer (pyro), a temperature sensor, modelled in the COMSOL model as one node of the susceptor), the heating power input, the cumulative heating energy, the cumulative cooling energy, and the temperature at some key nodes. Each run respects the maximum heating power of 1860 kW, and a total of 18.6 MWh heat energy is added to the system in each. The first 15 power profiles were designed by Manuel Sparta at Vianode, the rest by the author of the thesis intentionally to perturb the system is varied ways to help capture its dynamics, as any dynamics not bourne out in the data cannot will not be modelled by the surrogate model.

5.2 Preprocessing

Even the simplified COMSOL model has over 20,000 nodes, not all of which are of interest. The aim is to model and control the temperature of the graphite in the crucible, which lies roughly in the geometric centre of the crucibles, meaning it might suffice to only predict the temperature in those nodes. However, to allow the user to ensure that the predictions are physical, the whole of the two crucibles is kept. The nodes on the



(a) Batch 1



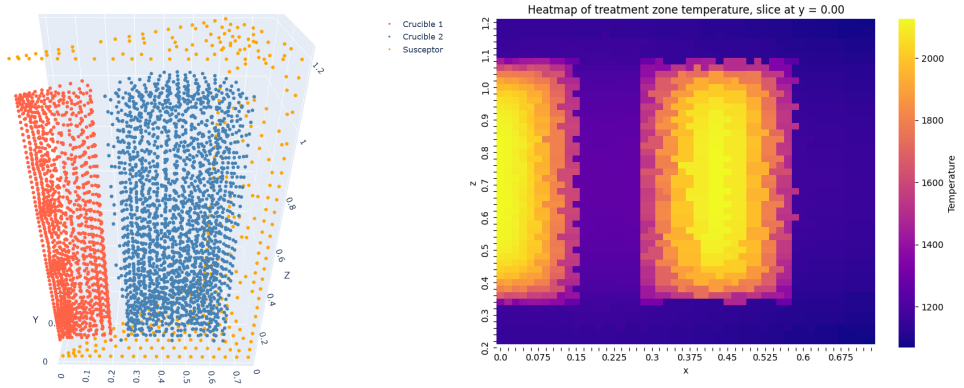
(b) Batch 2

Figure 5.2: Heating power profiles of all simulation runs.

inner wall of the susceptor and well as those on the outer wall of the outer shell will be predicted in our model. The former is a design requirement by Vianode, as keeping the susceptor temperature low is important, and the latter is to allow simple modelling of the cooling power (discussed later). This brings the number of nodes down to $n_Z = 4184$, which is our state dimension moving forward, depicted in Figure 5.3a.

As an example, the temperature at a planar slice of the treatment zone is shown in Figure 5.3b. Since only the nodes shown in Figure 5.3a are included. The temperature in the space between the crucibles and the susceptor wall is approximated with linear interpolation.

The data preprocessing for the LSTM entails some normalisation and recentering, and turning the time-series data into sequences. LSTMs can handle irregular sequence lengths, but it is more convenient in our application to use a fixed sequence length $w = 10$. Each sequence of sensor data is one input, whose corresponding output is the state (or POD state in the case of compressive training) at the timestep of the last element in the sequence. We will cleanly split the runs into training-validation-testing, putting the entirely of each run into one of the three. There was initial consideration on



(a) Nodes selected for modelling

(b) Heatmap of treatment zone temperatures for run 1, at timestep 30 (15 hours)

Figure 5.3: Visualization of simulation data

making sequences from all 35 runs and randomly splitting the sequences. This was tested and showed very good performance and lower overall error compared to splitting the full runs, however there is great danger of data-leakage in this approach. Two subsequent sequences from the same run are naturally very similar, hence training on one and testing on the other does not reflect the true performance of the model.

Before splitting the runs into training, validation and testing data, some modification must be applied. The pyro temperature in raw results data are just the temperature at the node closest to where the pyro would be. In the real system the pyro is unreliable above 2500°C, which we will model as a saturation at 2500°C. This saturation is added to all the runs. The raw results file do not record cooling power, but rather cooling energy. The cooling power is approximated by a numerical differentiation scheme. Furthermore, as cooling power is not a control input, it must be modelled if we wish to use it as an input into the NN models. In the COMSOL models, the cooling power is calculated as a simple heat transfer between the outer wall of the outer-shell and the cooling water (assumed to be at 20°C), by the following equation

$$P_{cooling} = \alpha \dot{m} C (T_{wall} - T_{water}), \tag{5.1}$$

where α is some thermal conductivity coefficient, $\dot{m}$ is the mass rate of the cooling water, C is the specific heat capacity of water. The three coefficients can be simplified to a single coefficient, α_{cool} which can be calculated using curve fitting from the datasets provided.

The 35 runs of the simulation are split randomly into training, validation, and testing data at this stage to avoid any data leakage. A random seed is set so the split is consistent each time the code is run. All preprocessing henceforth is compute from the training data only, and applied to all other sets. The pre-processing pipeline described below is inspired by the `pySHRED` package [20], though all the code is original - partially as a pedagogical exercise and partially for customisation.

To find a POD basis, all the temperature samples at all timesteps across all training runs are compiled into a single matrix which is centred and scaled, and then SVD is calculated. The centring and scaling is done using a `PyTorch` wrapper on the `scikit-learn` `StandardScaler`, we will call this scaler `YscalerPre`. This is necessary to ensure gradient flow as the scaling operations are later used inside our `PyTorch` NN models. The training data informed scaler and POD basis is used to transform all the data. A final `MinMaxScaler`

(also with a `PyTorch` wrapper), which we will call `YscalerPost`, is applies to the data to rescale it between 0 and 1 which makes it slightly easier for the NN models to learn.

The choice sensor measurements into the state-estimator can have a meaningful impact on the performance of the model, this is investigated in the sections to come. All preprocessing of the sensor data is done elementwise such that it can be applied here at this stage and will not be impacted by the choice of sensors or vice versa. Having already applied the saturation, the sensor measurements are scaled with another `MinMaxScaler`, which we will call `Uscaler`. Next the time series trajectories of sensor measurements are made into sliding window subsequences (each subsequence starts from the 2nd element of the previous) of length $w = 10$. Since the different runs are not run continuously (the end of one run does not align with the start of another) making sequences across runs would cause discontinuity. To avoid burn-in time (as well as to maximise the number of training samples) the runs are padded at the start for a number of timestep equal to the length of sequences that will be used. The padding is done is a physics-informed way, such that heating and cooling power is padded with zeros, and cumulative energies and the pyro measurement are padded with their initial values.

5.3 Building, training, testing the models

The entire codebase is made available on GitHub[1]. The raw datafiles are proprietary intellectual property of Vianode and are excluded from the public repository.

5.3.1 Sensors

The possible sensor data to choose from is summarised in Table 5.1. Many configurations will be tested in the following section. Due to their different use cases, how each sensor measurement is found is different for each model. For the purpose of the controller implementation, the state-estimator serves 2 key purposes: learning the latent encoding and a projection to the POD space. The dynamics are handled by the forecaster, meaning online in the control loop, the forecaster takes in some synthetic (predicted) sensor measurements and the control input as well as the current latent state, and predicts the next latent state. The state-estimator's SDN and POD basis are then used to compute the full state, which is used to compute the error. Since the state-estimator's LSTM is never used online, it never has to use synthetic sensor measurements and can use real sensor data (from the results files) in training. The forecaster has to predict future states based on some control sequence, and hence does not have access to real sensor data, meaning it must use synthetic sensor data, so it is also trained only on synthetic sensor data.

5.3.2 Compressive training and custom loss function

As with many physical datasets, most of the information can be captured with just a few POD-modes, as can be seen in Figure 5.4. Training the models to predict the output at each node would cause the models to be very large, leading to slower training and requiring more memory. As discussed in [34], we can take advantage of POD-compression with compressive training - training the model to only predict a truncated POD-state, and using the POD basis to get the full state. We use $r = 10$ POD-modes for the remainder of the report.

[1] https://github.com/abhyupandey21212/Master-Thesis-SINTEF

Sensor	Real data	Synthetic/ estimated	State-estimator configs	State-forecaster configs
Heating power	Results file	Control variable	1, 2	1, 2, 3, 5, 6
Cooling power	Numerical differentiation	Calculated from previous temperature using (5.1)	2	3, 4, 5
Heating energy	Results file	Integral of heating power	1	1, 2
Cooling energy	Results file	Integral of cooling power	1	1, 2
Net energy	Difference between heating and cooling energy	Difference between heating and cooling energy	2	3, 4
Pyro temperature	Results file	Picking pyro node from previous temperature	1, 2	1, 3

Table 5.1: Summary of sensors for state-estimator and -forecaster configurations

The default loss function used in [34] and many such applications in mean square loss. This takes the mean over the squared loss of all the prediction-output pairs in batch, and is equivalent to the square of the Euclidean distance. Notably, it weights the error in each element of a vector equally, which is not completely harmonious with our requirements. All POD modes are not equally important in reconstructing the full-state, indeed the singular values are proportional to their importance. This is normally captured in the POD coefficients themselves, the most important modes have greater magnitude coefficients. However, since we apply a `MinMaxScaler` elementwise, all coefficients becomes normalized between 0 and 1, which removes the natural POD weighting. Hence, a loss function to regain the weighting to use with compressive training is mean square error scaled (elementwise) by the normalised range ($\max_i - \min_i$) of each coefficient from the training data. Let

$$W = diag_{i\in 1,..,r}\left(\max_{X_{train}}(a_i) - \min_{X_{train}}(a_i)\right),$$

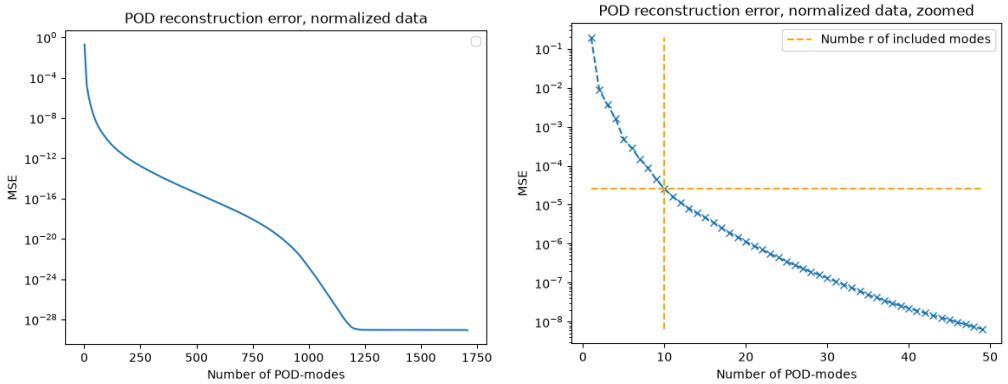
then the weighted loss function is

$$weighted\_loss(a_{pred}, a_{true}) = (a_{pred} - a_{true})^T W (a_{pred} - a_{true}),$$

which is a norm as W is symmetric positive definite.

5.3.3 State-estimator

The state-estimator model uses the SHRED-ROM structure [34], with 2 LSTM layers and 2 SDN layers. The LSTM has $n_x = 64$ hidden dimensions and input dimension equal to the number of sensor inputs, n_u. The SDN layers have the following input-output dimensions: $(n_x, 200), (100, output\_dim)$ where $output\_dim$ is the reduced POD-



(a) Full range of r

(b) Zoomed in

Figure 5.4: Reconstruction error from POD modes as a function of truncation size, r. The orange dashed line shows the error at $r = 10$ is on the order of 10^{-4}. $r = 10$ is used.

dimension r when using compressive training, and the number of (filtered) nodes in the FE model n_Z otherwise.

Two sensor configurations for the state-estimator are tested. Configuration one uses the following sensors: heating power, heating energy, cooling energy, pyro. Configuration 2 uses heating power, cooling power, net energy, and pyro (see Table 5.1). The rationale behind the configurations is that configuration 1 the first obvious way to use the sensors, it simply uses the raw sensor data given by the model. Configuration 2 uses processed sensor data to help the model better learn the dynamics. The change in temperature can be learned from the heating and cooling energy, while the current temperature could be learned from the net energy and pyro temperature. Part of the idea, too, was to avoid using aggregate quantities like total energy to improve extrapolation ability. The total energy input can only rise causing the model to quickly leave the training data range, whereas net energy is more likely to stay within the range (as it can also decrease when cooling power is applied). All other properties are identical across the two, and the same training-validation-testing split is used for both for comparison.

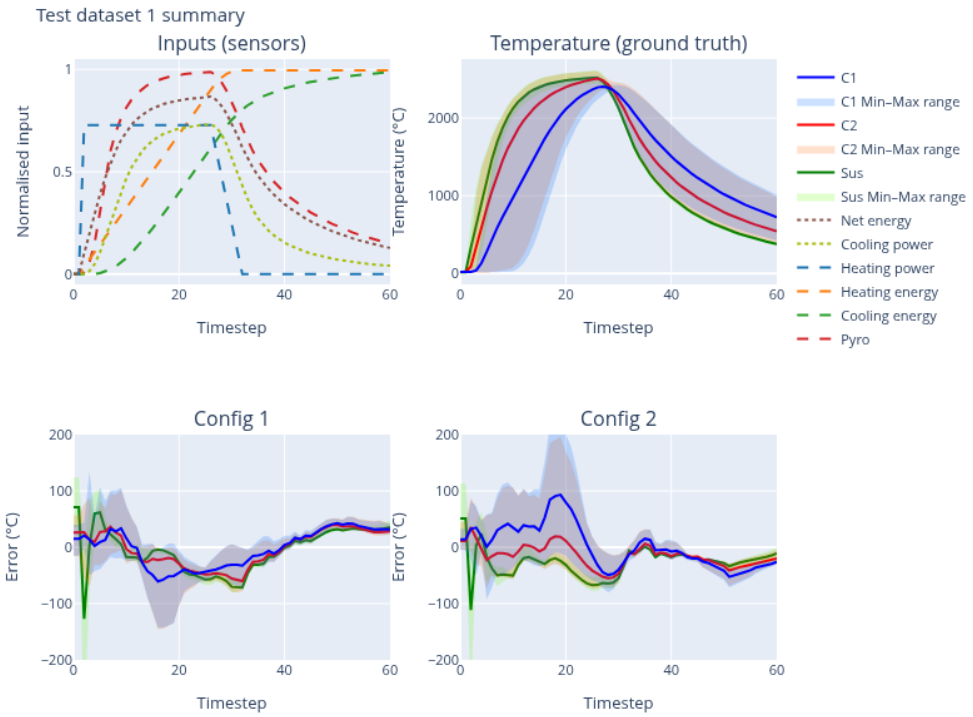


Figure 5.5: State-estimator test dataset 1 results summary. Very simple power curve, configuration 1 shows lower error, although error peaks when power is switched off. The coloured translucent regions represent the min/max range of the temperature across all nodes in a given region. Where these ranges overlap the colours mix.

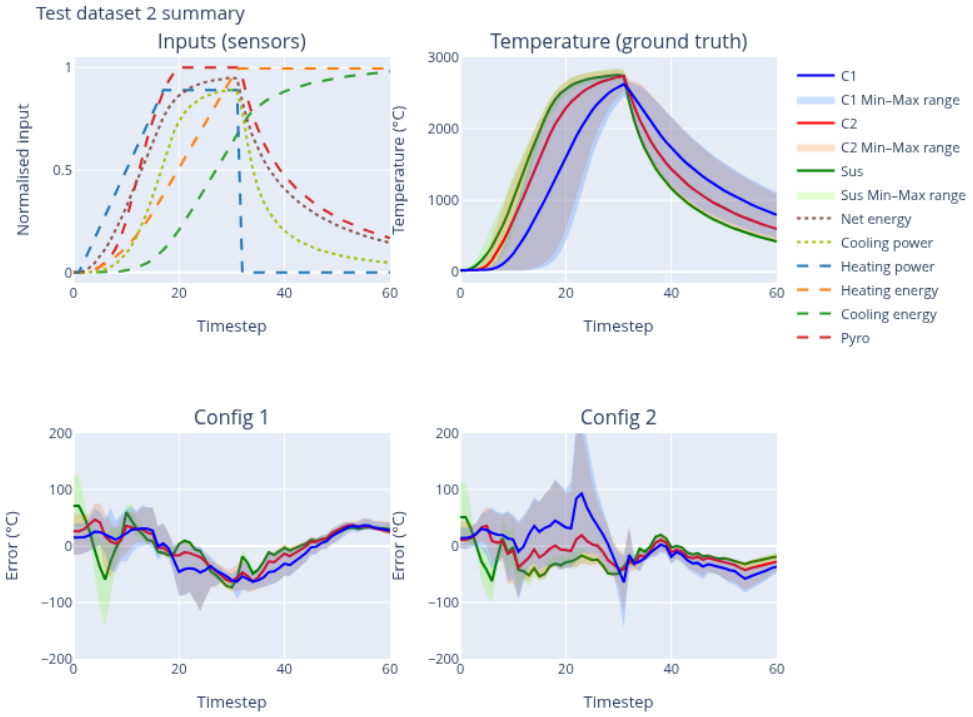


Figure 5.6: State-estimator test dataset 2 results summary.

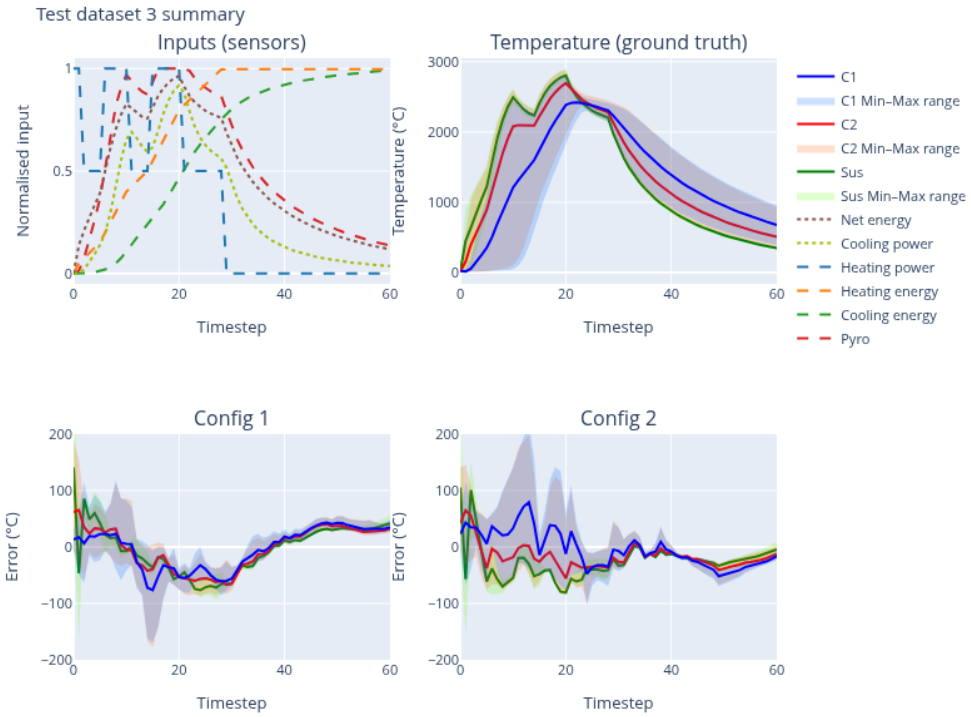


Figure 5.7: State-estimator test dataset 3 results summary. This test set has the most complex power dynamics (step-like function), configuration 1 still performs quite well.

Since all runs follow the same basic pattern (heating power up to a maximum total energy in the first 30 timesteps, then no heating power for another 30 timesteps), the error is naturally highest in the first half of each run. This makes average error over the whole run a less useful metric as it is pulled down by the low error in the very easy to learn dynamics in the second half. Figures 5.5, 5.6, and 5.7 show the error in temperature over the three runs in the testing dataset. The temperature is calculated by undoing the scaling transformations and multiplying by the POD basis. The error is computed per node and grouped based on geometrical regions as defined in Figure 5.3a. The min-max series shows the temperature or error at the node with the maximum and minimum error at each timestep. The top left graph shows the evolution of all sensors (see Table 5.1), and top right shows the temperature output from the COMSOL model. The flow of heat from the susceptor (Sus) to the inner crucible (C1) is apparent is how the temperature curve of C1 responds lower to changes in power, and the min-max spread reflects the flow of heat within the crucibles.

The general tendencies for each configuration carry across all runs, as is reflected in the similar shape of the error curves. Configuration 1 has consistently lower average error and a smaller min-max error spread in the first half of the runs. Configuration 2 seems to perform better in the second half in all three test runs, however the error is low for both configurations in that interval (about 50°C). Configuration 1 curiously shows a consistent tendency for a static over prediction by about 40°C in the latter quarter of the runs. Test run 3 shows the highest error for both configurations, likely because it has the most complex dynamics due to a square-wave-like heating power profile.

While configuration 1 clearly performs better, and will be used in the remainder, another trend emerges which will underscore much of the finding of this thesis: a poverty of stimulus / a lack of persistent excitation. There is simply not enough training data

to capture all the dynamics. There is a lot of data that shows that heating power at low temperatures causes an increase in temperature, and that zero power should decrease the temperature, but there is very little data depicting dropping temperature at low powers or other complicated dynamics. The runs being limited to a fixed total energy and following the same heating-cooling routine further exacerbates the issue, as we will see when analysing the forecaster model.

5.3.4 State-forecaster

The state-forecaster has a similar structure to the state-estimator, just smaller. It is made up of a single layer LSTM followed by a linear projection layer. It takes the current latent state as well as (synthetic) sensor measurements and predicts the next latent state. As state-estimator configuration 1 performed best, the state forecaster will predict the latent state learned by state-estimator configuration 1. Unlike the state-estimator which predicts the POD-state, the state-forecaster predicts the state-estimator latent state (the output of the state-estimator LSTM, before going into the SDN), this is the same structure as used in [34]. All dimensions of the predicted latent state are equally as important, hence the standard mean square error loss is used for training the forecaster model. Five sensor configurations were tested for the forecaster, all other properties were identical. The configurations are explained in Table 5.1. Each configuration was first tested with real data, the results are summarised in Table 5.2. The state-estimator's SDN is used to project from the predicted latent-states to the POD space. The final projection to the full-dimensional state from the POD space is the same as for the state-estimator. Configurations 4 and 5 are the clear worst performers, whereas the rest perform quite similarly. The performance of configurations 1, 2, and 3 will be further analysed in the MPC implementation section.

	Mean absolute error (°C)								
	Core 1			**Core 2**			**Susceptor**		
Config / Test run	1	2	3	1	2	3	1	2	3
1	6.70	4.49	11.3	11.4	8.48	20.6	6.70	4.49	11.3
2	6.05	6.75	11.6	10.4	12.6	21.4	6.05	6.75	11.6
3	5.32	6.47	12.7	7.57	7.61	22.5	5.32	6.47	12.7
4	8.56	9.26	13.2	16.0	17.9	24.5	8.56	9.26	13.2
5	8.61	8.59	14.2	11.9	12.2	23.2	8.61	8.59	14.2

Table 5.2: Summary of test dataset error for forecaster configurations, mean absolute error

Figure 5.8 shows the errors over time in each test set. For space the legend is omitted, refer to Figure 5.5 which has the same colours. As before test set 3 has the highest error due to its complex dynamics (see Figure 5.7. Another clear trend is the peak in error at the highway point, which is when the power is switched off for all runs, meaning the forecaster struggles most when there is a change in the heating power. This theory is supported by the peaks in error in test set 3 corresponding to the drops and jumps in the power curve square wave. All configurations easily learn the cooling down dynamics in the latter half of the runs. Once again configurations 4 and 5 perform particularly worse than the rest.

The results on the test data reveal how well the state-forecaster models capture the dynamics present in the testing data, however they do not elucidate whether the models

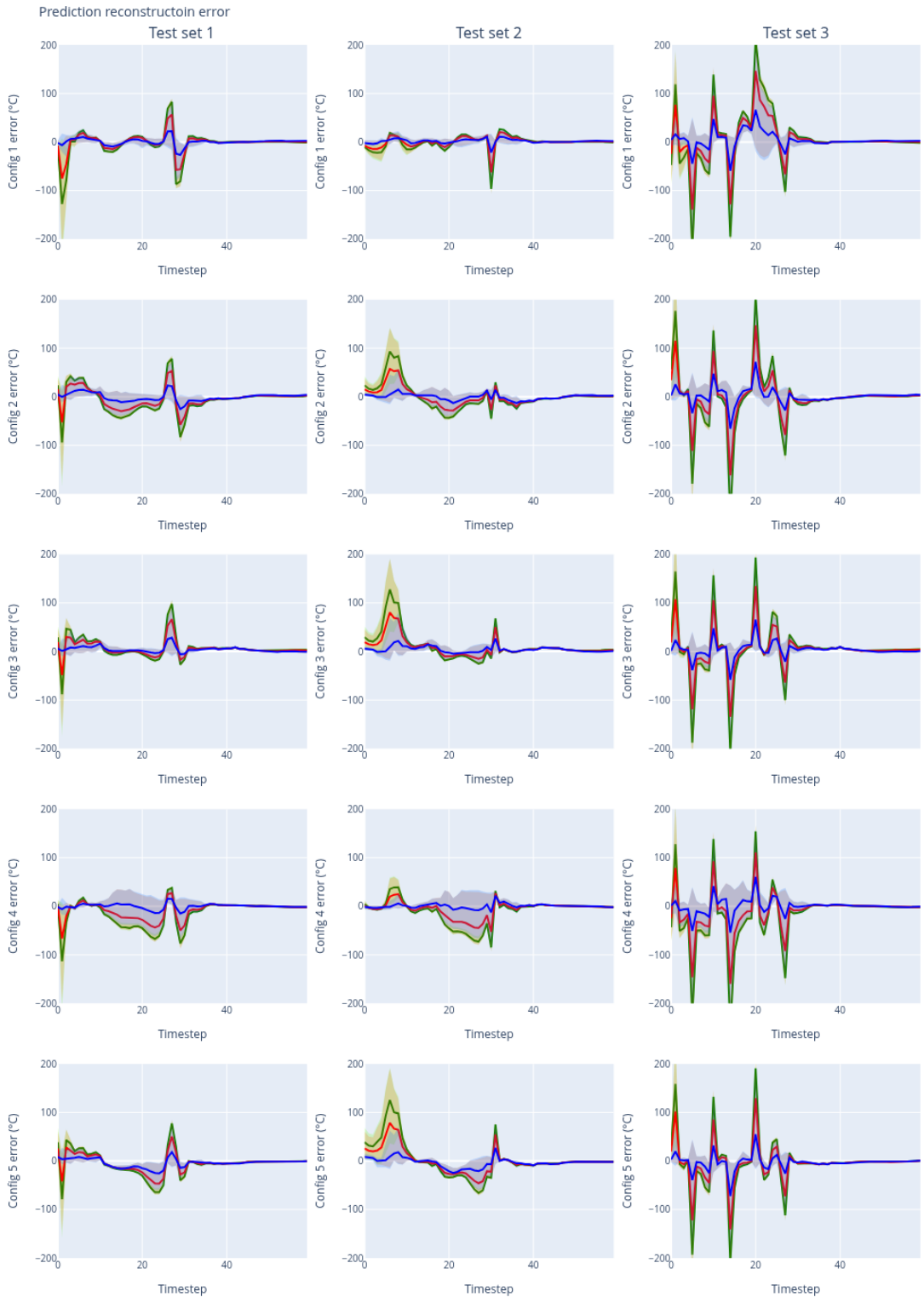


Figure 5.8: Error over time for each state-forecaster configuration. All configurations perform worst on test set 3 as its has most complex dynamics. Configurations 4 and 5 show particularly poor performance.

behave in a physical way. Physically, we should expect that if the power is never turned on, the state should evolve minimally from the initial state (assuming it started near ambient temperature). Similarly if the power is kept at 100% the temperature should continue to climb. Furthermore, we should expect the temperature to never drop below the ambient temperature (around 20°C). We expect that the susceptor should react quickly to changes in net power, whereas the crucibles should react more slowly, given their distance from the susceptor. This was tested by initialising the forecaster models at either the first state of a run in the test set (starting at ambient temperature) or the last state (after cooling back down to near ambient temperature) and running the forecaster for 300 timesteps, with zero input power, 100% input power, and zero power for 125 timesteps, 100% for 50 timesteps, and zero again for 125 timesteps. The results showed that none of the model strictly adhere to these physical principles, which can be attributed to a poverty in the training data. There is no training data depicting equilibrium behaviour, of letting the furnace cool down completely without any input power. In all runs, a fixed total energy is input, so when the input energy exceeds that amount the models begin to behave unphysically (temperature drops while power is at 100%).When any of the forecaster configurations are initialized with the unital state at ambient temperatures, the temperature rises even at zero input power as all the training data depicts only heating from ambient temperature. When instead the models are initialized at the final state of any run (where the temperature is on the order of 100°C and power has been turned off for 30 timesteps), the temperature continues to drop, stabilizing at negative temperatures as low as -1000°C. This highlights that while these models perform quite well on the kinds of dynamics they are trained on, but fail to actually capture the causal relationship between the input and the state trajectory. This phenomenon is called lack of *persistent excitation*, which is the concept that for a system to be identifiable, the input much be sufficiently rich, varying sufficiently independently of the state trajectory. Persistent excitation is defined rigorously in system identification theory [21]. This lack of persistence of excitation leads to *causal confusion* [13], since learning time evolution is easier than learning physics, the model learns time instead, even without an explicit time variable. Since the training and testing data are drawn from very similar distributions, the problem is not apparent with the testing data, however when a differently distributed input is applied the model struggles. The simplest solution would be to capture more data, with careful consideration to persistent excitation to ensure sufficient signal richness. Alternatively, a physics informed neural network (PINN) could be employed, forcing the model to learn parameters of a (simplified) physics model for the system. Both of these fall outside of the scope of this thesis.

Configurations 1 and 2 shows the most unphysical behaviour, and 3 seems to have the least unphysical behaviour. Configuration 3 still shows a rise in temperature at zero input, but the equilibrium temperature is ambient, unlike the other models which settle at either unphysically high (>1000°C) or low (<0°) temperatures. The susceptor nodes are also more reactive to power changes than the crucibles in configuration 3. The most obvious difference is that 1 and 2 both use integrated input energy and cooling energy, whereas configuration 3 takes only power measurements and net energy. Perhaps the inclusion of integrated sensor data leads to the models learning to increase temperature when those values are low and decrease when they are high as that is what is present in all the training and testing data.

The poor modelling of behaviour at ambient temperature with zero input power can be remedied by adding a filter on the output which sets the state back to the initial

state if the initial state corresponds to equilibrium at ambient temperature and the input power is zero. This filter will be applied to state-forecaster configuration 3 moving forward.

Chapter 6

Controller design

In this chapter, we implement a model predictive controller for the furnace using the surrogate models from the previous chapter. As our problem is not convex, we will implement the control law using the suboptimal-MPC framework for optimisation.

6.1 MPC problem statement

In this section we define two problem statements. The first is based on the complete list of desired properties provided by Vianode, and the second a simplified problem statement which will be implemented due to time constraints.

6.1.1 Advanced problem statement

We will implement an MPC controller to, starting from room temperature, apply power to maintain the core temperature in the two crucibles above a threshold, $\bar{T}_c$, while keeping the susceptor temperature below another threshold, $\underline{T}_s$. We also wish to add a cost on the variance of the temperatures of the nodes within the cores. The goal is to hold the crucible above its threshold temperature for a set time period Δt at which point the furnace is turned off. This final requirement is very simple to implement and has no bearing on the controller performance, so it is ignored.

Let x be the SHRED learned latent state of the state-estimator, this is the output of the state-forecaster. The shallow-decoder sdn projects this onto the POD basis. Recall however, that we centred and scaled the target POD outputs in training, so we must unscale to get the predicted POD coefficients. Let $\tilde{a} = sdn(x)$ be the predicted scaled POD state, and let and $a = post_1(\tilde{a})$ be the POD-coefficients. The normalised temperature at each node is then given as the vector $\tilde{y} = post_1(sdn(x))V^T$ where V is the right singular matrix, and the actual temperature is

$$y(x) = post_2(post_1((sdn(x))V^T) = post(sdn(x)),$$

If we wish to penalise the temperature of some node j being below some threshold $\underline{T}_j$, we may use Softplus (which is like ReLU but smooth so that its derivatives are well behaved) with a scaling factor $\beta > 0$ which determines how closely it resembles ReLU,

$$S(x) = \frac{1}{\beta} \ln\left(1 + e^{\beta x}\right),$$

such that $S(\underline{T}_j - y_j)$ is close to zero if $y_j > \underline{T}_j$ and close to $\underline{T}_j - y_j$ if $y_j < \underline{T}_j$. The greater β, the smoother the transition from constant zero to linear. For the suseptor nodes, we

penalise $y_j > \bar{T}_j$ by switching the terms in S. Let T_{des} be the vector of the temperature thresholds corresponding to each node (0 if there is no temperature threshold for that node), and let $e = T_{des} - y$. Let F be a diagonal matrix, the entries corresponding to core nodes are set to 1, those corresponding to suseptor nodes are set to -1, and the rest to 0, such that

$$[Fe]_j = \begin{cases} \underline{T}_j - y_j \text{ if } j \text{ corresponds to a core node,} \\ y_j - \bar{T}_j \text{ if } j \text{ corresponds to a suseptor node,} \\ 0 \text{ else.} \end{cases}$$

The first term of the stage cost is

$$l_1(x, u : T_{des}) = [S(F(T_{des} - post(sdn(x))))]^T \mathbf{Q}_1 [S(F(T_{des} - post(sdn(x))))],$$

where $\mathbf{Q}_1$ is some diagonal positive definite matrix (we used identity) whose entries determine the relative weight of the error at each node. Next, we design the second term of the stage cost l_2 to penalise the variance in the core temperatures. Let y_c a vector of only the core nodes, and let $\mu_c = \frac{1}{n_c} \mathbf{1}_{n_c \times n_c} y_c$ be a vector of the same shape as y_c where each entry is the mean of y_cm denoted $\bar{y}_c$, hence the variance is

$$\begin{aligned} \sigma^2 &= \sum (y_{c,i} - \bar{y}_c)^2 = (y_c - \mu_c)^T (y_c - \mu_c) \\ &= y_c^T (\mathbf{I}_{n_c \times n_c} - \frac{1}{n_c} \mathbf{1}_{n_c \times n_c})^T (\mathbf{I}_{n_c \times n_c} - \frac{1}{n_c} \mathbf{1}_{n_c \times n_c}) y_c \\ &= y_c^T \mathbf{Q}_c y_c. \end{aligned}$$

Since $\mathbf{Q}_c$ is defined by transpose product of a semi-definite matrix, it is symmetric positive definite. This can easily be done for all regions of interesting, and by stacking $\mathbf{Q}_c$ for each regions the variance in each region can be computed as an inner product with some matrix $\mathbf{Q}_2$, such that

$$l_2 = y^T \mathbf{Q}_2 y.$$

Finally, the input cost is captured by a third term l_3 defined with a symmetric positive definite matrix $\mathbf{R}$

$$l_3(x, u) = u^T \mathbf{R} u.$$

The state-forecaster model serves as the state update function, and the state-estimator's shallow-decoder and post-processor are our output function. The input to state-estimator is normalised to between 0 and 1, so the control constraint set is $\mathbb{U} = [0, 1]$. The state is the output of a 2-layer LSTM, and since LSTM outputs pass through a hyperbolic tangent filter, each dimension of the state must be in $[-1, 1]$, hence $\mathbb{X} = [-1, 1]^{n_x}$. Letting f denote the state-forecaster, we can state this formally as the following optimal control problem

$$\min_{\mathbf{x},\mathbf{u}} \left[V_f(x(N)) + \sum_0^N l\left(x(i), u(i)\right) \right]$$

where

$$l(x, u) = l_1(x, u) + l_2(x, u) + l_3(x, u)$$

subject to

$$\begin{aligned} x_k &= f(x_{k-1}, ..., x_{k-w}, u_{k-1}, ..., u_{k-w}) \ \ \forall k \leq N \\ \mathbf{u} &\in \{ \mathbf{u} \mid u(i) \in \mathbb{U} = [0, 1] \ \ \forall i \leq N - 1 \} \end{aligned} \tag{6.1}$$

6.1.2 Simplified problem statement

In the remainder of this thesis, we will implement the MPC controller on one of the two following simplified problem statements:

$$\mathbb{P}_1 : \mathbf{u}_1^0 = \min_{\mathbf{x},\mathbf{u}} \left[\sum_0^N l_1 \left(x(i), u(i) \right) \right] \tag{6.2}$$

$$\mathbb{P}_2 : \mathbf{u}_1^0 = \min_{\mathbf{x},\mathbf{u}} \left[V_f(x(N)) + \sum_0^N (T_{des} - post(sdn(x)))^T \mathbf{Q_1} (T_{des} - post(sdn(x))) \right] \tag{6.3}$$

both subject to (6.1). There is no cost imposed on the control input, and the variance cost is also removed.

In the proceeding sections, we will design a QIH-NMPC controller to solve the optimal control problem in (6.3), and an EMPC-inspired controller for (6.2). Before proceeding with MPC implementation, we first linearise the model to find the terminal cost and constraint for QIH-NMPC, which is vital to our later attempts to find stabilising conditions for the controllers.

6.2 Linearisation and LQR

In the theory chapters, we linearised around the origin when the goal was to drive the system to the origin, stating always that it can be generalised to any other equilibrium point by shifting the origin. The target set-point for the controller has now been defined as the vector T_{des}, which is an output and not a state. The state-to-output function is not invertible due to the decoder-only nature of the SDN, meaning we cannot directly find the corresponding desired state vector x_{des}. Instead, we could look at the testing data and find the state that most closely matches the desired output, and set that as the desired state, we denote it $\tilde{x}$. The problem arises with finding the control input$\tilde{u}$ to linearise around. Since the point around which the model is linearised must be an equilibrium point, $\tilde{u}$ must hold the system at that temperature.

Instead of guessing, we find $\tilde{x}$ and $\tilde{u}$ by using a simpler MPC controller (with no terminal cost or terminal constraint set and hence no stability guarantees) to drive the system to the desired equilibrium T_{des}, from where $\tilde{x}$ and may be extracted.

The Jacobian is computed by first converting the `PyTorch` model into `CasADi` using `Learning4CasADi`, and then using `CasADi`'s gradient function. Since the state-forecaster model is a sequential model, the Jacobian takes the previous w states and inputs and has dimensions $[n_x] \times [w \cdot (n_x + n_u)]$

$$J(f) : \begin{pmatrix} u_{k-1} \\ \vdots \\ u_{k-w} \\ x_{k-1} \\ \vdots \\ x_{k-w} \end{pmatrix} \mapsto \begin{bmatrix} \partial_{u_{k-w}} f_1^T & \cdots & \partial_{u_k} f_1^T & \partial_{x_{k-w}} f_1^T & \cdots & \partial_{x_k} f_1^T \\ \vdots & & \vdots & \vdots & & \vdots \\ \partial_{u_{k-w}} f_{n_x}^T & \cdots & \partial_{u_k} f_{n_x}^T & \partial_{x_{k-w}} f_{n_x}^T & \cdots & \partial_{x_k} f_{n_x}^T, \end{bmatrix}$$

this state and input history can be created by stacking $\tilde{x}$ and $\tilde{u}$. The model is implemented such that the first $w \cdot n_u$ rows correspond to the input dependence, and

the last $w \cdot n_x$ to the state, and the history is in chronological order, meaning rows $w-1 \cdot n_u$ to $w \cdot n_u$ of the Jacobian encode the dependence on the previous input, and $w \cdot n_u + (w-1) \cdot n_x$ to $w \cdot n_u + w \cdot n_x$ encode the dependence on the previous state. We compute the linearised state and input matrices as

$$\begin{aligned}\mathbf{A} &= \partial_{x_k} f(\tilde{x}, \tilde{u}) \equiv J(f)(\tilde{x}, \tilde{u})[:, w \cdot n_u + (w-1) \cdot n_x : w \cdot n_u + w \cdot n_x] \\ \mathbf{B} &= \partial_{u_k} f(\tilde{x}, \tilde{u}) \equiv J(f)(\tilde{x}, \tilde{u})[:, (w-1) \cdot n_u : w \cdot n_u].\end{aligned}$$

Keeping in mind that these are the state matrices for a discrete-time system, we check stability by ensuring the modulus of each eigenvalue of $\mathbf{A}$ is less than 1. Indeed, the linearised system is stable; hence, of course, it is also stabilisable. Furthermore, it is also controllable, as the controllability Grammian is full-rank. We only need stabilisability for the stability of QIH-NMPC.

The discrete LQ-controller, $\mathbf{K}_f$ is computed by solving the discrete algebraic Riccati equation (using the Python control library) for the simplified MPC problem (6.3) with the addition of very small (100 times smaller than $\mathbf{Q}_1$) input cost (as the discrete algebraic Riccati equation is does not guarantee a solution when the input cost is zero). This controller will be used as the terminal control law, and for computing the suboptimal control warm-start.

The optimality of the LQ-controller for the linearised system is theoretically guaranteed, but it is interesting to explore how it performs for the nonlinear state-forecaster. Note that, since the LQ-controller is computed from the linearisation at $(\tilde{x}, \tilde{u})$, that must be treated as the origin for the controller, meaning that to calculate and apply the control on the nonlinear model, the following transformations must be made

$$u_{LQR} = \mathbf{K}_f(x - \tilde{x}) + \tilde{u}. \tag{6.4}$$

As is apparent from Figure 6.1, the LQ-controller is capable of steering the nonlinear model close to the desired set-point and does so without any oscillations; indeed, the closed-loop system exhibits asymptotic stability. However, there is a meaningful static error, and the system is extremely overdamped, with a 95% settling time well above 1000 timesteps (500 hours).

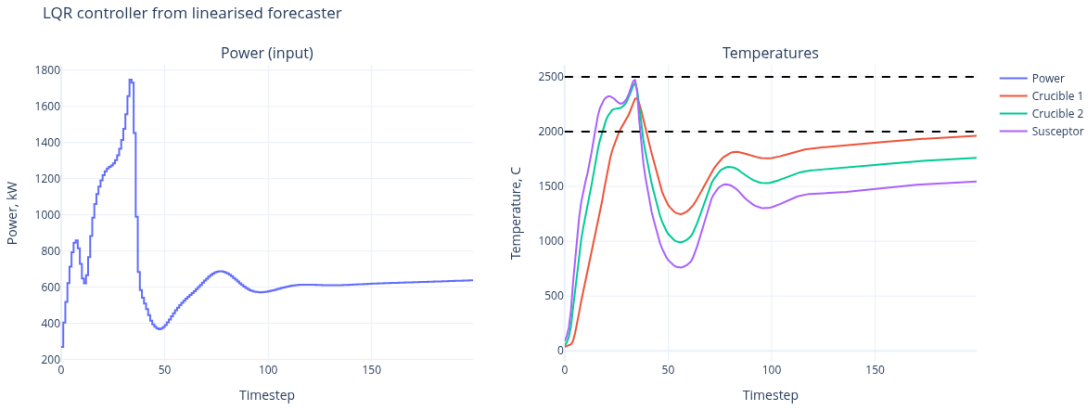
6.3 Stability analysis

The terminal cost and constraint set for the QIH-NMPC controller will be found by the method outlined in Theorem 4.4.2. We have already found the terminal controller $\mathbf{K}_f$ (6.4) by solving the discrete algebraic Riccati equation, the solution P to which defines the terminal cost

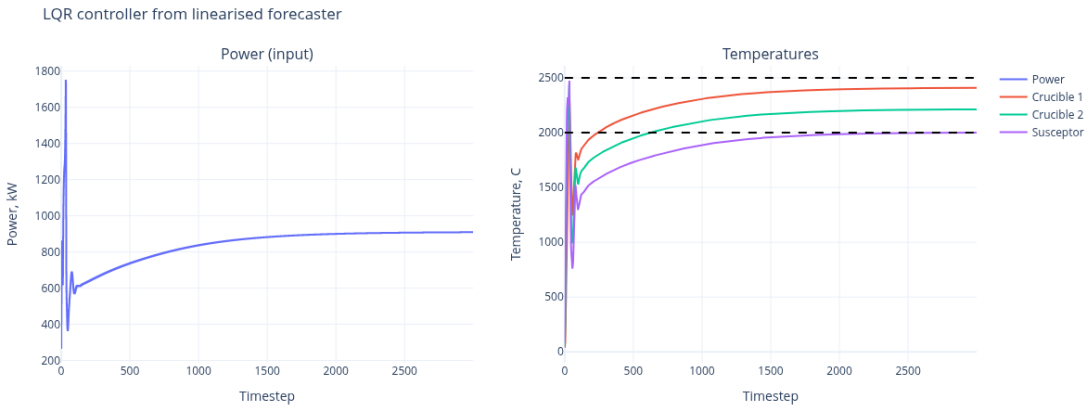
$$V_f : x \mapsto (x - \tilde{x})^T P (x - \tilde{x}).$$

To define a terminal constraint set, we must first find a bound on the second derivative of the linearisation error. Given its definition ((4.1)), as the difference between the LQ-controller closed-loop nonlinear system and the linearised system update equations, the second derivative of the linearisation error is equal to that of the nonlinear system update equation

$$\begin{aligned}\bar{f} &: x \mapsto f(x, \mathbf{K}_f x) \\ e &: x \mapsto \bar{f}(x) - \mathbf{A}_K x \\ &\implies \partial_x^2 e = \partial_x^2 \bar{f}.\end{aligned}$$



(a) First 200 timesteps



(b) 3000 timesteps

Figure 6.1: Closed-loop with LQ-controller from the linearised forecaster model. The lower dashed line is the crucible target, and the upper dashed line is the susceptor target.

The first and second derivatives of the activation function (2.3) are given in (2.4) and 2.5, respectively. The bounds are found numerically using the *differential_evolution* object from *scipy.optimize* which takes as inputs the bounds for the inputs of the activation function,

$$g = 0.2500$$
$$h = 0.7698.$$

We follow the method explained in Section 2.3 to find a bound

$$\|\partial_x^2 f_j(x)\| \leq 1622 \ \ \forall j.$$

This is quite a loose bound, which will prove to be a problem soon. The rest of the variables in (4.5) are found easily from the weights of the torch model. The with $\delta = 0.01$, we get $\alpha = 6 \cdot 10^{-5}$, meaning the terminal constraint set is

$$\mathbb{X}_f = \mathcal{B}(\tilde{x}, 6 \cdot 10^{-5}).$$

This is a very small set and would make optimisation extremely difficult - requiring many iterations, hence time-consuming, if a solution is even possible. In Theorem 4.4.1, the linearisation error is bounded by the derivatives of f, which contains steep nonlinear activation functions like the sigmoid function, meaning the bounds on the derivatives are very large, so the bounds on e may be very loose.

In [11], the bound of the second derivative of the linearisation error is found by simulation. This is not practical for our state-forecaster as it is an NN, however, we could try to bound e directly by simulation. This bound will not be rigorous; however, a tighter bound would lead to a larger terminal set, so it is of interest. Due to time limitations, this avenue was not explored. It is possible to prove stability without a terminal constraint, as shown in Section 4.5, however, that requires a constraint on the initial condition that depends on the terminal constraint, which puts us in a bind again. [12] outlines a way to prove stability without terminal cost or constraints given a sufficiently long prediction horizon. This could be fruitful; however, once again, due to time limitations, it was not explored. In the next section, we implement QIH-NMPC with a terminal cost, but no terminal constraint set. Consequently, we do not have a theoretical guarantee on the stability of the QIH-NMPC controller for optimal control problem (6.3).

Stability for the EMPC controller for (6.3) requires a slightly different approach as the cost function (due to the application of Softplus) is only non-zero when the crucible temperatures are below the threshold (or vice versa with the susceptor). This means it is not a set-point tracking MPC, instead, it attempts to drive the system into a set. This theory is explained in 4.7. Let us begin by showing that $V_f = x^T P x$ corresponding to the terminal LQ-controller gives cost detectability and can be used as W for Theorem 4.7.3. Since P is positive definite, we have

$$c_1 \|x\|_{\mathcal{A}}^2 \leq V_f(x) \leq c_2 \|x\|_{\mathcal{A}}^2, \tag{6.5}$$

where c_1, c_2 are the smallest and largest eigenvalues of P. The distance to the set $\|\cdot\|_{\mathcal{A}}$ is defined in (4.8). The stage cost l_1 is closely related to the distance from the set of desired temperatures (just smoother for differentiability). Importantly, it is an output cost, meaning the set of desired temperatures $\mathcal{Y}$ is not the same as $\mathcal{A}$, in fact, we define

$\mathcal{A} := y^{-1}(\mathcal{Y})$. Let us assume here that the stage-cost is identical to the distance from the set $d_{\mathcal{Y}}(x)$. Therefore, there must exist c_3, c_4 such that

$$c_3\|x\|_{\mathcal{A}}^2 \leq l_1(x) \leq c_4\|x\|_{\mathcal{A}}^2 \tag{6.6}$$

and we let $l_1 = \sigma$ the state-measure, it is positive definite with respect to the attractor. Combining (6.6) and (6.5), we get

$$\frac{c_1}{c_4} l_1(x) \leq V_f(x) \leq \frac{c_2}{c_3} l_1(x)$$

which satisfies (4.10).

Furthermore, since P is the solution to the discrete algebraic Riccati equation, $V_f : x \mapsto x^T P x$ is a Lyapunov function and therefore has the descent property by the LQR-closed-loop on some neighbourhood of $\tilde{x}$

$$V_f(f(x, u_{LQR})) - V_f(x) \leq -x^T \mathbf{Q}_K x.$$

Then showing (4.11) is the same as finding some $c_5 \geq -1$ such that $\forall(x, u)$

$$V_f(f(x, u_{LQR})) - V_f(x) \leq -\epsilon_0 l^*(x) + l_1(x, u) \leq (1 - \epsilon_0) l_1(x, u)$$

$$V_f(f(x, u_{LQR})) - V_f(x) \leq -c_5 l_1(x, u),$$

which holds if $\exists c_5 > -1$ such that

$$x^T \mathbf{Q}_K x \geq c_5 l_1(x, u). \tag{6.7}$$

This means the decrease rate of the terminal cost[1] must dominate the stage-cost up to some factor.

To prove exponential cost controllability, assume there exists some controller κ_e that drives the system exponentially into the attractor,

$$\|x(k)\| \leq \rho^k \|x(0)\|,$$

for some $0 < \rho < 1$. Now use (6.6) to get

$$l_1(x(k)) \leq c_4 \rho^{2k} \|x(0)\|_{\mathcal{A}}^2.$$

Using (6.6) again, we get

$$l_1(x(k)) \leq \frac{c_4}{c_3} \rho^{2k} l_1(x(0)).$$

Take the sum of both sides to the $N-1$th state,

$$V_N^0(x(0)) \leq \sum_0^{N-1} l_1(f(x(k), \kappa_e(x(k)))) \leq \frac{c_4}{c_4} l_1(x(0)) \sum_0^{N-1} \rho^{2k}.$$

We could take the sum to infinity, and since $0 < \rho < 1$, it must be increasing and convergent. So with reference to (4.9), $\hat{\gamma} = \frac{c_4}{c_4} \sum_0^{\infty} \rho^{2k}$, $\gamma_k = \frac{c_4}{c_4} \sum_0^{k} \rho^{2j}$, hence exponential cost controllability is satisfied.

Notice how (6.7) is easier than the set-point tracking QIH-NMPC proof ((4.2)) due to the scaling factor c_5. Then, is it possible to define a subset $\mathbb{X}_{\mathcal{A}} \subset \mathbb{X}$ on which (6.7)

[1] We call it the terminal cost for consistency; however, in Theorem 4.7.3 it is not strictly speaking a terminal cost as it is not part of the cost function.

holds, so that we have local stability on this subset of $\mathbb{X}$. The size of $\mathbb{X}_{\mathcal{A}}$ would depend on the choice of c_5, but that also affects the minimum horizon length

$$N > 1 + \frac{\hat{\gamma}(\gamma_0 + \hat{\gamma})}{c_5^2}.$$

Furthermore, the assumptions made here (like the existence of an exponentially fast controller) are not trivial, and hence this by no means constitutes a proof of the stability of this controller; it merely shows it should be possible to at least show local stability in a neighbourhood of $\mathcal{A}$.

6.4 Controller architecture

As mentioned at the start of the chapter, we are implementing a quasi-infinite horizon model predictive control and an economic model predictive control law. We use the suboptimal MPC algorithm, as a true global optimum is not guaranteed for non-convex problems such as this one, so the optimiser is helped along by the warm-start control sequence. In this section, we explain the controller architecture in detail. Since the QIH-NMPC and EMPC controllers differ only in their cost functions, the algorithms presented in this section are equally relevant for both.

The choice of $\bar{T}_c$, and $\underline{T}_s$ is, of course, important for Vianode's application, but is arbitrary to the controller design. However, it must be chosen carefully for this prototype control due to the limitations of the NN models. As discussed in previous sections, due to the limitations of the training data, the models exhibit some nonphysical behaviours. Furthermore, all training data has a fixed amount of total energy input, meaning the models are unreliable outside this range. This is somewhat overcome by our choice of state-forecaster configuration 3, which uses heating and cooling power and net energy. However, it too is unreliable when any of these values significantly exceeds the training range (normalised between 0 and 1). To help remain within this range, we set the following thresholds

$$\bar{T}_c = 2500°\text{C},$$
$$\underline{T}_s = 2000°\text{C},$$

which are significantly lower than Vianode's targets. The goal is to maintain crucible temperature above $\underline{T}_s$ for $\Delta t = 2$ hours.

6.4.1 Single-shooting controller

The single-shooting controller includes the system dynamics as part of the prediction loop. The state constraint is imposed implicitly through the system dynamics, and since we do not impose a terminal constraint, the only constraint imposed explicitly on the optimisation routine is the control constraint. A more detailed explanation of the difference between single- and multiple-shooting is given in Section 4.8. This is imposed as an inequality constraint

$$\mathbf{u} \in [0, 1]^N.$$

A PyTorch "cost wrapper" computes the cost over the prediction horizon. This wrapper model takes as input the *control input history* and *sensor measurement history* sequences of length $w - 1$ and *state history* sequence of length w required by the LSTM to make state predictions, the *control plan* sequence of length N over the prediction horizon, and the *target state* (the set-point we wish to reach and stabilise at). We used

$w = 10$, $N = 5$. The control and sensor history sequences are of length $w-1$ because the wth state history sequence is the current state, so the first control input in the control plan corresponds to the wth control input. The forward call of the wrapper model runs the prediction loop, which uses the state-forecaster and state-estimator to compute the cost over a given control plan from a given control, sensor, and state history. The prediction loop is explained in the following pseudo-code:

```
def single_shooting_horizon_cost(state_history, control_history,
    sensor_history, control_plan, target_state)
    for j in range(N):
        #Pluckinng out current state and control
        control_now = control_plan[j]
        state_now = state_history[-1]
        #Computing current sensor data
        sensor_now = synthetic_sensor_measurement(state_now)
        #Forecasting next state
        state_next = forecaster(
            state_history,
            [control_history, control_now],
            [sensor_history, sensor_now]
            )
        #Computing stage cost
        cost += stage_cost(state_next, target_state)
        #Rolling forward history
        state_history = [state_history[:-1], state_next]
        control_history = [control_history[:-1], control_now]
        sensor_history = [sensor_history[:-1], sensor_now]
        #Repeat over the prediction horizon N
    #Computing terminal cost on the final state of the prediction horizon
    cost += terminal_cost(state_next)
    return cost
```

Listing 6.1: Single-shooting prediction horizon loop

The computation of synthetic sensor measurements is explained in the previous chapter. Of course, when running the real system, true sensor data from the system (or a COMSOL model) would be fed into the state-forecaster at each step to correct the state predicted by the forecaster. This would avoid the build-up of errors over time, like the update step of a Kalman filter. However, without access to the COMSOL model or the real system, we must rely purely on the forecaster. At this point, what is meant by "sensor measurements" is completely detached from actual sensors. Perhaps it is better understood as secondary inputs into the state-forecaster that are entirely determined from previous control inputs and states. However, we will continue to abuse this terminology to stay consistent with the state-estimator.

The cost function for the QIH-NMPC controller is computed as indicated in (6.3), and by (6.2) for the EMPC controller (the terminal cost is zero for the latter). Using `Learning4CasADi`, this `PyTorch` model object is converted into a `CasADi` function which we will call `cost_l4c`. The following single-shooting suboptimal MPC loop uses IPOPT and `cost_l4c` to find the optimal control inputs for a potentially infinite control horizon. As the future states are computed in the prediction loop, the only optimisation decision variable is the control input.

```
def single_shooting_MPC_loop(state_history, control_history,
    sensor_history, control_plan_warmstart, target_state)
    while True: #Loops infinitely
        #Compute optimal control trajectory steps to minimise the cost
    over the prediction horizon
        control_plan_opt = minimise(
```

```
            objective=cost_l4c,
            guess=control_plan_warmstart,
            parameters = [
                state_history,
                control_history,
                sensor_history
                ],
            inequality_constraints = {0 <= control_plan[j] <= 1},
            equality_constraints = {}
            )
        state_now = state_history[-1]
        sensor_now = synthetic_sensor_measurement(state_now)
        #Apply optimal control trajectory for 1 step
        control_now = control_plan_opt[0]
        state_next = forecaster(
            state_history,
            [control_history, control_now],
            [sensor_history, sensor_now]
            )

        #Update warm start
        state_N = prediction_loop(
            state_history,
            control_history,
            sensor_history,
            control_plan_opt
            )
        control_N = terminal_controller(state_N)
        control_plan_warmstart = [control_plan_opt[1:], control_N]

        #Rolling forward history
        state_history = [state_history[:-1], state_next]
        control_history = [control_history[:-1], control_now]
        sensor_history = [sensor_history[:-1], sensor_now]
        #Repeat
```

Listing 6.2: Single-shooting MPC loop

At each MPC step, the warm start is updated using the current optimal control trajectory and the terminal controller (6.4) according to (4.7) where $\zeta(x, \mathbf{u}) = \bar{\mathbf{u}}_w(x, \mathbf{u})$ as there is no terminal constraint set.

The exact definition of the helper functions (`synthetic_sensor_measurement`, `stage_cost`, `terminal_cost`, etc) is not given here, but can be found in the GitHub repository[2]. The equations used have been detailed in previous sections.

6.4.2 Multiple-shooting controller

As explained in previous chapters, the multiple-shooting controller incorporates the system dynamics as equality constraints, meaning the future control inputs, states, and synthetic sensor measurements are optimisation decision variables. This leads to a meaningfully simpler cost function, at the expense of a more complicated optimisation setup, there are also more decision variables (higher-dimensional optimisation problem), as can be seen in 6.4. It also means the loop can be replaced with matrix operations applied to the entire prediction horizon at once, which is more efficient. In fact, as shown in Code block 6.3, the horizon cost does not explicitly depend on the control plan.

[2]https://github.com/abhyupandey21212/Master-Thesis-SINTEF

```
def multiple_shooting_cost(state_history, control_history, sensor_history
    , state_plan, control_plan, sensor_plan)
    cost = stage_cost(state_plan)
    state_N = state_plan[-1]
    cost += terminal_cost(state_N)
    return cost
```

Listing 6.3: Multiple-shooting horizon cost

```
def single_shooting_MPC_loop(state_history, control_history,
    sensor_history, control_plan_warmstart, target_state)
    while True: #Loops infinitely
        #Compute optimal control and state trajectory to minimise the
    cost over the prediction horizon
        [control_plan_opt, state_plan_opt] = minimise(
            objective=cost_l4c,
            guess=[control_plan_warmstart,state_plan_warmstart]
            parameters = [
                state_history,
                control_history,
                sensor_history
                ],
            inequality_constraints: {0 <= control_plan <= 1}
            equality_constraints: {
                state_plan[j] = forecaster(
                    [state_history[:-w-j], state_plam[:j]],
                    [control_history[:-w-j], control_plam[:j]],
                    [sensor_history[:-w-j], sensor_plam[:j]]
                    ),
                sensor_plan[j] = synthetic_sensor_measurement(
                    state_plan[j-1]
                    )
                }
            )
        control_now = control_plan_opt[0]
        state_now = state_history[-1]
        sensor_now = synthetic_sensor_measurement(state_now)
        #Apply optimal control trajectory for 1 step
        control_now = control_plan_opt[0]
        state_next = forecaster(
            state_history,
            [control_history, control_now],
            [sensor_history, sensor_now]
            )

        #Update warm start
        state_N = state_plan_opt[-1]
        control_N = terminal_controller(state_N)
        control_plan_warmstart = [control_plan_opt[1:], control_N]

        #Rolling forward history
        state_history = [state_history[:-1], state_next]
        control_history = [control_history[:-1], control_now]
        sensor_history = [sensor_history[:-1], sensor_now]
        #Repeat
```

Listing 6.4: Multiple-shooting MPC loop

Multiple-shooting is supposed to perform better [6], and indeed it is the default method implemented in the `do_mpc` Python library. However, the speed advantage of multiple-shooting did not bear out for our application, instead, it took consistently twice

as long as single-shooting. A thorough investigation into the cause is out of the scope of this thesis, however, it seems clear that the sequential nature of the NN models has something to do with it. The most obvious reason is simply the increase in the number of decision variables from $N \cdot 1 = 5$ to $N \cdot (1 + n_x) = 325$. Furthermore, when applying the dynamics as equality constraints, the constraint on each state typically only depends on the previous state. That is not the case with our LSTM models, which require the w previous states. This means the Jacobian of the equality constraint is not nearly as sparse (see Code block 6.5), whereas multiple-shooting relies on sparsity to reduce the problem size [17]. There is another possible concern regarding the `Learning4Casadi` translation layer, as perhaps it is unable to pass on the sparsity of any matrices that might be sparse. These combined seem to result in the loss of possible speed-up from multiple-shooting.

```
1    This is Ipopt version 3.14.11, running with linear solver MUMPS
     5.4.1.
2
3    Number of nonzeros in equality constraint Jacobian...:    42240
4    Number of nonzeros in inequality constraint Jacobian.:        0
5    Number of nonzeros in Lagrangian Hessian.............:        0
```

Listing 6.5: Snippet of IPOPT output for multiple-shooting

Both single-shooting and multiple shooting did provide very similar trajectories, which at least confirms that the algorithms are correctly implemented. All computations were done on the same hardware in the same power utilisation and performance settings (IPOPT running on a single thread, AMD Ryzen AI 7 350 laptop CPU, without GPU acceleration, running Fedora Linux 44 Workstation). Single-shooting (depending on whether or not terminal cost was active) took around 9-12 minutes to compute the optimal control trajectory for 100 timesteps. Multiple-shooting took 25 minutes. Both are feasible, as the timesteps in the COMSOL model were 30 minutes. The real system would likely be run with a higher polling rate. Vianode indicated timesteps of 10 minutes might be used, which is well within the capabilities of this controller. Given this controller runs so much faster than real-time, there is a lot of freedom to increase the horizon N, which would increase the size of the provably stable region $\mathbb{X}_{\mathcal{A}}$ as well as better approximate the infinite-horizon cost[18].

6.5 MPC controller performance

In this section, we discuss the performance of the model predictive controller. In Section 6.2 we mentioned using an MPC controller without terminal cost. This was a single-shooting controller with the architecture as described in 6.4.1 applied to the optimal control problem in 6.3 (quadratic stage-cost). The trajectory produced by this controller, as shown in Figure 6.2, does not exhibit stability - though the oscillators appear bounded, we cannot guarantee even marginal stability. This is not surprising, as we have not applied any terminal cost or constraint that would guarantee stability for this QIH-NMPC implementation, and as we concluded previously, we are unable to find a suitably large terminal constraint set to do so.

We turn now to the EMPC controller for the optimal control problem in 6.2. Due to the ReLu-like Softplus function on the cost, there is a region where the stage-cost is zero (when the crucible temperature is above $\underline{T}_c$ and susceptor temperature is below $\bar{T}$. This is in contrast to the QIH-NMPC controller, where the stage cost is only zero at a single point (the target or set-point T_{des}). We can think of these regions as an attractor.

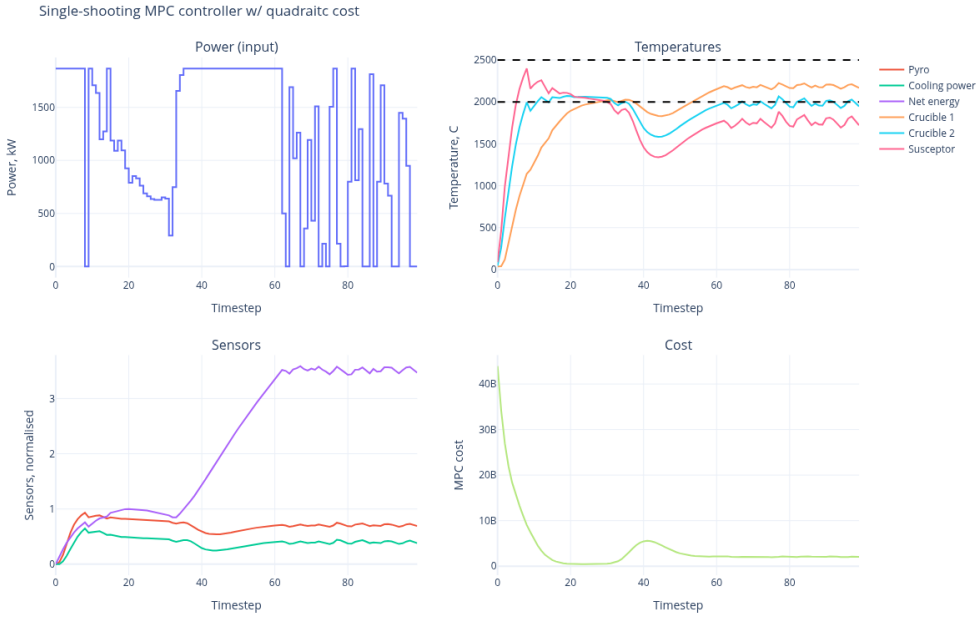


Figure 6.2: QIH-NMPC closed-loop trajectory, 100 timesteps. Dashed lines in the top right graph show the upper threshold for the susceptor and the lower threshold for the crucibles. The output and input show oscillations.

Graphically, the system is at T_{des} when the susceptor is at the top dashed line and the crucibles are at the lower dashed line in Figure 6.2. The attractor set $\mathcal{A}$ is the region between the two dashed lines.

Figure 6.3 shows the control input and temperature trajectories for the EMPC controller (without a terminal cost or constraint), as well as those of the sensors and the MPC horizon cost at each timestep. This controller performs significantly better than the LQ-controller, and it seems to stabilise the system in 20 timestep (10 hours). Unlike the controller with the QIH-NMPC controller, this controller does not produce oscillations. This is consistent with the stability analysis performed in the previous section.

For the sake of curiously, the terminal cost was added to the cost function of the EMPC controller. As shown in Figure 6.5, this reintroduces oscillations. The terminal cost is quadratic, like the stage-cost in (6.3). This suggests that the appearance of stability is due to the "forgiving" stage-cost function in problem statement (6.2) as opposed to (6.3). Anywhere inside $\mathcal{A}$, the stage-cost is zero. Compare this to (6.3) where as soon as the system is not at T_{des} the cost is non-zero. Neither controller has been proven to be stable; indeed, there no guarantee that the state will not exit $\mathcal{A}$ after some longer period of time. However, we can conclude that the forgiving nature of the cost function leads to a more relaxed controller, which does not need to constantly correct the state, which induces oscillations as seen in Figure 6.2.

Since the goal is to hold the crucible temperatures above $\underline{T}$ for $\Delta t = 2$ hours, we could say that this controller is sufficient. Notice, however, that at timestep 30 (the final timestep depicted in Figure 6.3) the normalised net energy reaches 1. Figure 6.4, which shows the trajectory for 100 timesteps (50 hours), shows how, as soon as the net energy falls outside the training range, the state-forecaster model behaves unphysically. As we should expect, due to the power decrease between timesteps 15 and 30, the temperature

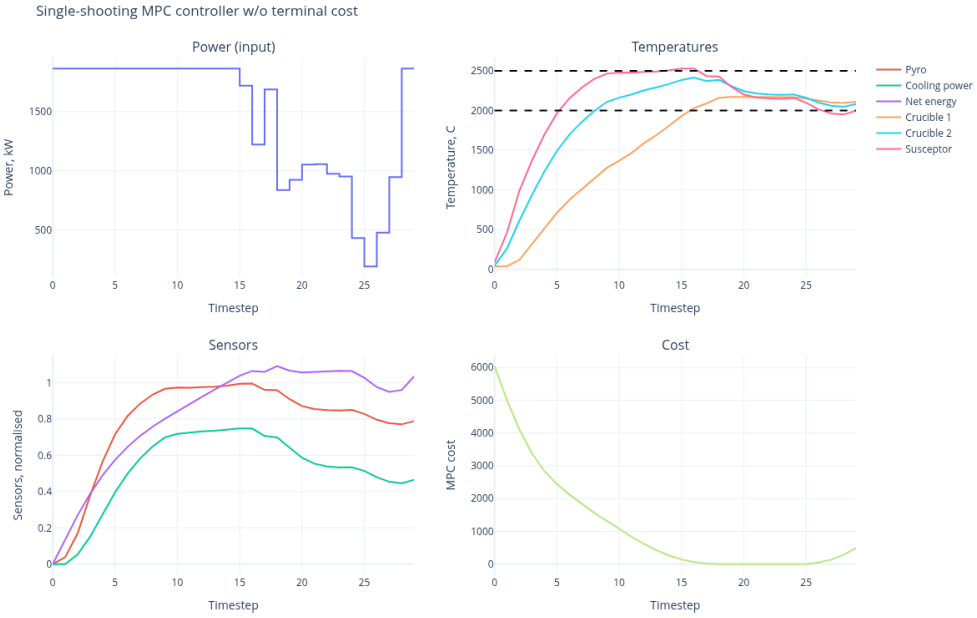


Figure 6.3: EMPC closed-loop trajectory, 30 timesteps. Dashed lines in the top right graph show the upper threshold for the susceptor and the lower threshold for the crucibles. The controller exhibits stability and is able to maintain the temperature within the desired range for over 10 timesteps (5 hours).

decreases, and it decreases faster in the susceptor than in the crucibles. The controller attempts to correct for this, the power is increased to the maximum, and the state-forecaster seemingly does not respond to the power for very long. Furthermore, when the temperature does begin to increase around timestep 40, the susceptor temperature lags behind the core temperature, which is completely unphysical as the susceptor is itself the heat source. We may conclude that the MPC algorithm is implemented correctly, but the state-forecaster model is fragile.

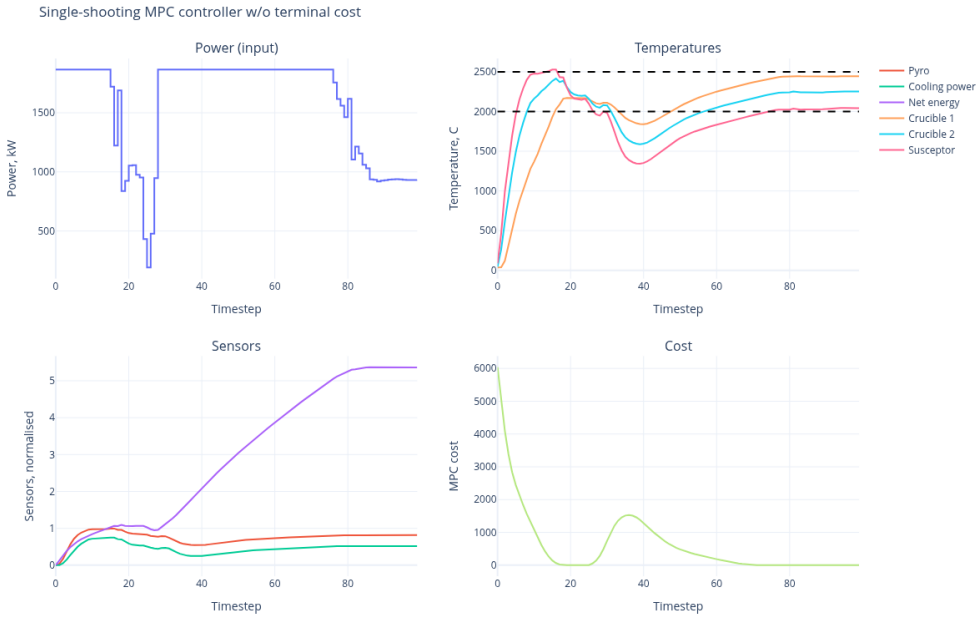


Figure 6.4: EMPC closed-loop trajectory, 100 timesteps. The full 100-timestep trajectory shows the same unphysical behaviour of the forecaster as QIH-NMPC, but no oscillations.

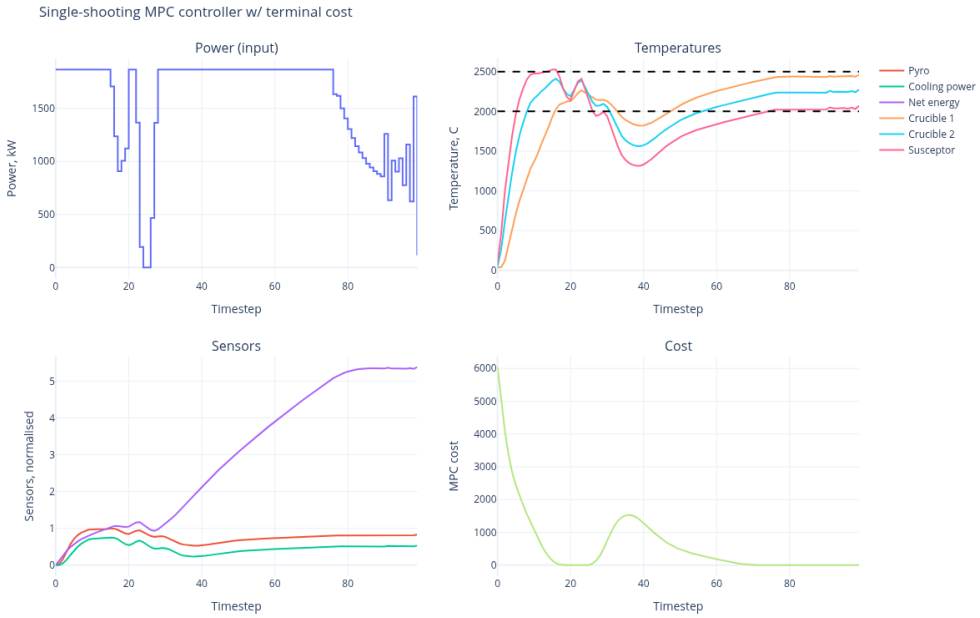


Figure 6.5: EMPC with V_f closed-loop trajectory, 100 timesteps. Adding the terminal cost reintroduces oscillations.

Part III

Conclusion

Chapter 7

Conclusion

7.1 Discussion and further work

The aim of this thesis was to create a machine learning surrogate model following the SHRED-ROM architecture laid out in [34] using compressive training, and to connect it to a model predictive controller. This is a novel combination, and was implemented successfully. However, there were many places were further work is required to make the surrogate model and the controller actually ready for application.

The state-estimator and especially the state-forecaster surrogate models showed good to excellent performance on the testing data, but exhibited egregious unphysical behaviour when sanity-checking the models (for example, initialising at ambient temperature and setting the power to zero still led to a rise in temperature). This suggests *(i)* the testing data was too similar to the training data to catch this; *(ii)* the training data was not varied enough for the model to fully learn the physics, it seems to also have learned the patterns in the data; *(iii)* perhaps another model type like a PINN might be better suited to this application. Attempts were made to collect more varied training data, starting from the original 15 COMSOL runs; 20 additional runs with more complicated and dynamic power profiles were designed to better capture the dynamics. However, the constraints placed on the datasets (constant total energy over all runs, power off after 30 timesteps) limit the variety of the data. The purpose of the surrogate model was to run much faster than the COMSOL model, and indeed inference times for the model are near instantaneous. With more training data, chosen strategically to teach the model the physics, this model could be usable. The new data should include idle dynamics (no power, ambient temperature) and longer runs where the model is allowed to cool down fully to ambient temperature and be heated up multiple times.

There was initial consideration for a type of model called multi-fidelity-POD (MF-POD), which learns to predict a high-fidelity model output from a low-fidelity model. This would require running the low-fidelity model in real-time, which was not possible in this thesis due to lack of access to the COMSOL model. This seems promising and is another avenue for further consideration. Similar to MF-POD, given access to the COMSOL model, one could also gather more useful data by connecting the COMSOL model itself to an MPC scheme and setting the desired trajectory such that all relevant dynamics appear (for example, a very long run with many heating and cooling cycles with varied power profiles in the heating and cooling phases). This would take a long time to compute, but would be part of the offline phase and would make capturing data

easier and one no longer has to guess what inputs give desired output dynamics for training data. This could be implemented and allowed to run over the course of days or weeks, capturing rich training data.

Two MPC controllers were implemented: quasi-infinite horizon nonlinear model predictive control (QIH-NMPC) with set-point tracking and quadratic stage-cost; and economic model predictive control (EMPC) controller with tracking to an attractor set. The sequential nature of the models made connecting to MPC more challenging, but a controller was successfully implemented. This is the novel contribution of this work. The EMPC controller showed good performance and was able to drive the system into the desired output range, whereas the QIH-NMPC controller produced oscillations.

Due to the somewhat opaque nature of the models, bounding the linearisation error proved very challenging, and while a loose bound was found, it was too loose to be useful for proving stability of the MPC controller. These bounds are needed to compute the terminal constraint set, which was computed as a ball of radius 10^{-5} around the set-point, which was too small for the optimisation to find a solution. Stability was not proven for either controller. However, it was shown that it is possible to show local stability for the EMPC controller; further work here could establish stability. The bad performance of the QIH-NMPC controller could be attributed to the fact that it was implemented with no terminal constraint set. Methods for proving stability without a terminal constraint are presented in the literature review; this could be an avenue for future work.

Two numerical optimisation strategies were implemented: single-shooting and multiple-shooting. Multiple-shooting proved, surprisingly, to be the worse-performing of the two, likely due to the sequential state-forecaster, which makes the equality-constraint Jacobian much less sparse (multiple-shooting relies on the sparsity of the equality-constraint Jacobian for its performance gains). The single-shooting controllers run decently fast, computing the optimal control sequence for 100 timesteps in about 9-10 minutes, each timestep taking on the order of seconds. Computing each timestep of the control trajectory means optimising the control input over the prediction horizon and returning only the first input. There is room here to increase the prediction horizon as we only need to compute 1 timestep per actual timestep of the system, and longer prediction horizons make it easier to prove stability [18][12].

Robustness is an important area of analysis for systems with uncertainty (or more accurately errors) like ours, but robustness analysis was outside the scope of this thesis due to time limitations.

Lastly, we detailed in the Theory chapter how, due to the non-convex nature of the optimisation problem, a global optimum is not guaranteed, and discussed how this affects stability. Once stability for the MPC schemes tested has been established, this wrinkle should also be addressed, as those proofs assume the optimal control sequence found at each iteration is the true global optimum.

7.2 Conclusion

SHRED-ROM continues to be promising for this surrogate modelling application for an industrial graphite furnace, but the models created exhibited unphysical behaviour,

which is a problem that must be addressed before it can be considered viable. A few strategies for addressing this problem have been highlighted, most centring around better data collection.

Connecting a sequential model like SHRED-ROM to a model predictive control law proved to be a challenge, though not an insurmountable one, and two controllers were made — one of which showed good performance. Stability proofs eluded us for either controller, the difficulty of which owed in part to the models' sequential nature. An outline for how one could go about proving at least local stability for the EMPC controller was given. On relatively pedestrian hardware, the controller ran reasonably fast; fast enough that a longer prediction horizon for better stability and a tighter bound on the infinite-horizon cost may be considered.

Reduced-order surrogate modelling connected to MPC is shown to be a promising path for optimal control of the furnace; however, other models like MF-POD or PINNs might prove to be better suited.

Bibliography

[1] Joel A E Andersson et al. 'CasADi – A software framework for nonlinear optimization and optimal control'. In: *Mathematical Programming Computation* 11.1 (2019), pp. 1–36. DOI: 10.1007/s12532-018-0139-4.

[2] W.F. Arnold and A.J. Laub. 'Generalized eigenproblem algorithms and software for algebraic Riccati equations'. In: *Proceedings of the IEEE* 72.12 (1984), pp. 1746–1754. DOI: 10.1109/PROC.1984.13083.

[3] Steven L. Brunton and J. Nathan Kutz. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. 2nd ed. Cambridge University Press, 2022.

[4] H. CHEN and F. ALLGöWER. 'A Quasi-Infinite Horizon Nonlinear Model Predictive Control Scheme with Guaranteed StabilityThis paper was not presented at any IFAC meeting. This paper was accepted for publication in revised form by Associate Editor W. Bequette under the direction of Editor Prof. S. Skogestad.' In: *Automatica* 34.10 (1998), pp. 1205–1217. ISSN: 0005-1098. DOI: https://doi.org/10.1016/S0005-1098(98)00073-9. URL: https://www.sciencedirect.com/science/article/pii/S0005109898000739.

[5] Frank E. Curtis, Olaf Schenk and Andreas Wchter. 'An Interior-Point Algorithm for Large-Scale Nonlinear Optimization with Inexact Step Computations'. In: *SIAM Journal on Scientific Computing* 32 (Nov. 2010), pp. 3447–3475. DOI: 10.1137/090747634.

[6] Moritz Diehl et al. *Fast Direct Multiple Shooting Algorithms for Optimal Robot Control*. Jan. 2007.

[7] Wendell H. Fleming and H. Mete Soner. *Controlled Markov Processes and Viscosity Solutions*. 2nd. Vol. 25. Stochastic Modelling and Applied Probability. New York: Springer, 2006. DOI: 10.1007/0-387-31071-1.

[8] M. Frangos et al. 'Surrogate and Reduced-Order Modeling: A Comparison of Approaches for Large-Scale Statistical Inverse Problems'. In: *Large-Scale Inverse Problems and Quantification of Uncertainty*. John Wiley Sons, Ltd, 2010. Chap. 7, pp. 123–149. ISBN: 9780470685853. DOI: https://doi.org/10.1002/9780470685853.ch7. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/9780470685853.ch7. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/9780470685853.ch7.

[9] Alex Graves and Jürgen Schmidhuber. 'Framewise phoneme classification with bidirectional LSTM and other neural network architectures'. In: *Neural Networks* 18.5 (2005). IJCNN 2005, pp. 602–610. ISSN: 0893-6080. DOI: https://doi.org/10.1016/j.neunet.2005.06.042. URL: https://www.sciencedirect.com/science/article/pii/S0893608005001206.

[10] Klaus Greff et al. 'LSTM: A Search Space Odyssey'. In: *IEEE Transactions on Neural Networks and Learning Systems* 28.10 (2017), pp. 2222–2232. DOI: 10.1109/TNNLS.2016.2582924.

[11] Devin W. Griffith, Sachin C. Patwardhan and Lorenz T. Biegler. 'Quasi-Infinite Adaptive Horizon Nonlinear Model Predictive Control'. In: *IFAC-PapersOnLine* 51.18 (2018). 10th IFAC Symposium on Advanced Control of Chemical Processes ADCHEM 2018, pp. 506–511. ISSN: 2405-8963. DOI: https://doi.org/10.1016/j.ifacol.2018.09.374. URL: https://www.sciencedirect.com/science/article/pii/S2405896318320603.

[12] G. Grimm et al. 'Model predictive control: for want of a local control Lyapunov function, all is not lost'. In: *IEEE Transactions on Automatic Control* 50 (June 2005), pp. 546–558. DOI: 10.1109/TAC.2005.847055.

[13] Pim de Haan, Dinesh Jayaraman and Sergey Levine. *Causal Confusion in Imitation Learning*. 2019. arXiv: 1905.11979 [cs.LG]. URL: https://arxiv.org/abs/1905.11979.

[14] Sepp Hochreiter and Jürgen Schmidhuber. 'Long Short-Term Memory'. In: *Neural Computation* 9 (Nov. 1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[15] Matthias Höger and Lars Grüne. 'On the Relation Between Detectability and Strict Dissipativity for Nonlinear Discrete Time Systems'. In: *IEEE Control Systems Letters* 3.2 (2019), pp. 458–462. DOI: 10.1109/LCSYS.2019.2899241.

[16] Sergey Ioffe and Christian Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. 2015. arXiv: 1502.03167 [cs.LG]. URL: https://arxiv.org/abs/1502.03167.

[17] Christian Kirches et al. 'Block-structured quadratic programming for the direct multiple shooting method for optimal control'. In: *Optimization Methods and Software* 26 (June 2013), pp. 239–257. DOI: 10.1080/10556781003623891.

[18] Johannes Köhler, Melanie N. Zeilinger and Lars Grüne. 'Stability and Performance Analysis of NMPC: Detectable Stage Costs and General Terminal Costs'. In: *IEEE Transactions on Automatic Control* 68.10 (2023), pp. 6114–6129. DOI: 10.1109/TAC.2023.3235244.

[19] John F. Kolen and Stefan C. Kremer. 'Gradient Flow in Recurrent Nets: The Difficulty of Learning LongTerm Dependencies'. In: *A Field Guide to Dynamical Recurrent Networks*. 2001, pp. 237–243. DOI: 10.1109/9780470544037.ch14.

[20] J. Nathan Kutz et al. *PySHRED: Package for Shallow Recurrent Decoding*. https://github.com/pyshred-dev/pyshred. 2025.

[21] Eugene Lavretsky and Kevin A. Wise. *Robust and Adaptive Control: With Aerospace Applications*. Springer, 2024.

[22] D. Limon et al. 'On the stability of constrained MPC without terminal constraint'. In: *IEEE Transactions on Automatic Control* 51.5 (2006), pp. 832–836. DOI: 10.1109/TAC.2006.875014.

[23] Massachusetts Institute of Technology. *6.036: Introduction to Machine Learning*. https://openlearninglibrary.mit.edu/courses/course-v1:MITx+6.036+1T2019/course/. MIT Open Learning Library, accessed 2026-08-06. 2019.

[24] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 2e. New York, NY, USA: Springer, 2006.

[25] University of Oslo. URL: https://www.uio.no/english/about/designmanual/profile-in-use/latex/.

[26] Yonggi Park et al. 'Recurrent Neural Networks for Dynamical Systems: Applications to Ordinary Differential Equations, Collective Motion, and Hydrological Modeling'. In: (Feb. 2022). DOI: 10.48550/arXiv.2202.07022.

[27] Chinmay Rajhans, Sachin C. Patwardhan and Harish Pillai. 'Discrete Time Formulation of Quasi Infinite Horizon Nonlinear Model Predictive Control Scheme with Guaranteed Stability'. In: *IFAC-PapersOnLine* 50.1 (2017). 20th IFAC World Congress, pp. 7181–7186. ISSN: 2405-8963. DOI: https://doi.org/10.1016/j.ifacol.2017.08.602. URL: https://www.sciencedirect.com/science/article/pii/S2405896317309771.

[28] J. Rawlings, D.Q. Mayne and Moritz Diehl. *Model Predictive Control: Theory, Computation, and Design*. Jan. 2017.

[29] Tim Salzmann et al. 'Learning for CasADi: Data-driven Models in Numerical Optimization'. In: *Learning for Dynamics and Control Conference (L4DC)*. 2024.

[30] Tim Salzmann et al. 'Real-time Neural-MPC: Deep Learning Model Predictive Control for Quadrotors and Agile Robotic Platforms'. In: *IEEE Robotics and Automation Letters* (2023). DOI: 10.1109/LRA.2023.3246839.

[31] Sina Sharifi and Mahyar Fazlyab. *Provable Bounds on the Hessian of Neural Networks: Derivative-Preserving Reachability Analysis*. 2024. arXiv: 2406.04476 [cs.LG]. URL: https://arxiv.org/abs/2406.04476.

[32] Sahil Singla and Soheil Feizi. 'Second-Order Provable Defenses against Adversarial Attacks'. In: *CoRR* abs/2006.00731 (2020). arXiv: 2006.00731. URL: https://arxiv.org/abs/2006.00731.

[33] Russ Tedrake. *Underactuated Robotics. Algorithms for Walking, Running, Swimming, Flying, and Manipulation*. 2023. URL: https://underactuated.csail.mit.edu.

[34] Matteo Tomasetto et al. 'Reduced order modeling with shallow recurrent decoder networks'. In: *Nature Communications* 16.1 (2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-65126-y. URL: http://dx.doi.org/10.1038/s41467-025-65126-y.

[35] Vianode. URL: https://www.vianode.com/news/vianode-launches-first-recycled-graphite-product-for-more-sustainable-evs-and-batteries.

[36] Stefan Volkwein. 'Model Reduction Using Proper Orthogonal Decomposition'. In: (Jan. 2008).

[37] Andreas Wächter and Lorenz Biegler. 'On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming'. In: *Mathematical Programming* 106 (Apr. 2005), pp. 25–57. DOI: 10.1007/s10107-004-0559-y.

[38] Jan P. Williams, Olivia Zahn and J. Nathan Kutz. 'Sensing with shallow recurrent decoder networks'. In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 480.2298 (Sept. 2024), p. 20240054. ISSN: 1364-5021. DOI: 10.1098/rspa.2024.0054. eprint: https://royalsocietypublishing.org/rspa/article-pdf/doi/10.1098/rspa.2024.0054/513220/rspa.2024.0054.pdf. URL: https://doi.org/10.1098/rspa.2024.0054.